



惠州学院

HUIZHOU UNIVERSITY

毕 业 论 文 （设 计）

中文题目： 基于技术交流群体的智能聊天室

设计与实现

英文题目： Design and Implementation of an

Intelligent Chat Room for Technical

Communication Groups

姓 名 麦旻轩

学 号 2214100540

专业班级 22 软件工程 1 班

指导教师 刘利副教授

提交日期 2026 年 3 月

教务部制

诚信声明

本人郑重声明：所呈交的毕业论文（设计），是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议，除论文（设计）中已经注明引用的内容外，本论文（设计）不含任何其他个人或集体已经发表或撰写过的作品成果。对本论文（设计）的研究做出重要贡献的个人和集体均已在论文（设计）中以明确方式标明。本声明的法律结果由本人承担。

本人签名：

年 月 日

摘 要

面向技术讨论的在线交流，不只是发送普通文本消息，还经常伴随代码片段、报错日志、接口参数、文件附件和阶段性结论。通用聊天工具虽然能够完成基础沟通，但在历史回读、技术内容展示和讨论结果沉淀方面仍然不够贴合开发场景。针对这些问题，设计并实现了一套面向技术交流群体的智能聊天室系统 Mini-Chat-Bar。

系统采用前后端分离架构。前端使用 Vue 3 构建 Web 界面，并通过 Electron 封装桌面端；后端基于 Node.js、Express 和 Socket.IO 实现接口服务与实时通信；MongoDB 用于保存用户、房间、私聊消息、聊天室消息、收藏和 AI 相关数据。围绕技术交流场景，系统实现了注册登录、私聊、聊天室、文件分享、历史回读、收藏管理、AI 问答和讨论总结等功能。AI 模块采用检索增强生成思路，在聊天室问答时结合历史消息索引结果组织上下文，使返回内容尽量贴近当前讨论。在部署方式上，系统也保留了本地私有化部署思路，业务服务与数据库可按 Docker 容器方式部署在个人开发机或团队内网环境中，便于控制消息数据和业务配置的留存位置。

测试结果表明，系统已经跑通主要业务链路。登录、资料读取、消息列表和收藏列表等普通接口保持毫秒级响应；在单机本地环境下，100 个 Socket 客户端广播成功率保持 100%；AI 问答和讨论总结能够正常返回并写入历史记录，但耗时明显高于本地业务接口，主要受外部模型服务影响。整体来看，完成的系统已经能够满足课程项目和毕业设计演示需求。

关键词：智能聊天室；实时通信；AI 助手；Vue 3 框架；RAG 技术

Abstract

Technical-oriented online communication involves not only plain text messages but also code snippets, error logs, API parameters, file attachments, and interim conclusions. General-purpose chat tools can support basic communication, but they are less suitable for developer scenarios that require efficient history review, technical content display, and knowledge retention. To address this gap, this paper designs and implements Mini-Chat-Bar, an intelligent chat system for technical communities.

The system adopts a separated front-end and back-end architecture. The front end uses Vue 3 for web interfaces and Electron for desktop packaging, while the back end uses Node.js, Express, and Socket.IO for API services and real-time communication. MongoDB stores users, chat rooms, private messages, group messages, favorites, and AI-related data. For technical communication scenarios, the system supports user authentication, private messaging, group chat, file sharing, message history review, favorite management, AI-based question answering, and discussion summarization. The AI module uses retrieval-augmented generation to incorporate indexed historical messages into the current context. The system also supports local private deployment through Docker containers on personal development machines or internal networks, which helps control data storage and configuration.

Experimental results show that the system completes the main business workflows. Common interfaces such as login, profile retrieval, message lists, and favorites maintain millisecond-level response times. In single-machine tests, message broadcasting remained stable with 100 Socket clients. AI-based question answering and discussion summarization worked correctly and were recorded in message history, but their latency was significantly higher than local services because they depended on external model services. Overall, the system meets the needs of course projects and graduation project demonstrations.

Key words: Intelligent Chat Room; Real-time Communication; AI Assistant; Vue 3; RAG Technology

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 即时通讯系统与协作平台的发展	1
1.2.2 Web 实时通信技术研究现状	2
1.2.3 AI 辅助交流技术研究现状	2
1.3 研究内容与目标	2
1.4 论文结构安排	3
2 开发技术简介	4
2.1 技术选型与架构概述	4
2.2 前端实现技术	5
2.3 后端服务与实时通信技术	5
2.4 数据存储与消息组织技术	6
2.5 AI 辅助技术	6
3 系统需求分析	8
3.1 需求概述	8
3.2 业务需求分析	8
3.2.1 用户分析	8
3.2.2 业务场景描述	9
3.2.3 业务流程分析	10
3.3 功能需求分析	11
3.3.1 用户管理模块	12
3.3.2 即时通讯模块	12
3.3.3 聊天室模块	13
3.3.4 AI 助手智能问答模块	13
3.3.5 收藏模块	13
3.4 非功能需求分析	14
3.4.1 性能需求	14
3.4.2 安全性需求	14
3.4.3 可用性需求	14

3.4.4 数据分析需求	14
3.4.5 可扩展性与可维护性需求	15
3.5 用户用例分析	15
3.5.1 技术交流人群用例分析	15
3.5.2 个人开发者用例分析	16
3.5.3 普通交流用户用例分析	17
4 系统设计	19
4.1 系统总体设计	19
4.1.1 系统架构设计	19
4.1.2 系统总体流程设计	19
4.1.3 系统功能结构设计	20
4.2 功能模块设计	21
4.2.1 用户管理模块	21
4.2.2 即时通讯模块	22
4.2.3 聊天室模块	23
4.2.4 AI 助手智能问答模块	24
4.2.5 收藏模块	25
4.3 界面设计	26
4.3.1 登录与注册界面	26
4.3.2 主聊天业务界面	27
4.3.3 沉浸式技术聊天室界面	28
4.4 数据库设计	28
4.4.1 E-R 图设计	28
4.4.2 数据库逻辑结构设计	29
4.5 核心底层机制与部署拓扑设计	32
4.5.1 WebSocket 即时通信协作机理	32
4.5.2 系统物理部署与网络拓扑架构	32
5 系统实现	34
5.1 后端实现	34
5.1.1 后端项目结构说明	34
5.1.2 核心模块实现	35
5.1.3 接口设计	40

5.1.4 示例数据	42
5.1.5 通信机制说明	42
5.1.6 展示内容	43
5.2 Web 前端实现	44
5.2.1 登录/注册页面	44
5.2.2 聊天主界面	46
5.2.3 AI 对话界面	48
5.2.4 消息列表与会话管理	49
5.2.5 设置与个人中心	51
5.2.6 Web 端消息流转与状态管理	51
5.3 桌面端实现	52
5.3.1 系统托盘功能	52
5.3.2 本地通知	53
5.3.3 与后端和 WebSocket 通信	54
6 系统测试	55
6.1 测试环境与测试方法	55
6.1.1 测试环境配置	55
6.1.2 测试方法与策略	56
6.2 功能测试	56
6.2.1 功能测试用例设计原则	56
6.2.2 接口与实时通信测试	56
6.2.3 Web 端功能测试	58
6.2.4 桌面端功能测试	63
6.2.5 功能测试结果总结	64
6.3 性能测试	65
6.3.1 响应时间测试	65
6.3.2 并发性能测试	66
6.3.3 资源占用测试	68
6.3.4 性能测试结果总结	70
7 总结与展望	71
7.1 工作总结	71
7.2 实践体会	71

7.3 系统不足与展望	72
参考文献	74
致谢	76
附录	77

1 绪论

1.1 研究背景与意义

面向技术讨论的在线交流，与日常社交聊天并不一样。在课程项目联调、接口排错、版本确认和开源协作等场景中，参与者反复交换的往往是代码片段、报错日志、接口参数、测试结果和临时结论。消息一多，真正有用的内容就容易被后续讨论淹没，后来加入的人也难以及时接上上下文，只能重新追问，沟通成本随之增加。

现有聊天工具能够承担部分交流任务，但在技术讨论场景中仍有明显不足。通用社交软件偏向日常沟通，对技术内容展示、历史回看和结果留存考虑不多；功能更完整的协作平台覆盖面虽广，却往往带来更高的部署和使用成本。因而，围绕讨论过程组织消息、保留关键结论并支持后续回看，已成为技术交流系统的现实需求^[1]。如果系统还能支持个人电脑或团队内网中的本地部署，就能更好控制消息数据和讨论记录的留存位置。技术讨论中还常有一类高频需求：遇到陌生术语、报错信息或代码片段时，希望直接在当前界面完成划词解释，而不是反复切出聊天窗口搜索。

与此同时，大语言模型和检索增强技术的发展，也让聊天系统有了新的扩展方向。AI 的价值不在于脱离上下文单独回答问题，而在于结合当前会话历史继续参与问答，在需要时完成问题梳理、阶段总结和术语解释^[2]。这样一来，用户不必在聊天工具、搜索工具和独立 AI 工具之间来回切换，沟通链路更短，信息也更容易沉淀。因此，设计一套面向技术交流群体的智能聊天室系统，具有较强的实际应用价值和明确的研究意义。

1.2 国内外研究现状

1.2.1 即时通讯系统与协作平台的发展

现有即时通信系统的研究，早已不只停留在“发送一条消息”这一层面。消息留痕、群组组织、文件共享、权限控制和安全通信，已经逐渐成为这类系统的常见能力。针对具体使用场景对聊天系统做定制化裁剪，也是近年来较常见的实现思路。

这对本文的启发比较直接。技术交流群体需要的不是一套笼统的大平台，而是围绕问题讨论本身展开的轻量化工具：消息要清楚，历史要能回看，关键内容要能保留下来，后加入的人也要能尽快看懂前文。

1.2.2 Web 实时通信技术研究现状

在 Web 场景下，实时通信问题已经有比较成熟的处理方式。早期轮询和长轮询能够实现近实时效果，但请求频率较高，服务端压力也更明显。WebSocket 普及之后，浏览器与服务器可以保持持续连接，消息推送、在线状态同步和多人广播都更适合放在这一机制下完成^[3]。

从工程角度看，当前更常见的做法是把普通请求和实时连接分开使用：前者负责登录鉴权、数据读取和消息落库，后者负责在线状态、增量通知和广播同步。这样处理的好处很务实，链路更清楚，排查问题也更直接。本文的系统设计也是建立在这一思路之上。

1.2.3 AI 辅助交流技术研究现状

近年的 AI 研究已经从单纯追问“模型能不能回答问题”，逐渐转向“模型怎样结合外部信息更稳定地回答问题”。已有研究表明，大语言模型在问答、归纳和文本生成方面表现出较强能力，但当输入缺少上下文时，回答仍然容易变得空泛，甚至偏离原问题^[4]。

为了解决这一问题，检索增强生成成为近几年较受关注的方向。它的基本思路不是让模型只依赖参数记忆，而是在生成之前补充与当前问题相关的外部信息，从而提高回答的相关性和可用性^[5]。

针对开发者聊天场景中的阶段总结与多轮追问，已有研究也开始关注摘要方法、对话历史管理以及长上下文利用问题^[6]。

对技术交流场景来说，这一点尤其重要。同一句提问，放在不同聊天室、不同项目阶段里，含义可能完全不同。AI 如果不能接入历史讨论，就很难真正理解当前问题。也正因为如此，本文把 AI 看成聊天链路的一部分，而不是一个脱离场景的独立问答入口。

1.3 研究内容与目标

本文以本人设计并实现的 Mini-Chat-Bar 为对象，围绕技术交流群体的使用特点，完成一套轻量化智能聊天室系统的分析、设计、实现与测试。研究内容主要包括四个方面：一是结合课程项目协作、联调排错和日常技术讨论等场景，梳理目标用户、业务流程与核心需求；二是完成系统总体架构、功能模块和数据组织设计，形成支撑实时通信与 AI 辅助的技术方案；三是实现私聊、聊天室、历史回读、收藏、侧边栏 AI 对话、结合聊天记录上下文问答、讨论总结与划词

解释等功能；四是通过功能测试和性能测试验证系统的可用性与稳定性。

本文的目标不是泛泛讨论“智能聊天”这一概念，而是做出一套能够实际运行的产品原型 Mini-Chat-Bar，并验证它是否适合技术交流场景。该产品既要满足消息收发、历史回看和多人协作等基础交流需要，也要让 AI 以更自然的方式融入讨论过程，例如在侧边栏继续追问、在聊天室里结合历史消息回答问题、对选中的文本做解释，并把有价值的结果保留下来，减少重复沟通。

从价值上看，这套系统可以为课程项目、小型开发团队和技术兴趣社群提供一套门槛相对较低的交流工具，帮助用户更快定位问题、理解术语并回看已有结论。从实现特点上看，本文的侧重点不在于提出新的模型算法，而在于把即时通信、历史消息组织和 AI 辅助放到同一产品链路里：AI 不是独立悬浮在外的问答窗口，而是能够利用聊天记录参与当前讨论，回答结果又能继续被引用、收藏和回看。这种围绕具体使用场景进行整合的做法，也是本课题相对有新意、同时更有现实意义的地方。

1.4 论文结构安排

后文按“需求分析、总体设计、具体实现、测试验证”的顺序展开。

第一章为绪论，交代课题背景，并梳理即时通信、实时通信与 AI 辅助相关的研究现状。

第二章介绍相关技术，说明前端界面组织、后端服务处理、实时通信、数据存储及 AI 检索增强等技术在本项目中的使用方式。

第三章进行系统需求分析，围绕目标用户、典型场景、功能需求与非功能需求，明确系统需要完成和暂不实现的内容。

第四章阐述系统总体设计，包括整体架构、模块划分、数据库设计，以及认证、消息流转、聊天室交互和 AI 辅助等核心流程。

第五章说明系统实现，结合前后端代码与界面，介绍用户、聊天、聊天室、收藏、AI 模块及 Electron 桌面端功能的具体实现。

第六章给出系统测试，从测试环境、功能测试和性能测试三个方面验证运行情况，展示接口、实时通信和 AI 调用链路的测试结果。

第七章为总结与展望，归纳已完成的工作，并说明后续仍可继续改进的方向。

2 开发技术简介

2.1 技术选型与架构概述

本章不再把项目中用到的技术逐个写成教材式介绍，而是围绕 Mini-Chat-Bar 的实际需求说明技术选型的理由。开发过程中，系统最先暴露出的并不是某个单独页面的问题，而是几类能力需要同时成立：既要完成登录、资料读取、消息保存和收藏回看等稳定的数据处理，也要支撑私聊提醒、聊天室广播、在线状态变化和 AI 增量回复这类实时交互，还要兼顾桌面端通知与托盘常驻等本地能力。单靠一种技术很难把这些事情都处理好，因此项目最终采用了前后端分层、实时通信与数据存储协同配合的方案。

如图 2.1所示，系统整体可以分为接入层、前端应用层、服务层以及数据与智能辅助层。

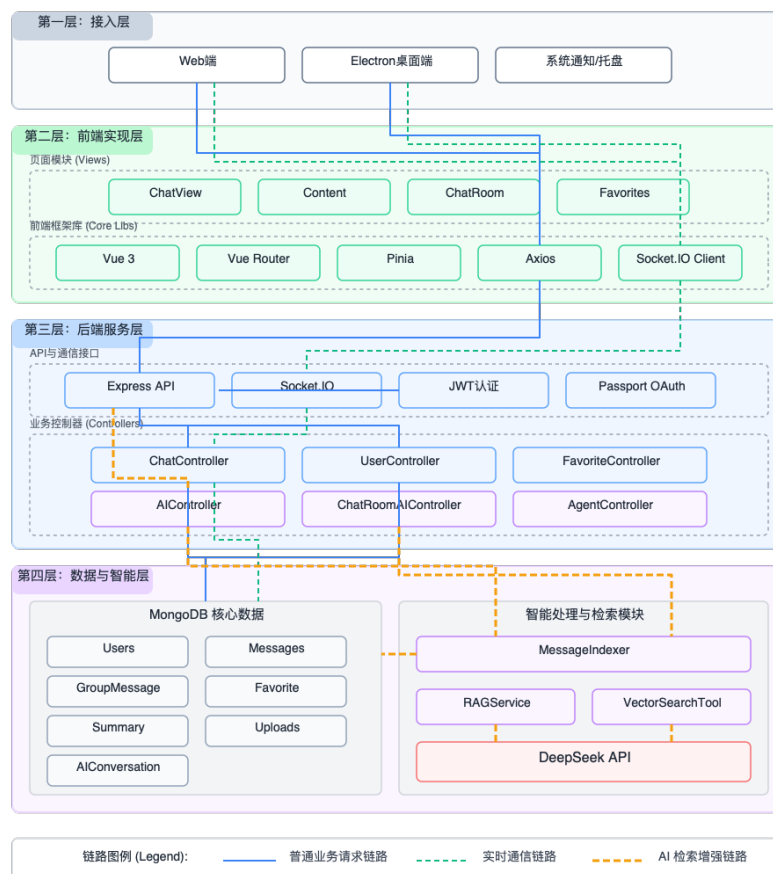


图 2.1 系统技术架构图

这样的划分不是为了把架构图画得复杂，而是因为系统中的不同任务确实

需要不同链路来承担。前端主要负责界面呈现、会话切换和用户交互；服务层负责权限校验、业务处理和消息流转；数据层负责用户资料、聊天记录和收藏内容的持久化；AI 服务层则负责问答、总结和解释等增强能力。这样拆开以后，稳定的数据读写与需要即时反馈的实时交互各走合适的路径，系统结构会更清楚，后续实现和排查问题也更直接。

图 2.1 并不是套用通用模板得到的，而是依据本系统已经完成的功能结构整理出来的。图中把用户管理、即时通信、聊天记录、文件管理和 AI 辅助几个部分放在同一套架构下，说明它们如何共同支撑私聊、聊天室、收藏回看和智能问答等功能。这样的表达方式也为后续章节展开总体设计、功能实现与测试验证提供了统一的基础。

2.2 前端实现技术

前端部分最明显的特点，是同一套界面中同时存在登录鉴权、会话切换、实时消息显示、聊天室详情、收藏回看和 AI 辅助等多种交互。Vue 3 在这个项目里的价值，主要体现在把这些界面拆成职责相对清楚的部分，而不是单纯为了追求新框架^[7]。聊天主界面采用会话列表、消息区和详情区的三栏结构后，私聊、聊天室和收藏等功能就可以围绕各自的区域展开，界面关系也更容易理清。

状态管理由 Pinia 承担，但并没有把所有界面状态都收进全局。跨页面共享的会话状态和消息数据由状态管理统一维护，局部弹窗、搜索词和短时交互则继续放在组件内部处理。这样的划分更贴合本项目的实际需要，也减少了页面联动时的复杂度^[8]。配合 Vue Router 管理页面切换、Vite 支撑开发阶段的快速调试，再结合 Electron 补足托盘、通知和桌面常驻能力，前端部分既能保持 Web 端开发效率，也能满足桌面端展示需求。

Electron 的接入同样是围绕现有页面补能力，而不是另写一套桌面客户端。这样做保留了 Web 界面的复用效率，同时也补上了桌面常驻、消息提醒和窗口切换这些在实际使用和答辩演示中都比较直观的交互能力。

2.3 后端服务与实时通信技术

后端采用 Node.js 和 Express，并不是因为它们 Web 项目里足够常见，而是因为在这个系统从一开始就同时存在两类请求形态：一类是登录、资料读取、消息查询和收藏处理这类可以较快结束的普通请求，另一类是消息发送、在线同步和 AI 流式回复这类会持续交互的链路。把这两类请求放在同一套 JavaScript 运

行环境里处理，有利于减少前后端联调时的切换成本^[9]。

本项目里 HTTP 和 Socket.IO 的分工很清楚，二者并不是互相替代。普通请求主要负责身份认证、数据读写和结果持久化，实时通道主要负责把新消息、在线状态和增量内容及时同步给在线成员。这样处理的好处很实际：稳定数据通过普通接口处理，实时变化通过长连接同步，既便于保证消息可靠落库，也便于后续定位问题^[10]。

身份认证方面，系统采用 JWT 维护登录状态。无论用户采用邮箱密码、验证码还是第三方登录，后续访问都统一进入同一套鉴权流程。这样处理既方便权限控制，也减少了不同登录方式带来的后端分支复杂度^[11]。

2.4 数据存储与消息组织技术

从数据组织看，虽然私聊消息、聊天室消息、收藏快照、AI 回复和讨论总结最终都会显示在聊天界面里，但它们的结构、读取方式和保存重点并不相同。私聊更强调双方关系下的历史回读，聊天室更强调按房间分页加载和多人共享，收藏更强调快照留存，AI 对话和总结则需要保留上下文信息。基于这种差异，系统采用 MongoDB 存放主要业务数据^[12]。

结合实际访问路径，系统把用户、聊天室、消息、收藏和 AI 相关数据分别组织，历史消息重点围绕会话或房间读取，收藏内容则便于后续筛选和回看。文件资源与消息记录分开保存，只在消息中保留必要的展示与定位信息。这样的设计不追求形式上的复杂，而是服务于当前项目里最常见的读取与回看需求。

索引设置同样跟着读取方式走。常用查询集中在最近会话、按时间回读历史消息、按房间查看讨论和按用户查看收藏等场景，因此数据库优化也围绕这些高频路径展开。对本科毕业设计的系统规模来说，这样的组织方式已经足够支撑消息分页、历史检索和收藏沉淀等主要功能。

2.5 AI 辅助技术

本系统中的 AI 不是附加在聊天界面旁边的演示功能，而是直接参与技术讨论过程。侧边栏 AI 助手更适合单人连续追问，也可以用于对选中的术语、报错信息或代码片段做解释；聊天室内的 AI 问答和阶段总结则面向多人协作，回答结果能够继续留在房间里被所有成员查看和引用。

如果只把用户最后输入的一句话直接交给模型，回答很容易脱离前文。为了解决这个问题，项目在聊天室问答链路中加入了消息索引与检索增强。问答发起

前，系统会先从当前房间的历史消息中召回相关讨论，再把这些上下文与用户问题一并提交给模型。这样得到的回答更贴近房间里已经形成的讨论脉络，而不是把每次提问都当作一条孤立的新问题。

考虑到 AI 回复明显慢于普通接口，系统采用流式输出方式逐步返回内容，让提问者和房间成员都能及时看到回答进展。这种做法虽然没有改变模型本身的响应速度，但能明显改善交互体验。

当前实现还保留了一个务实的降级方案：当外部向量或 embedding 服务不可用时，系统仍可退回本地检索方式，先保证开发环境和答辩环境能够跑通整条链路。结合历史消息回读、聊天室总结与 AI 回复持久化功能，AI 在本系统里的角色更接近“利用已有讨论提供辅助”，而不是替代用户完成全部交流。

为了更清楚地说明大语言模型与系统内业务数据的结合方式，图 2.2 给出了本系统 RAG（检索增强生成）的核心流程。

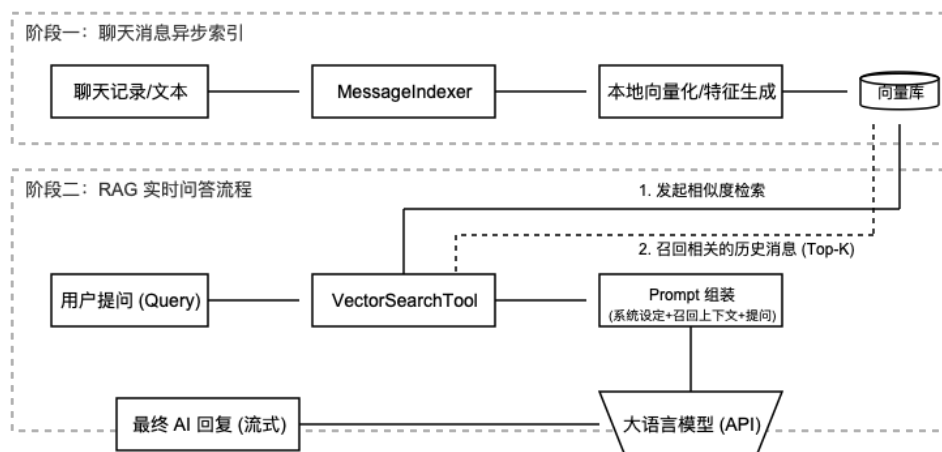


图 2.2 RAG 检索增强生成原理图

如图 2.2 所示，整个智能问答可以概括为“先组织历史上下文，再生成回答”两个环节。用户发送问题后，系统先整理与当前话题相关的历史消息，再将其与当前提问一并提交给模型；模型返回结果后，系统按普通消息或流式消息的形式展示出来。这套设计的重点不在于堆叠更多 AI 术语，而在于让回答尽量贴近当前聊天室已经形成的讨论上下文。

3 系统需求分析

3.1 需求概述

本章围绕 Mini-Chat-Bar 面向的实际使用场景展开需求分析。本文关注的不是一般社交聊天，而是技术讨论中的高频操作，例如登录后进入私聊或聊天室、发送代码和文件、回看历史消息、对关键内容做收藏，以及在无人及时回复时调用 AI 继续处理问题。第 1 章已经说明，这类场景对上下文连续性、消息组织方式和讨论结果沉淀都有更高要求，因此需要把需求拆得更细，不能只停留在“实现聊天功能”这一层。

结合当前系统定位，需求分析主要回答四个问题：系统服务哪些用户；这些用户在什么场景下使用系统；系统至少需要提供哪些功能；为了保证后续实现和演示，系统在性能、安全和可维护性方面应满足哪些基本要求。后文将按照业务需求、功能需求、非功能需求和用户用例四个部分展开。

3.2 业务需求分析

业务需求分析的目标是先把“谁在用、怎么用、为什么用”说清楚。只有用户、场景和流程明确，后续的模块设计和实现细节才有依据。

3.2.1 用户分析

根据本系统的使用目标，用户可以概括为技术交流人群、个人开发者和普通交流用户三类。三类用户都需要基础聊天能力，但关注重点并不相同。

（1）技术交流人群

技术交流人群包括课程项目组成员、实验室小团队、开源协作成员等。这类用户最常见的操作是在会话中发送代码片段、报错日志、接口参数和测试结果，然后围绕同一个问题持续讨论。他们关注的不是“聊得热闹”，而是问题能不能尽快定位、结论能不能保存、后来的人能不能看懂前面的讨论。因此，这类用户对聊天室、历史回读、引用回复、文件共享和 AI 辅助问答的依赖最强，这类面向问题定位的持续讨论方式，与面向任务协作的即时交流系统研究结论较为一致^[13]，使用流程如图 3.1 所示。

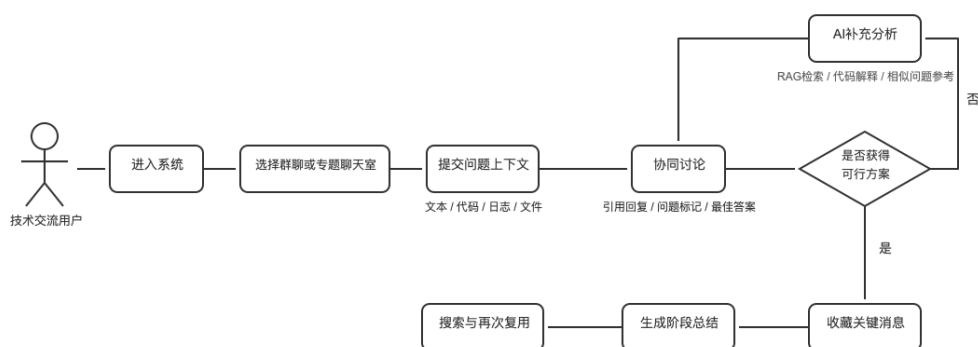


图 3.1 技术交流人群使用流程示意图

（2）个人开发者

个人开发者通常独立完成学习或开发任务，遇到问题时更倾向于先自己查资料、再向系统提问。他们对侧边栏 AI 助手、历史检索和收藏功能的需求更明显：一方面，希望快速得到代码解释、报错分析或实现建议；另一方面，希望把有价值的回答和讨论沉淀下来，形成个人资料库。当单人问答不足以解决问题时，这类用户也会临时创建聊天室，与其他开发者集中讨论。

（3）普通交流用户

普通交流用户可以是测试人员、产品人员或只需要跟进阶段进展的参与者。这类用户并不一定频繁发起技术讨论，但需要较低门槛地进入房间、查看阶段性结论、下载共享文件和回读历史消息。他们更看重界面是否直观、总结是否清楚，以及能否在不参与完整讨论过程的情况下快速了解当前进展。

需要说明的是，本文在本节只为技术交流人群绘制专门的使用流程图，并不意味着另外两类用户不重要，而是因为技术交流人群正是本课题的核心目标对象。这类用户的操作链路最完整，能够把聊天室协同、AI 补充分析、结果收藏与后续复用等关键环节连成一条主线，最能体现系统为何围绕“技术讨论”来设计。相比之下，个人开发者和普通交流用户虽然同样会使用聊天、检索、总结和收藏等能力，但前者更偏向先单人求解再扩展协作，后者更偏向低门槛进入、查看结论和获取资料，其流程更接近对主链路的简化或局部展开。因此，本文保留图 3.1 作为核心用户流程展示，另外两类用户则在后文通过典型场景和用例分析分别说明，避免把结构相近的流程重复绘制。

3.2.2 业务场景描述

结合上述用户特征，可以归纳出本系统中较为典型的三类业务场景。

（1）场景一：多人协同定位接口异常

项目成员在聊天室中集中处理接口异常问题。发起者先发送报错信息、请求参数和相关代码，其他成员在同一房间继续补充判断；如果短时间内没有形成明确结论，可以直接在房间中触发 @AI 进行辅助分析。问题解决后，关键回答和结论还应允许被收藏，方便后续再次查阅。

（2）场景二：个人开发者快速验证实实现思路

个人开发者在实现某个功能时，可能先通过 AI 面板询问框架用法、错误原因或代码修改建议。如果单轮问答不能解决问题，用户可以新建一个与当前主题相关的聊天室，把问题发给其他成员继续讨论。讨论结束后，重要结论可以保存在收藏页中，避免下次遇到同类问题时重复检索。

（3）场景三：普通交流用户整理阶段性讨论结论

测试人员或产品人员未必全程参与讨论，但在查看功能变更、确认交付内容或准备汇报材料时，需要快速掌握前面的讨论结果。此时他们更依赖聊天总结、历史回读和共享文件下载等功能，以减少重新沟通的成本。

3.2.3 业务流程分析

从系统角度看，一次完整的业务流程通常包括登录进入、选择会话、发送或接收消息、触发 AI 辅助、整理关键信息和后续回看几个阶段，如图 3.2 所示。

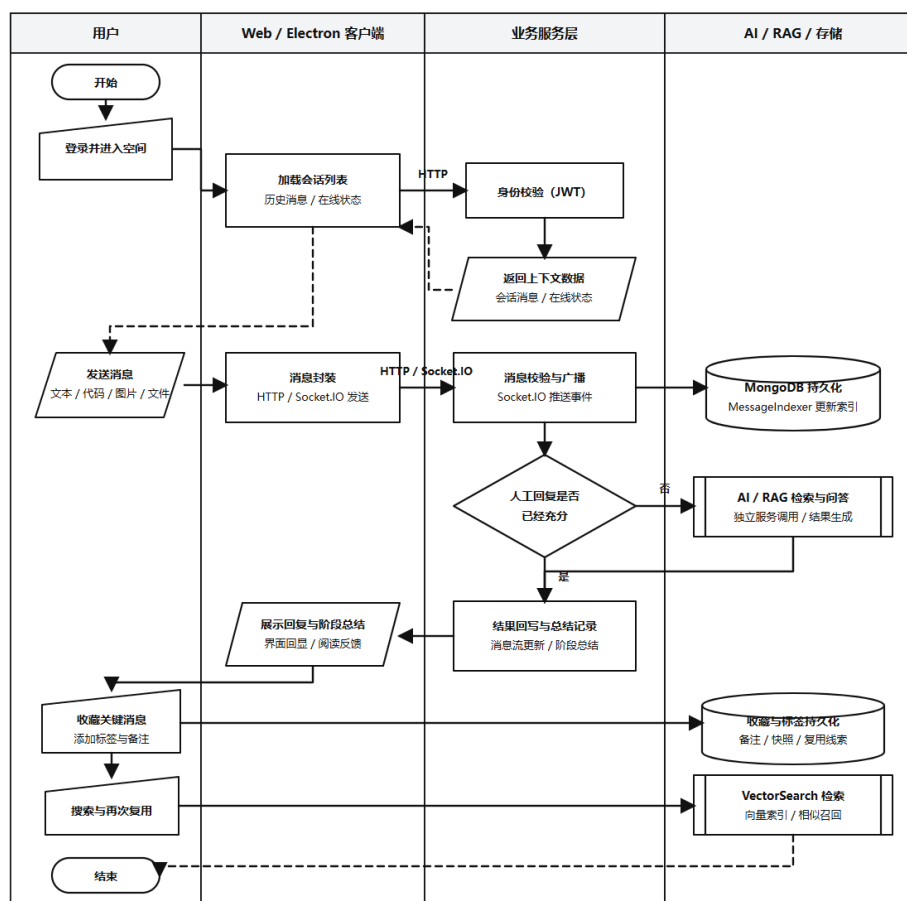


图 3.2 系统业务流程图

用户首先完成注册或登录，进入聊天主界面后选择私聊或聊天室。在交流过程中，系统需要支持文本、代码、图片和文件等多种消息形式；如果讨论涉及持续协作，还需要显示房间成员和在线状态。遇到技术问题时，用户可以在侧边栏 AI 面板单独提问，也可以在聊天室中直接发起 @AI 提问。讨论结束后，重要消息可以加入收藏，聊天室消息和 AI 回复应允许通过历史记录再次读取；对于设置了有效期的聊天室，任务结束后还应转为归档状态，供后续查阅。

也就是说，图 3.1 侧重说明核心目标用户如何围绕技术问题形成完整求解链路，图 3.2 则用于概括三类用户共享的系统级业务主线，二者的表达重点并不重复。

3.3 功能需求分析

根据业务需求，系统功能可归纳为用户管理、即时通讯、聊天室、AI 助手和收藏五个核心模块，具体功能汇总见表 3.1。各模块之间的结构关系将在第 4 章系统设计部分结合功能结构图进一步展开说明。

表 3.1 系统功能需求汇总

功能模块	核心功能点	功能说明	主要服务对象
用户管理模块	注册登录、资料维护、在线状态	完成身份认证、用户资料读取与更新，并为后续会话和权限控制提供基础数据	全部用户
即时通讯模块	私聊、消息发送、历史回读、富媒体支持	承担私聊消息收发与历史回看，支持文本、代码、图片和文件等消息类型	全部用户
聊天室模块	房间创建、加入方式、成员状态、归档管理	提供面向技术讨论的多人协作空间，支持公开或密码加入、在线人数同步和到期归档	技术交流人群、个人开发者
AI 助手模块	单人问答、房间问答、总结生成、上下文检索	在侧边栏或聊天室内提供技术问答、代码解释和讨论总结能力，并结合历史消息提升回答相关性	技术交流人群、个人开发者
收藏模块	收藏、分类、检索、回看来源	将关键消息、代码或结论保存下来，支持后续检索与再次回看	全部用户

3.3.1 用户管理模块

用户管理模块负责系统的身份入口和资料基础。它不仅要处理注册与登录，还要为会话页、聊天室和收藏页提供稳定的用户身份数据。

- (1) 支持邮箱密码登录、邮箱验证码登录以及第三方登录，保证不同用户可以顺利进入系统。
- (2) 登录成功后生成有效会话信息，并在页面刷新或重新进入时保持登录状态。
- (3) 支持读取和修改昵称、头像等基础资料，使聊天列表、消息记录和个人心中的信息保持一致。
- (4) 支持在线状态同步，为联系人列表和聊天室成员列表提供状态依据。

3.3.2 即时通讯模块

即时通讯模块是系统的核心链路，主要负责私聊消息的发送、接收、展示和历史回读。由于本系统面向技术交流，消息类型不能只限于普通文本。

- (1) 支持用户之间进行私聊，并能够及时同步新消息。
- (2) 支持文本、代码片段、图片、文件等多种消息形式，保证技术讨论内容能够被完整表达。
- (3) 支持引用回复、消息状态反馈和历史消息分页读取，便于围绕具体内容继续讨论。
- (4) 支持最近会话、未读消息和消息搜索等基础能力，降低回看和定位信息的成本。

3.3.3 聊天室模块

聊天室模块主要服务于多人协作讨论，适合处理某个专题、某次发布或某个待解决问题。相比私聊，聊天室更强调成员组织、消息广播和阶段性归档。

- (1) 支持创建聊天室，并设置名称、技术方向、加入方式和有效期等基础信息。
- (2) 支持公开加入和密码加入等方式，满足不同讨论范围的需要。
- (3) 支持成员进入房间后的在线人数同步、历史消息回读和新消息广播。
- (4) 支持聊天室到期后转为归档状态，避免长期闲置房间继续产生无效消息。

3.3.4 AI 助手智能问答模块

AI 助手模块负责补充人工讨论中的空档期，帮助用户更快获得初步答案。该模块既要支持单人连续追问，也要支持结合聊天记录的多人的上下文问答，并承担术语解释与阶段总结等辅助任务，这与对话历史管理型问答系统的设计重点相吻合^[14]。

- (1) 支持侧边栏 AI 助手，供用户单独输入问题、代码或报错信息，并对选中的术语、文本或代码片段进行解释。
- (2) 支持在聊天室中通过 @AI 触发问答，使房间成员可以共同查看 AI 返回结果。
- (3) 支持结合历史消息进行检索增强，使 AI 回答尽量贴合当前讨论上下文，减少重复说明前情的成本。
- (4) 支持根据一段时间内的聊天内容生成讨论总结，便于后来加入的成员快速了解前情。

3.3.5 收藏模块

收藏模块用于把聊天过程中的临时信息转化为可回看的长期资料，这也是本系统区别于普通聊天工具的重要部分。

- (1) 支持对私聊消息、聊天室消息和 AI 回复执行收藏操作。
- (2) 支持为收藏内容增加备注或标签，方便后续按主题整理。
- (3) 支持在收藏页中查看已保存内容，并保留来源信息，便于回到原始会话继续阅读。
- (4) 支持基础检索与筛选能力，帮助用户快速定位历史问题和解决方案。

3.4 非功能需求分析

除业务功能外，系统还需要满足基本的性能、安全、可用性和可维护性要求。这些要求虽然不直接对应某个按钮或页面，但会直接影响系统能否稳定运行。

3.4.1 性能需求

本系统主要面向课程项目、小型技术团队和答辩演示场景，因此性能要求以“稳定可用”为主，而不是追求大规模平台级指标。普通业务接口如登录、资料查询、消息列表读取和收藏列表读取应保持较快响应，常见查询在教学演示规模下应控制在秒级以内；私聊和聊天室消息发送后需要及时回显，在线用户应在可感知实时范围内收到更新。对于 AI 功能，由于依赖外部模型服务，其耗时可以明显高于普通接口，但界面需要提供思考状态或流式反馈，避免用户误以为系统失去响应^[15]。

3.4.2 安全性需求

系统需要保证身份认证、访问控制和数据传输的基本安全。用户密码应采用单向散列方式保存，避免明文存储；登录后所有受保护接口都应进行身份校验，并限制未授权用户访问他人资料、私聊记录和受限聊天室。对于聊天室加入、文件上传和 AI 上下文读取，也需要结合当前用户权限做边界检查，防止越权读取或恶意输入影响系统稳定性^[16]。

3.4.3 可用性需求

系统界面应保持清晰直接，避免把高频操作埋得过深。登录、切换会话、发送消息、查看历史、触发 AI 和收藏内容都应在较少步骤内完成。对于技术交流场景来说，代码、文件和普通文本消息需要有明显区分，避免阅读混乱；对于桌面端和 Web 端，还应保证基础操作逻辑尽量一致，减少用户在不同端之间切换时的学习成本。

3.4.4 数据分析需求

虽然本系统不是数据分析平台，但为了后续优化系统运行和用户体验，仍需要保留一些基础统计项。这些数据主要服务于后续观察系统负载、房间使用情况、AI 调用频率和收藏使用情况，不直接面向普通用户展示，核心数据项见表 3.2。

表 3.2 系统数据分析核心数据项

数据类别	关键数据项	采集目的	更新方式
系统活跃数据	在线用户数、连接持续时间	观察系统负载和在线波动情况	实时记录/按日汇总
聊天室使用数据	房间创建数、过期房间数、成员活跃度	判断聊天室功能的实际使用情况	实时记录/按周汇总
消息流量数据	私聊消息量、聊天室消息量、文件消息量	估算消息存储和网络传输压力	实时记录/按日汇总
AI 调用数据	提问次数、总结次数、响应耗时	评估 AI 模块使用频率和调用开销	调用时记录/按日汇总
收藏沉淀数据	收藏总量、标签使用情况、检索频率	判断知识沉淀功能的利用效果	实时记录/按周汇总

3.4.5 可扩展性与可维护性需求

系统应保持清晰的模块边界，便于后续增加新消息类型、替换 AI 服务或扩展桌面端功能。前后端分离结构、接口分层和独立的控制器/路由设计有助于降低修改时的耦合度；同时，系统还应保留必要的日志与错误信息，便于联调、测试和后续维护。

3.5 用户用例分析

在需求分析阶段，用例的作用是从用户视角检验系统是否覆盖了主要操作路径。结合前文的用户分类，下面分别说明三类用户的典型用例。前文的图 3.1 已经用完整流程图突出技术交流人群这一核心目标用户，图 3.2 则概括了三类用户共享的系统业务闭环。

基于这一分工，本节不再为个人开发者和普通交流用户额外重复绘制相近的流程图，而是分别抽取各自最具代表性的用例，进一步说明他们在 AI 求解、历史回看、文件获取和结果沉淀上的使用重点。这样既能把不同用户的差异说明白，也能避免流程图内容在不同用户之间反复重复。

3.5.1 技术交流人群用例分析

技术交流人群的使用路径最完整，通常会同时涉及聊天室、消息回读、AI 提问和收藏等多个功能，用例如图 3.3 所示。

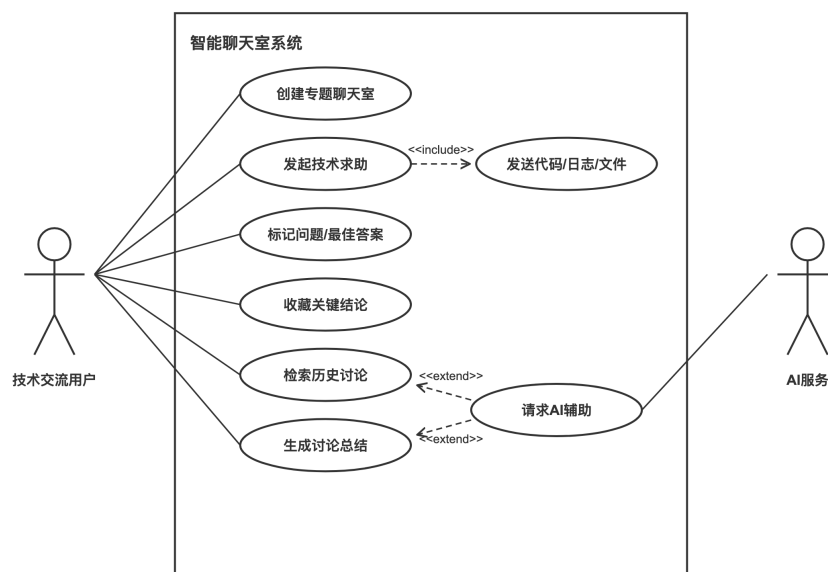


图 3.3 技术交流人群用例图

(1) 用例一：发起技术问题求助

当项目成员在开发或联调过程中遇到问题时，可按以下流程完成求助：

- ① 进入对应聊天室，发送问题描述、报错日志和相关代码片段。
- ② 其他成员在房间内继续补充分析，必要时通过 @AI 获取辅助回答。
- ③ 问题解决后，将关键消息或结论加入收藏，便于后续回看。

(2) 用例二：创建专题聊天室组织集中讨论

当某个任务需要多人集中协作时，可按以下流程操作：

- ① 创建聊天室，设置房间名称、技术方向、加入方式和有效期。
- ② 邀请相关成员加入房间，围绕同一主题集中讨论并共享文件。
- ③ 任务结束后保留历史记录，聊天室在到期后转为归档状态。

3.5.2 个人开发者用例分析

个人开发者的典型操作更偏向“先自己验证，再决定是否扩大讨论范围”，用例如图 3.4 所示。

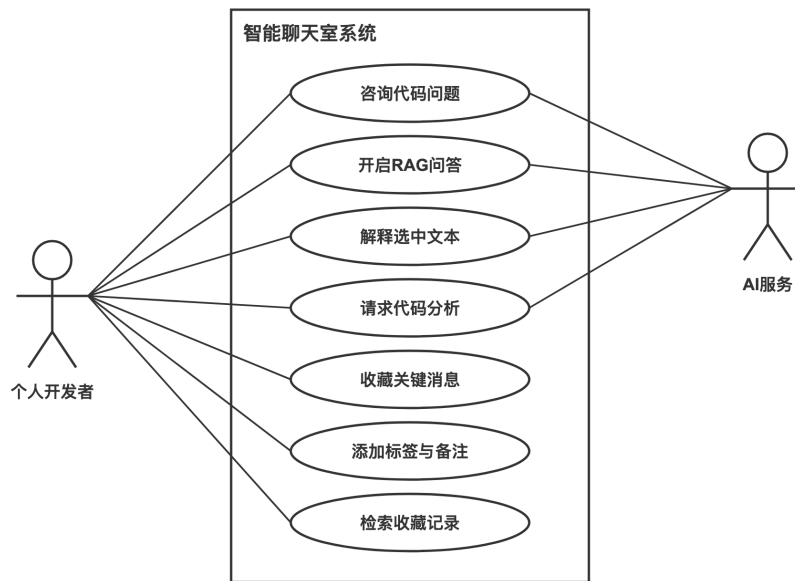


图 3.4 个人开发者用例图

(1) 用例一：向 AI 助手咨询代码问题

当用户独立开发时，可以通过以下流程获取辅助支持：

- ① 在 AI 面板中输入代码片段、报错信息或具体问题；遇到陌生术语或局部文本时，也可以直接触发解释。
- ② 系统返回解释、修改建议或进一步排查方向，连续追问时保留当前 AI 会话上下文。
- ③ 将有参考价值的回答保存到收藏页，方便今后再次查看。

(2) 用例二：收藏并整理关键讨论内容

当用户遇到有长期参考价值的内容时，可按以下流程处理：

- ① 对目标消息执行收藏操作，并补充必要的备注或标签。
- ② 后续在收藏页中按关键词或分类回看对应内容，形成个人资料积累。

3.5.3 普通交流用户用例分析

普通交流用户的操作相对简单，重点在于快速了解讨论进展和获取相关资料，用例如图 3.5 所示。

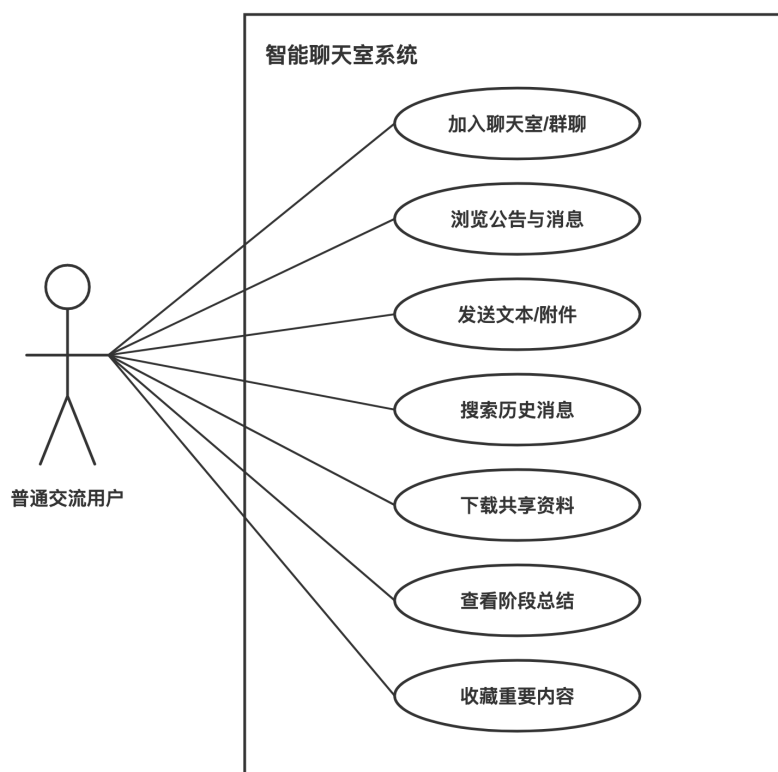


图 3.5 普通交流用户用例图

(1) 用例一：加入聊天室参与日常沟通

普通交流用户参与房间讨论时，可按以下流程进行：

- ① 通过公开入口或密码进入目标聊天室，查看公告和历史消息。
- ② 根据需要补充问题、确认进度或下载房间中共享的资料文件。

(2) 用例二：检索历史消息并获取阶段性结论

当用户需要快速了解某个问题的处理结果时，可按以下流程操作：

- ① 通过历史消息检索或聊天总结功能定位相关讨论内容。
- ② 查看总结结果、打开原始消息或下载关联文件，完成信息回读。

本章从用户、场景、流程、模块和约束几个层面说明了系统需求。可以看出，Mini-Chat-Bar 的核心并不只是“把消息发出去”，而是让技术讨论中的消息、结论和辅助回答能够被组织、回看和继续利用。后续章节将在此基础上进一步展开系统总体设计与具体实现。

4 系统设计

4.1 系统总体设计

系统的设计重点在保证实时通信能力基础上，整合技术问答、消息检索和知识沉淀等增强功能。这要求明确前后端分工，协调实时与常规业务链路，并统一消息记录和模型数据的存储体系。

4.1.1 系统架构设计

Mini-Chat-Bar 采用前后端分离架构。前端基于 Vue 3，结合 Vite、Pinia 和 Vue Router 处理界面渲染与状态管理；后端运行于 Node.js，通过 Express 提供 RESTful 接口，并由 Socket.IO 维持实时双向通信。这样的分层把界面逻辑和业务处理拆开了，后续继续加功能时更容易维护^[7-10]。

前端主要由验证页、主聊天区和收藏浏览页构成，主窗口采用左导航、中列表、右详情的三栏布局，移动端再按小屏重新组织内容。状态由 Pinia 管理，HTTP 请求统一经 Axios 封装；后端则按认证、房间调度、消息收藏和 AI 调度拆分模块，MongoDB 配合 Mongoose 完成数据持久化，AI 问答链路补入文本向量检索以支撑后续语义追踪。整体分层见图 4.1。

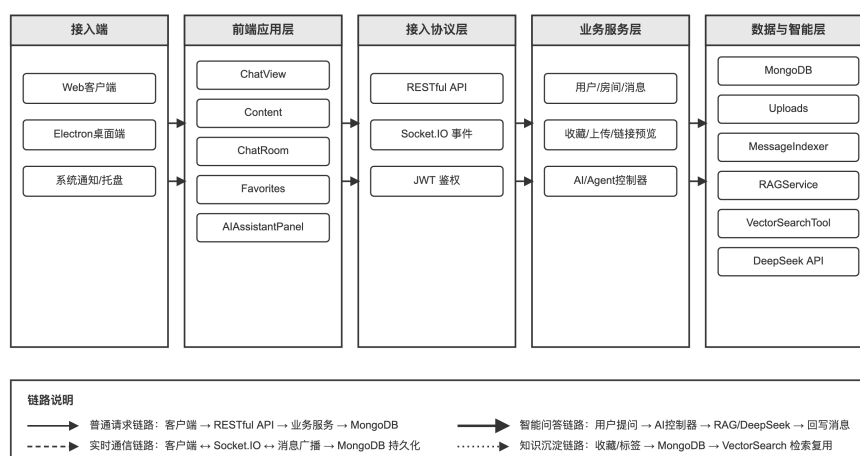


图 4.1 系统架构图

4.1.2 系统总体流程设计

系统主运行流经身份校验、空间分发、讯息吞吐、AI 辅助及知识沉淀五个阶段。用户通过邮箱密码或 OAuth2 准入并获取 JWT 令牌。授权后步入指定会话室内触发请求或实时互动。

日常的文本与各类超媒体文件依托 Express 校验落盘，并在成功后透过 Socket.IO 立发至涉及受众。若为专题室内产生的问题，用户可上溯呼叫 AI 进行

疑难排队，大模型基于语汇执行知识抽提和答案重投。交互结束后，参与者一键锁定摘要或者优质代码集，完成学习循环，如图 4.2。

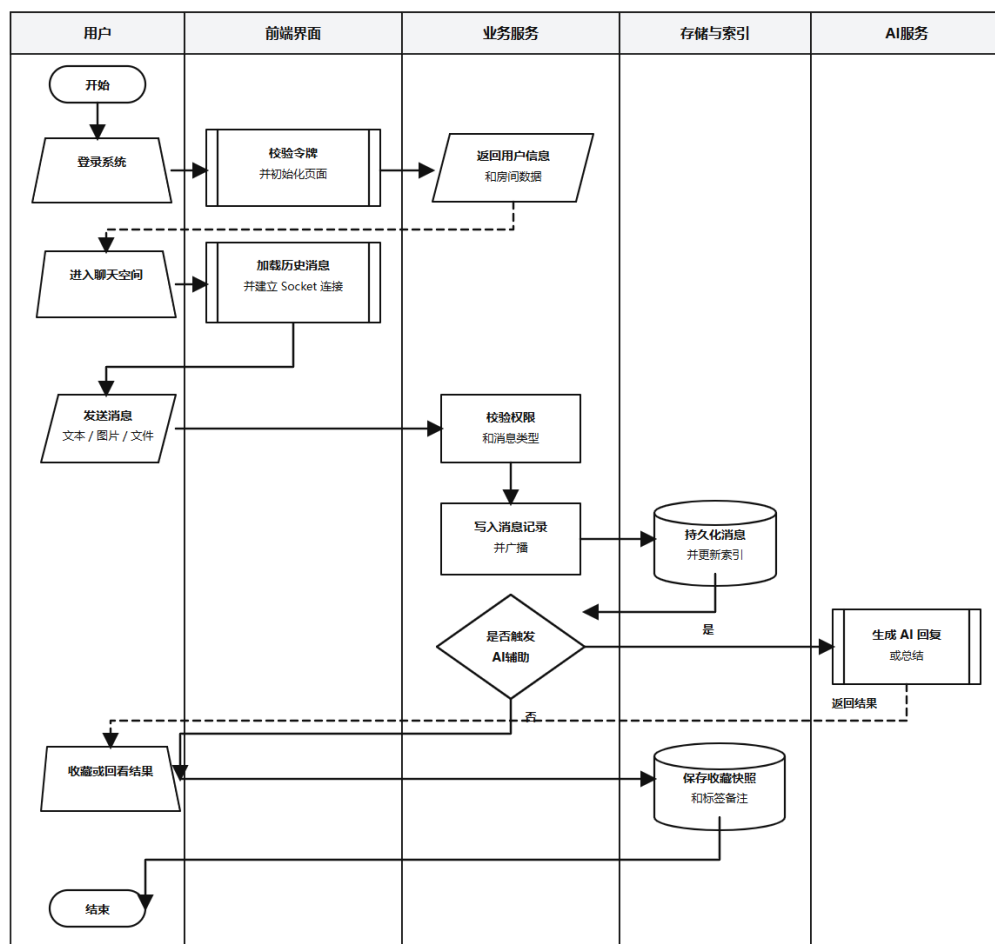


图 4.2 系统总体流程图

4.1.3 系统功能结构设计

依据业务目标，全栈划归五个核心子域：包含用户资料底座的基础管理部、承载常规点对点互动的即时信使部、维系动态短效聚集的技术聊天室群防部、统管 RAG 策略重传的大模型控制核心，以及知识私有化存储的收藏归档仓。

这种网块化设计隔离了各司职能。后续增补语音支持抑或调配更优越的 LLM API 时，不至于连带推翻主体链路（结构规划详参图 4.3）。

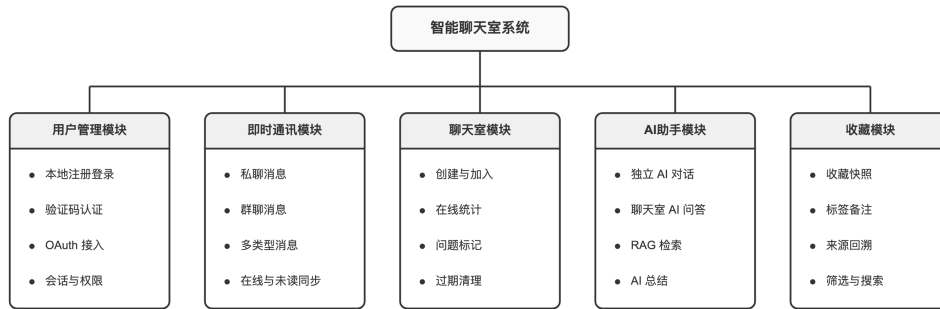


图 4.3 系统功能结构图

4.2 功能模块设计

本节针对前述架构向下细化逻辑边界。

4.2.1 用户管理模块

负责系统入场通道的门禁职能，由原生密文核对与第三方委托链路拼接。常规通路兼顾新户查重与密匙哈希存放，而外围依赖 Passport.js 对接 GitHub/Google 授信返回以直接过桥下发凭证。这种混合手段确保任何群像都能保持一致的安全身份游历全站^[11,17]。在信息闭环上，其衍生的唯一身份标签（ID）渗透至所有附带操作记录中，模块内部运作序列可见图 4.4。

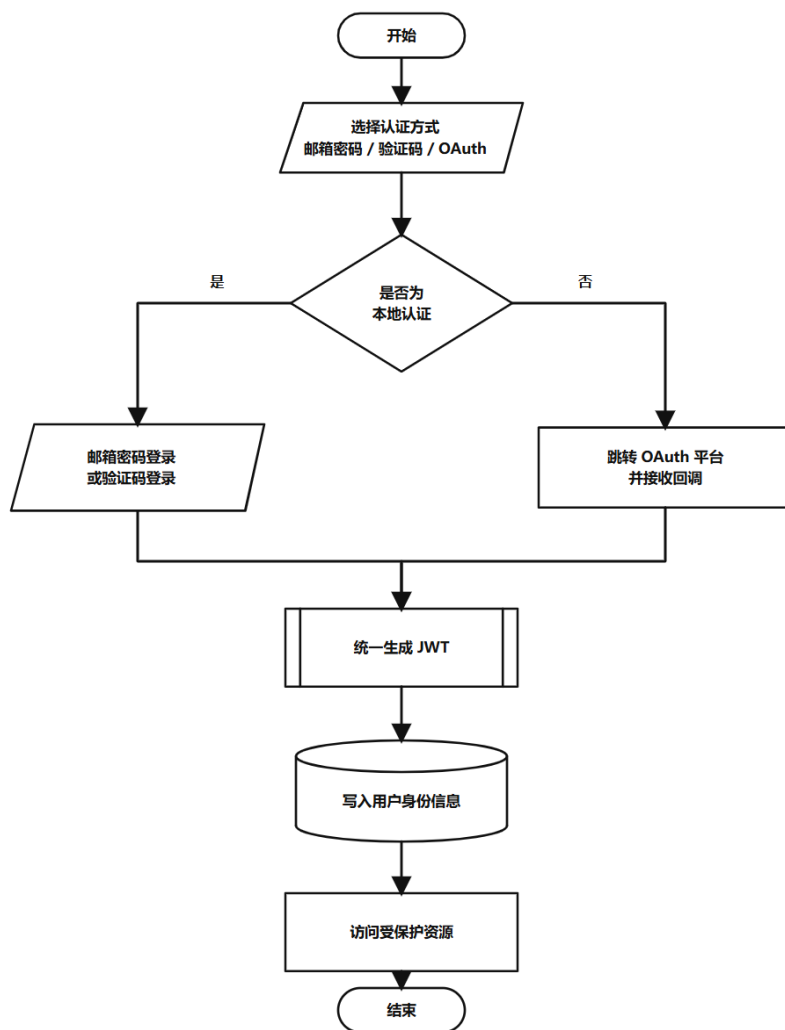


图 4.4 用户模块流程图

4.2.2 即时通讯模块

作为最强负荷中心,剥离“查询与记录走 HTTP”及“推送防丢赖 WebSocket”的双车道策略是最优解^[10,16]。其涵盖的单播与室内多播通过预先在服务端暂挂内存句柄完成投递定向,且保障所有动作须强制写入私联或通联群表后再响应成功。对各类带参卡片及媒体支持格式在此统一组装格式后交付前端编译展示,如图 4.5 呈现。

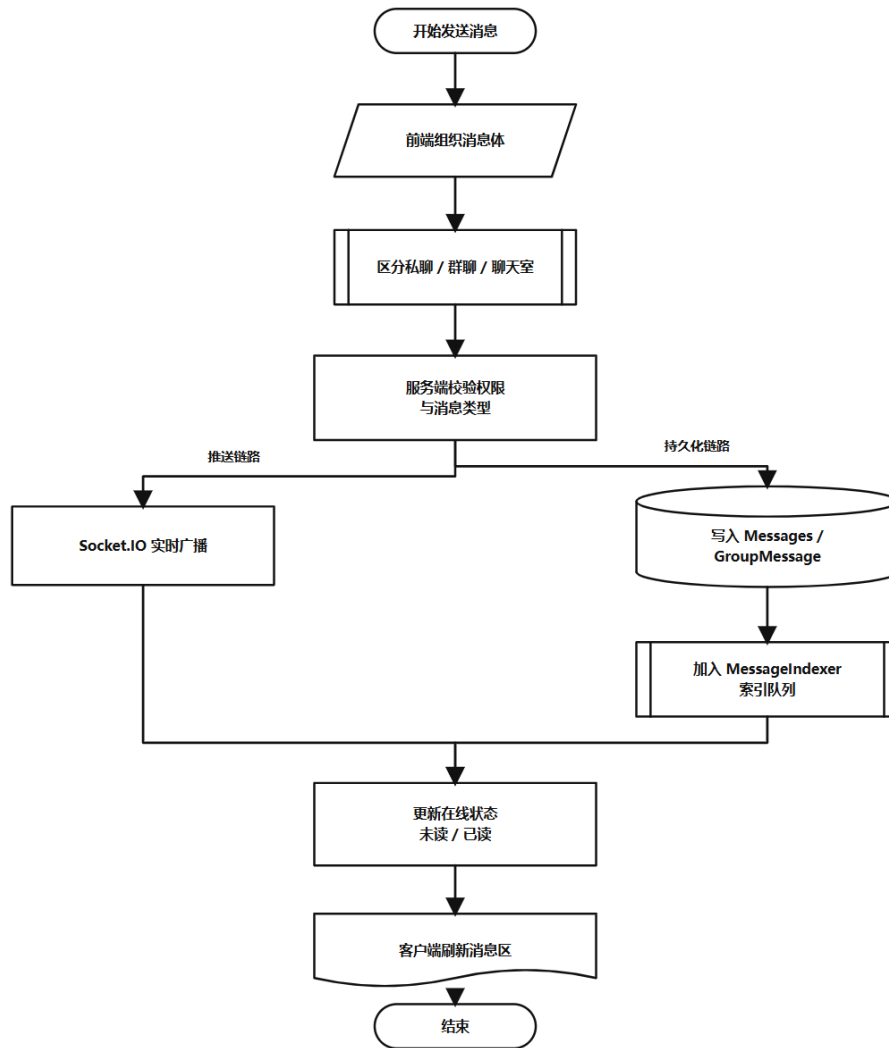


图 4.5 即时通讯流程图

4.2.3 聊天室模块

针对有时间敏感度的议事群落实时闭环。有异于松散群，其准入必须经过密令验证或外发邀请约束，且挂载死亡时效预计算。在超期或解散时冻结接应端并执行归档流。整个存活性机制大幅降解消隐群落给算力带来的长期牵连负担，流程流向展示于图 4.6。

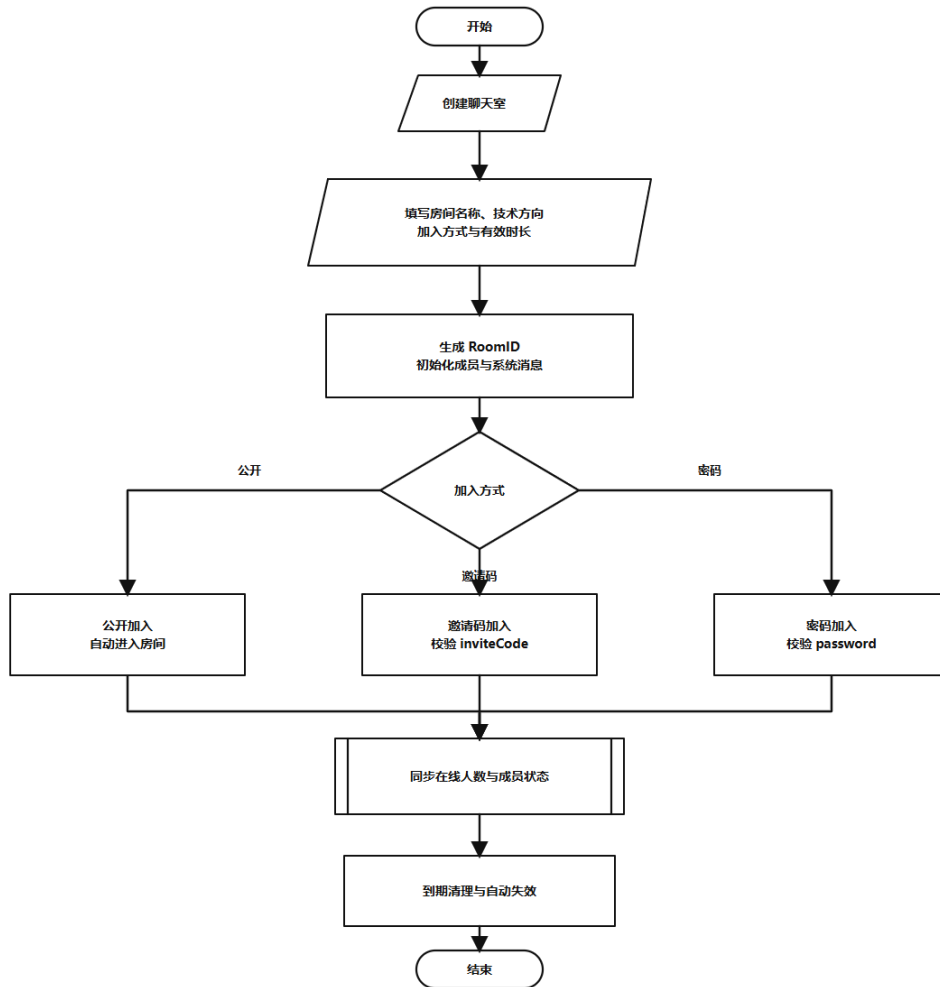


图 4.6 聊天室流程图

4.2.4 AI 助手智能问答模块

其定位为系统常驻脑力智囊。为了使其不发生断层错想，采取 RAG 先知模式（获取最近房间热议点以拟合提示文本）再执行长程推断递交请求。更可贵的是通过接入流式管道避免整段应答的黑屏滞后，将零散吐出的词标碎片借调 WebSocket 透明公放，极具联调效能。具体数据组织请看图 4.7。

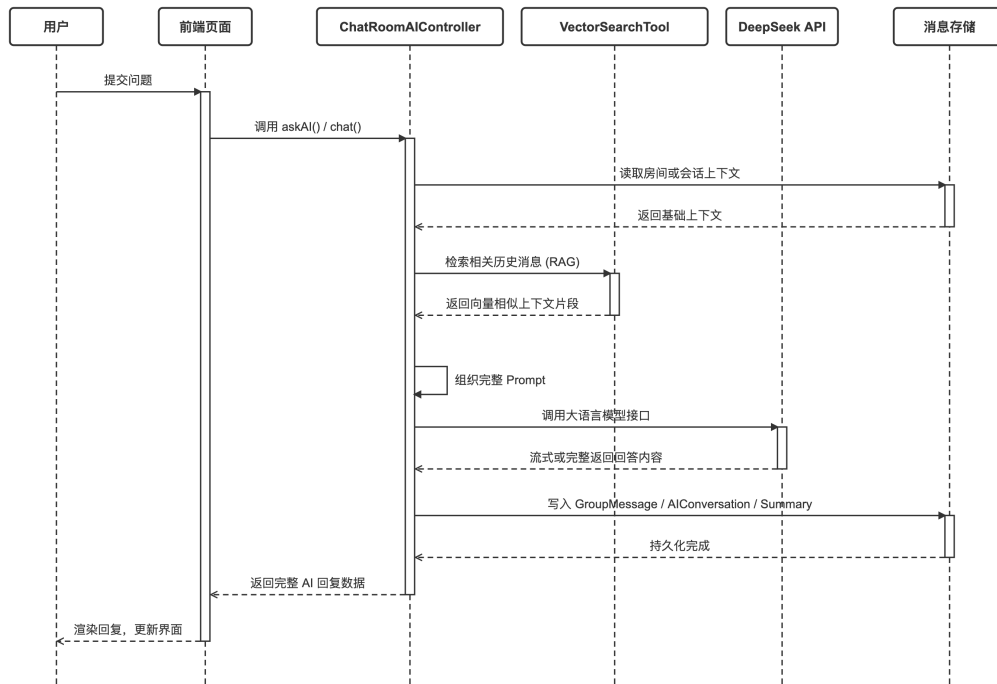


图 4.7 AI 助手智能问答模块时序图

4.2.5 收藏模块

作为学习结果提取仓，剥离简单的指针指向，改用全景深拷贝留存，涵盖附件原件及发信者源信息等。即使对岸清空数据库该资料仍能完好反哺。同时内置二次自拟 Tag 打签机制帮助实现后期回源的绝对指向，交互执行轴如图 4.8。

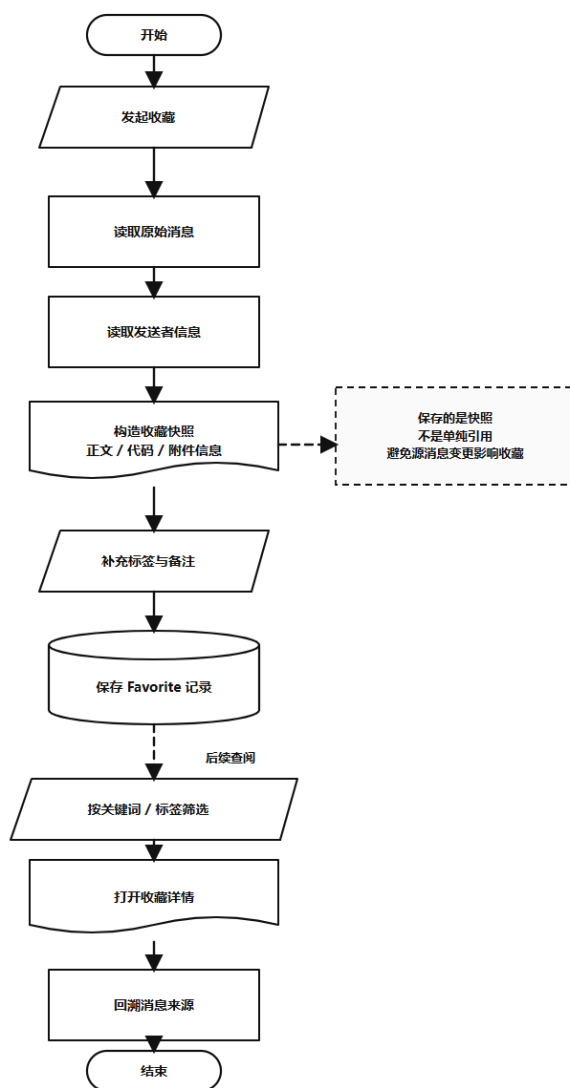


图 4.8 收藏流程图

4.3 界面设计

将底层数据外显于符合现代审美的窗体内，保障长效使用的舒适化。

4.3.1 登录与注册界面

化繁为简的双面卡片切换模式，底部分嵌 OAuth 授信方通道。杜绝周边杂波，在初印中注入高可信任感体验，面板构成见图 4.9。



图 4.9 登录与注册界面结构图

4.3.2 主聊天业务界面

排斥折叠及多余抽屉，贯彻左列菜单及总览条目的稳定三分天下定式布局。使得在应对宽泛的代码及工程级审视上不拘一格，为大区块浏览预留开阔的主体可视面积^[7,18]，如图 4.10所示。



图 4.10 主聊天界面结构图

4.3.3 沉浸式技术聊天室界面

在原本三分体基础上执行结构异化：强制上顶倒数挂钟，主区域附带悬浮控制板以专精聚焦回答等子集处理，输入框挂接代码专用预设，塑造隔绝闲聊的一体化科研级探讨室，见图 4.11。

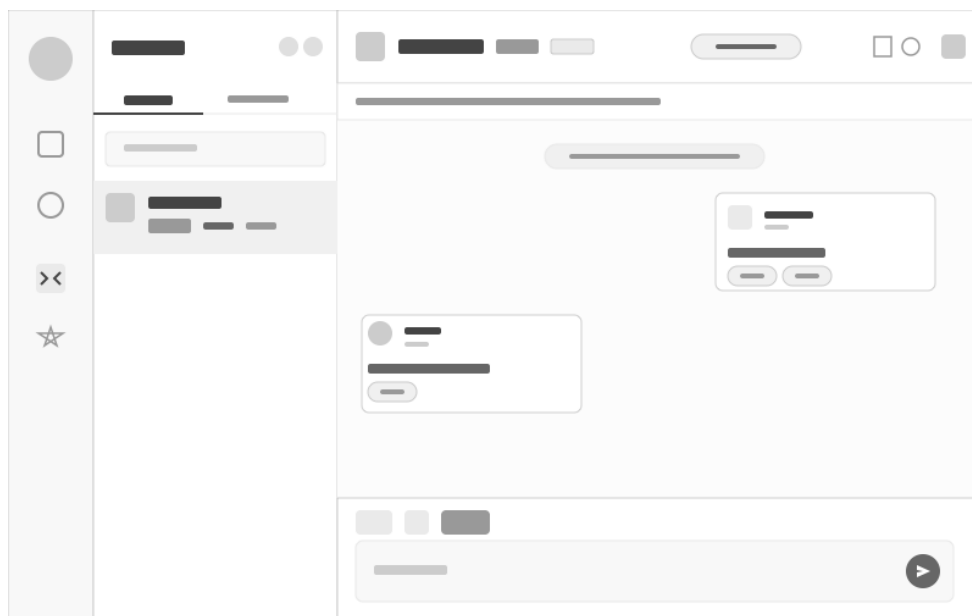


图 4.11 聊天室界面结构图

4.4 数据库设计

4.4.1 E-R 图设计

由于选用文档级数据库 MongoDB 应对非结构化变动，在逻辑蓝图规划上核心依旧是用户群发源。所有室内收发、点赞附加物均内嵌至信件对象关联里，以此极大节约联合调阅开销（图 4.12反映各骨干模型辐射绑定链路）。

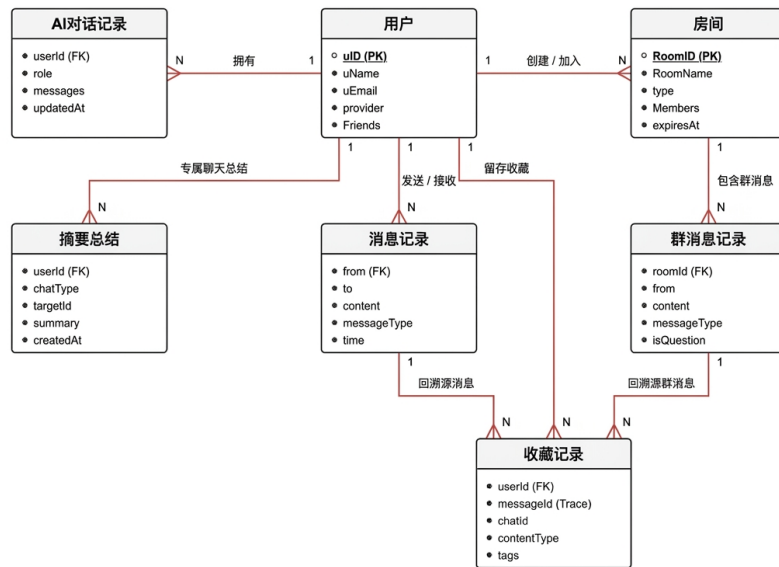


图 4.12 系统数据库 E-R 图

4.4.2 数据库逻辑结构设计

摒弃死板的过度范式追求，推崇根据拉高读写配比执行特长设计策略^[12]，各重要集合阐释如下列七表归述。

(1) 用户表设计

聚集基本权能与跨平台接入标识记录，为系统的认证校验提供坚实凭据，如表 eftar:4-1 所示。

表 4.1 用户表结构

字段名	主键	数据类型	是否为空	说明
_id	是	ObjectId	否	MongoDB 主键
uID	否	String	否	业务身份标识编码
uName	否	String	否	用户称呼展示
uEmail	否	String	否	联系列箱地址
Password	否	String	是	三方代理准入可放行空项
uAvatar	否	String	否	头像图形寻址
Friends	否	Object[]	是	知己互联花名册
googleId / githubId	否	String	是	分配给外部通道的唯一凭印
provider	否	String	否	账号发源通道标识

(2) 聊天室表设计

承载动态小群配置与管控时限及准入门规细则储备库，如表 eftar:4-2 所示。

表 4.2 聊天室表结构

字段名	主键	数据类型	是否为空	说明
_id	是	ObjectId	否	文档主从识别码
RoomID	否	String	否	组群明修与名号映射
Creator	否	String[]	是	首发者和护法席列
Admins	否	String[]	是	实绩登记簿带加入时序
Members	否	Object[]	是	管控是否强制时限判读位
type	否	String	否	会话科研标签
techDirection	否	String	是	加封口诀或者验证屏障手段
joinType / inviteCode	否	String	是	到时自动封号查验终点时间戳
expiresAt	否	Date	是	

(3) 私聊消息表设计

供单一好友两方查阅的最简讯息流通表结构设定，如表 eftab:4-3 所示。

表 4.3 私聊消息表结构

字段名	主键	数据类型	是否为空	说明
_id	是	ObjectId	否	序列号
from	否	String	否	发付标的及递达对象绑定
to	否	String	否	记录打标期号
time	否	Date	否	多模态图文数据内容挂载端
content	否	Mixed	是	提供对岸已达核查的回执手段
fileInfo	否	Mixed	是	
isRead	否	Mixed	是	
readTime	否	Mixed	是	

(4) 群聊与聊天室消息表设计

集成答录、引述甚至多状态流变的全功能大容量记录册，如表 eftab:4-4 所示。

表 4.4 群聊与聊天室消息表结构

字段名	主键	数据类型	是否为空	说明
roomId	否	String	否	高频呈现项数据双重落盘供列表反向快显
fromName	否	String	否	各类承载物正本及其判读型号
content	否	String	是	对高光代码块解析保留与静态盘体定位
messageType	否	Object	是	专业答题版块挂起的专设探针位标
codeInfo	否	Boolean	否	跟评群众心情刻度线数组与赞数集
fileInfo	否	Object[]	是	
isQuestion	否	Object[]	是	
isSolution	否	Object[]	是	
reactions	否	Object[]	是	
upvotes	否	Object[]	是	

(5) 收藏表设计

构建坚韧的深度原样复写拷贝，确保跨空间复用的持续保真，如表 eftab:4-5 所示。

表 4.5 收藏表结构

字段名	主键	数据类型	是否为空	说明
userId messageId	否	String	否	组成独有约束锚点保障收纳单向不重复
chatId contentType	否	String	否	原形场标识和归纳成分品格定调
content codeInfo	否	Mixed	是	主力载荷和补充属性防失效复制快件
tags note	否	Mixed	是	方便主人进行搜索查探定制化口头提要

(6) AI 对话表设计

截留并组织连片思维轮次的前瞻储备池，如表 eftab:4-6 所示。

表 4.6 AI 对话表结构

字段名	主键	数据类型	是否为空	说明
userId role	否	String	否	人物归系与智能代行形象特征装扮
rolePrompt messages	否 否	String Object[]	是 是	微调指示提示录像带 控制篇幅的上下文交互接龙集锦历史

(7) 聊天总结表设计

经后置解析归纳剥离出可指导后续的议题框架库，如表 eftab:4-7 所示。

表 4.7 聊天总结表结构

字段名	主键	数据类型	是否为空	说明
targetId chatType	否	String	否	从何处吸纳提取精华出处的底单指向
summary	否	Object	否	高价值结晶,横跨标题及动作执行指示
statistics msgCount	否	Mixed	是	反馈提取浓缩比例尺参照系数展示

4.5 核心底层机制与部署拓扑设计

4.5.1 WebSocket 即时通信协作机理

跳出低延迟泥潭的杀手锏。服务端通过建立双路多线循环机制对各类房间状态广播发话；在客户端提单存盘落妥响应时，方执行分发。保障无乱序失踪事故（详情参考图 4.13 执行机理）。

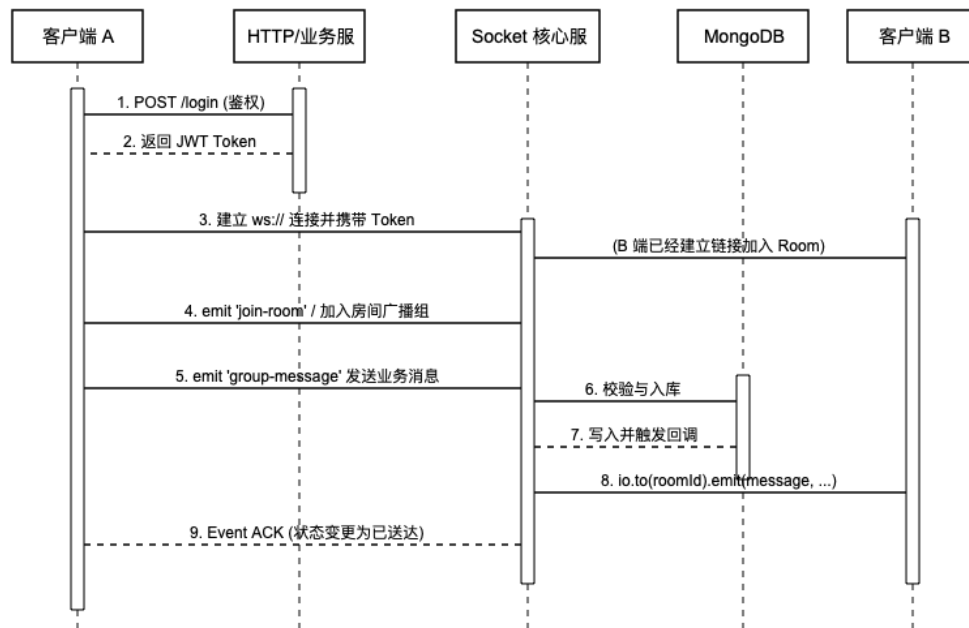


图 4.13 Socket 即时通信底层时序图

4.5.2 系统物理部署与网络拓扑架构

将前文种种落实在坚实算力平台上不可或缺的一环：Nginx 入口拦截调度转发分配负荷，内装 Express / Socket.IO 中继业务轮盘。外挂独立 Mongo 库镇守长夜储备，兼顾将核心推理算量脱壳调往远端 DeepSeek 基座计算并以异步带回（全视角物理走线观图 4.14）。

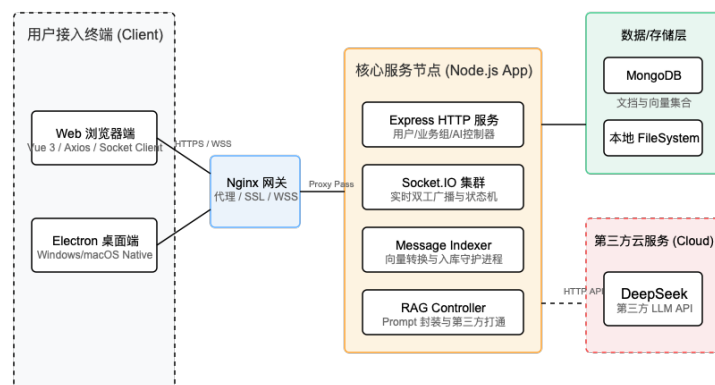


图 4.14 系统物理部署与网络拓扑架构图

从部署关系看，Web 端与桌面端共享同一套后端服务入口，实时通信、历史索引与 AI 能力围绕业务服务层统一组织。这样的物理拓扑既方便课程项目演示，也为下一章围绕后端、Web 前端和桌面端展开的具体实现说明提供了清晰落点。

5 系统实现

这一章主要回答一个问题：**Mini-Chat-Bar-main** 在代码里到底是怎么实现出来的。前面做需求分析和总体设计的时候，更多是在讲系统要做什么、为什么这样拆；到了这一章，就更关心实际落地的部分了。结合项目目录可以看出，整个系统不是一套单体前端，也不是单纯的聊天页面加几个接口，而是由 **server** 目录下的后端服务和 **ccb** 目录下的 **Web/Electron** 前端两部分一起组成的。前者负责鉴权、消息存储、AI 调用和 **Socket.IO** 实时通信，后者负责聊天界面、状态同步、桌面端托盘和通知交互^[9,10,18]。下面按后端、**Web** 前端和桌面端三个部分展开说明。

考虑到毕业论文篇幅限制，正文只保留实现思路、关键流程和代表性界面，能够直接反映模块落地方式的核心源码节选统一整理在附录中。其中，登录鉴权、消息处理、聊天室、AI、收藏、前端界面和桌面端托盘相关代码分别见附录 A 至附录 I。

5.1 后端实现

5.1.1 后端项目结构说明

从源码结构来看，后端并不是把所有逻辑都堆在 **server.js** 里，而是按职责拆成了路由、控制器、模型、**Socket** 事件、AI 代理和服务模块几层。这样拆的好处很实际：新增接口时先加路由，再补控制器；消息模型变更时只动 **models**；实时通信异常时直接去 **sockets** 里查，不会来回翻整棵目录。整体上，后端核心目录可以概括为配置层、路由层、控制器层、数据模型层、实时通信层以及 AI 服务与工具层。

- (1) **config** 主要处理数据库连接和 OAuth 配置。像 MongoDB 连接在 **db.js** 里完成，Google、GitHub 登录策略则在 **passport.js** 中注册。
- (2) **routes** 负责把请求路径分给不同业务模块。用户、私聊、聊天室、AI、收藏、上传这些接口都在这一层挂接。
- (3) **controllers** 负责真正的业务逻辑，是后端最核心的一层。前端点登录、发消息、拉历史记录，本质上最后都会落到这里。
- (4) **models** 对应 MongoDB 文档结构。项目里把私聊消息和群聊消息分开建模，这一点和前端界面划分是对得上的。
- (5) **middlewares** 目前最关键的是 **auth.js**，它负责 JWT 校验，后端很多接口能不

能访问都靠这一层先拦一下。

- (6) `sockets` 负责 `Socket.IO` 事件。私聊、群聊、语音房间和代码协作分别独立放置，后面出问题时会比较好排查。
- (7) `agents`、`services` 和 `tools` 一起组成 AI 能力。这里不是简单写个“调模型接口”的函数，而是做成了带步骤编排的 `Agent` 结构。

从实现方式上说，这样的目录拆分不算特别复杂，但对一个聊天系统来说已经够用了。后续如果还要继续扩，比如继续加消息审核、AI 角色管理或者更细的权限控制，也基本能沿着这套结构往下加，不至于把旧代码越改越乱。

5.1.2 核心模块实现

(1) 用户模块

功能说明: 用户模块主要由 `UserController.js`、`routes/user.js` 和 `config/passport.js` 组成，支持邮箱密码登录、邮箱验证码登录、邮箱注册，以及 Google、GitHub 的 OAuth 登录。登录成功以后，后端会返回 JWT，前端再把 `token` 和用户信息保存到本地，并继续初始化 `Socket` 连接和在线状态^[11,17]。

实现要点: 用户登录成功后，后端会统一生成带有效期的 JWT，并在响应中同时返回用户标识、昵称、邮箱与头像等基础资料，方便前端一次完成身份保持、用户信息回填和 `Socket` 初始化。

实现逻辑说明: 这部分实现比较扎实，没有花很多“包装性的”东西。普通注册时，后端先校验邮箱是否合法、密码强度够不够，再用 `bcrypt` 做密码哈希；普通登录时，用邮箱查用户，再比较哈希后的密码；验证成功后才生成 `token`。验证码登录的逻辑则是先走 `emailService`，把验证码暂存在内存结构里，用户提交验证码后再去核验。OAuth 登录稍微多一步，但思路也清楚：先从第三方平台取回资料，再尝试用第三方 ID 或邮箱查本地账户，如果没查到就自动创建。这样做的好处是，项目最后只维护一套 `Users` 集合，不会因为登录方式不同把用户表拆成几份。

另外，后端在 `/api/user/info` 里统一返回当前用户的基础信息，这个接口在前端其实用得频繁。登录完成后要靠它补齐头像和邮箱；设置页修改完资料之后也会重新请求它；OAuth 回调回来以后，前端也是先靠这个接口确认当前是谁，再决定进入哪一页。也就是说，用户模块虽然表面上只是“登录和注册”，但它其实是整个系统的入口。

邮箱密码登录与 JWT 下发的代表性实现见附录 A。

（2）聊天模块

功能说明：聊天模块分成私聊和群聊两条主线。私聊这一块主要落在 ChatController.js 和 Messages.js 上；群聊和技术聊天室则更多由 GroupController.js、Room.js、GroupMessage.js 这几处一起处理。除了最基础的文本消息，项目还支持图片、文件、语音、代码片段、引用回复和问题标记。

实现要点：群聊消息在进入数据库前会先补齐发送者昵称、头像、消息类型和引用信息等字段，写库完成后再更新房间活跃时间，并通过房间广播把完整消息对象同步给在线成员。

实现逻辑说明：群聊消息发送时，后端会先确认房间是否存在、当前用户是不是成员，再从 Users 集合里补齐当前昵称和头像，最后把消息写入 GroupMessage。这里有一个挺重要的处理细节：群消息文档里直接冗余保存了 fromName 和 fromAvatar，这样前端拉历史消息时就不用再逐条去查用户表，聊天室里消息一多，这个差别会很明显。私聊部分的思路类似，不过数据结构更简单，核心字段主要是 from、to、content 和 isRead。

从项目代码看，群聊并不只是一个“多人版私聊”。GroupMessage.js 里额外留了 quotedMessage、isQuestion、questionStatus、isSolution、reactions 等字段，这些字段和前端的技术讨论功能是一一对应的。比如用户在聊天室里标记“这条是问题”，后端保存的其实不是一个临时 UI 状态，而是实打实写到消息文档里；最佳答案、解决方案、表情反应也是同样的做法。换句话说，聊天室那套“像论坛但又比论坛轻”的交互，底层是有数据模型支撑的，不是前端临时拼效果。

私聊消息查询与发送的代表性代码见附录 B，技术聊天室创建的核心代码见附录 C。

（3）AI 智能检索与流式交互（RAG & SSE）

逻辑架构说明：AI 模块的核心突破点在于打破了“单向死板问答”的模式，将 /api/chatroom-ai/ask-stream 设计成了一个融合 RAG 向量检索与 Server-Sent Events (SSE) 双向广播的实时流式处理器。当用户在聊天室发送带有 @AI 的问题时，控制器不会简单把请求透明代理给 DeepSeek，而是会先接管上下文^[2,5,19]。

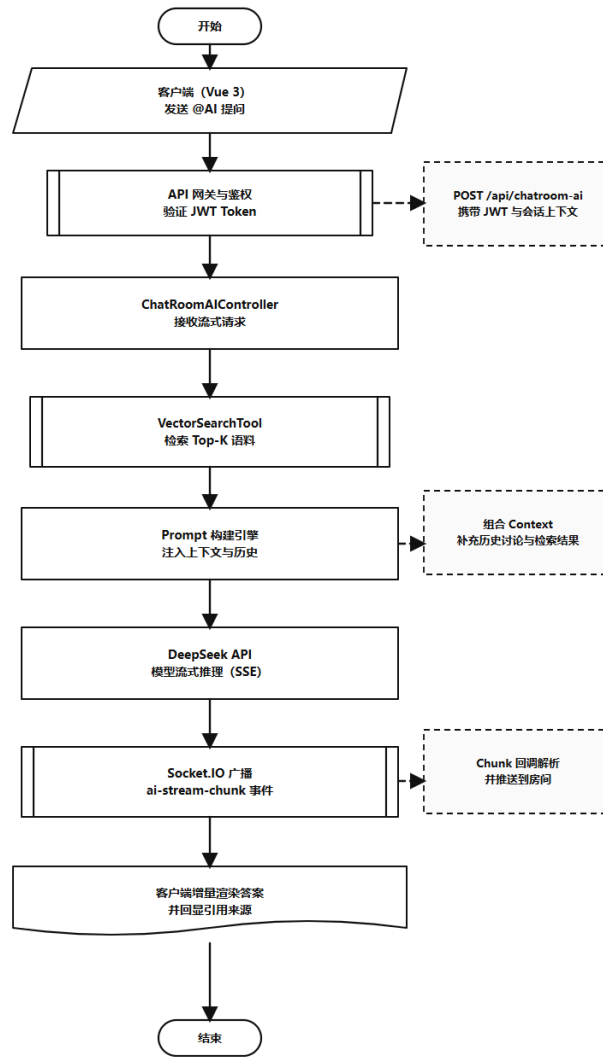


图 5.1 RAG 智能流式问答核心处理流

如图5.1所示，其实际执行链为：先通过 VectorSearchTool 在 MongoDB 历史集合中迅速抽取 Top-5 强相关聊天记录，将其组装进 Prompt，接着向后端发起 axios responseType: 'stream' 的请求。

实现要点： ChatRoomAIController.js 在拼装好检索上下文后，会发起流式模型请求，并把增量内容同时写回 SSE 响应和 ai-stream-chunk 房间广播事件，从而让提问者与同房间成员都能实时看到回答逐段生成。

实现机制延伸： 从上述源码可以看出，该接口非常巧妙地将 HTTP 的 SSE 协议与 WebSocket 长连接多播结合在了一起。提问者本人通过传统的长 HTTP response 依次渲染打字机效果，而同一房间内的其他观测者，则通过全局监听 ai-stream-chunk 事件获取增量块。对于计算极其耗时的大语言模型而言，这种分块下发机制彻底消灭了首字节等待时间（TTFB），使大规模技术群体的协同解题

体验做到了极致平滑^[20]。

聊天室 AI 流式回复与结果落库的代表性代码见附录 D。

(4) WebSocket 模块

功能说明：WebSocket 主要由 `sockets/chat.js` 和 `sockets/room.js` 处理。前者负责私聊消息、在线状态、头像同步等点对点事件，后者负责群聊加入、在线人数、输入状态、群消息广播等房间级事件。除此之外，`voiceRoom.js` 和 `codeCollaboration.js` 又把语音交流和代码协作单独拆了出去^[10,16]。

实现要点：WebSocket 层分别维护“用户到 socket 集合”和“房间到在线用户集合”的映射关系，在广播消息前先完成连接归属和用户去重，再回推在线人数与消息增量。

实现逻辑说明：这一层最实用的地方在于，它不是拿 `socket.id` 直接当用户标识来用，而是做了两层映射。私聊这边，`users` 存的是“一个用户对应一组 socket 连接”，这样同一个账号在多个窗口同时登录也不会丢消息；群聊这边，`roomUsers` 记录的是“某个房间里有哪些 `userId` 在线”，所以在线人数统计按用户去重，不会因为一个人开两个窗口就多算两次。这个处理虽然看起来只是多套了一个 Set，但对聊天工具来说挺关键，不然在线人数和未读提醒会很容易乱掉。

还有一点也很明显：项目没有把所有实时事件混在一起处理。私聊消息、群消息、语音连麦、代码协作是分文件写的，这样后面改群聊，不容易误伤语音房间；查 Bug 的时候也能更快定位到具体层面。对于一个功能越来越多的聊天系统来说，这种拆法比“都塞在一个 socket 入口里”稳很多。

房间在线状态维护与群消息广播的核心实现见附录 E。

(5) 数据模型与辅助模块

功能说明：后端除了核心聊天主链路，还补了不少辅助能力，包括上传、收藏、问题追踪、代码片段保存、消息索引和链接预览。这些模块单看都不算大，但它们把系统从“能聊”往“更适合技术交流”推了一步。

实现要点：群消息模型除了发送者、内容和时间等基础字段外，还扩展了引用回复、问题标记、表情反应和附件信息等结构，为收藏、历史检索和 AI 分析提供了统一的数据基础。

实现逻辑说明：这部分能看出项目并不是只在前端做“功能按钮”，而是把对应的数据结构也补齐了。比如引用回复、问题标记、表情反应，这些都是消息模型自带的字段；收藏模块则会把消息快照保存下来，即使原消息被删，收藏内容也还能留住；上传模块先把文件落到服务器，再把文件地址写回消息记

录，所以图片、语音和文件在历史记录里都能长期回看。还有 MessageIndexer 和 RAGService，它们负责把文本消息做向量化索引，给后面的 AI 检索做准备。说得直白一点，这些辅助模块不是“可有可无的小功能”，它们已经和主聊天流程绑在一起了^[12,21]。

收藏数据写入与来源回读的代表性实现见附录 F。

（6）聊天室洞察与智能话题聚类分析（Room Insights）

对于一个活跃的技术交流平台，除了被动等待用户输入指令外，后台系统还需要具备“宏观把控房间态势”的数据加工能力。项目中专门设计了 getRoomInsights 接口来实现对房间消息生命周期的全局审视。

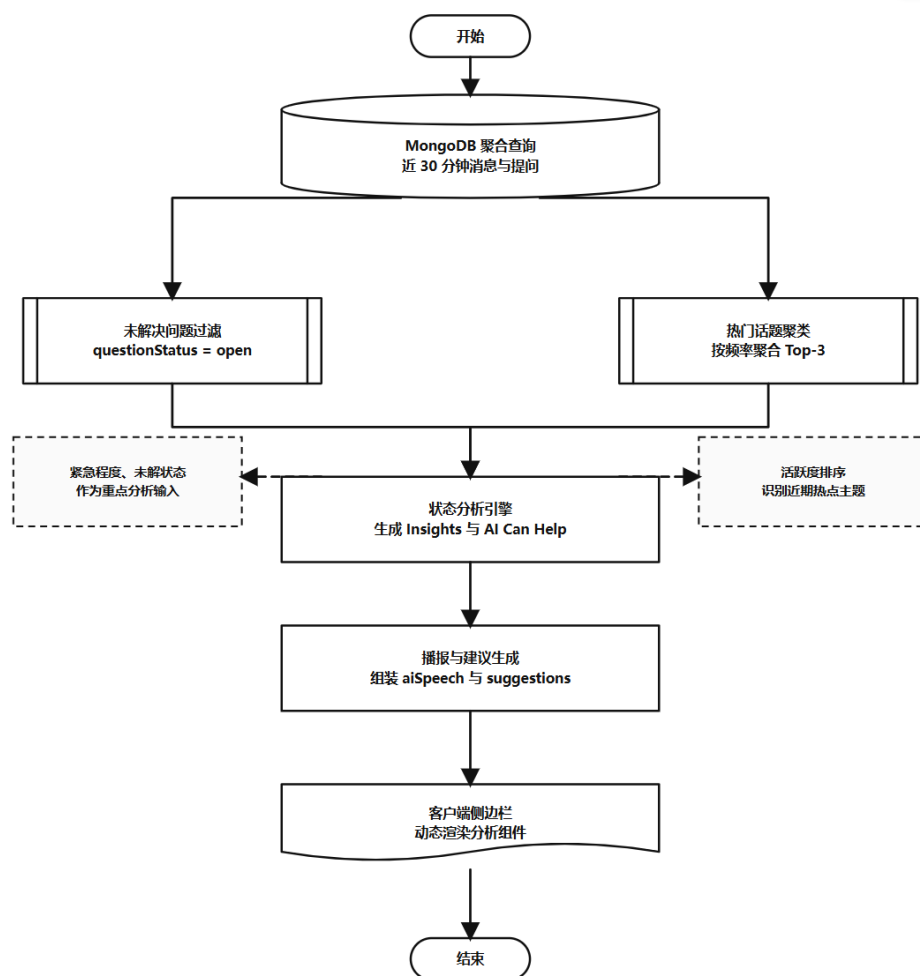


图 5.2 聊天室实时洞察与状态分析流

如上图5.2所示，该组件并非通过机器学习来处理繁重语料，而是极为巧妙地结合了 MongoDB 文档数据库的灵便聚合特性。

实现要点：洞察模块会先筛出未解决问题，再基于最近消息构建话题映射，

表 5.1 普通 REST 接口设计

接口名称	请求路径	请求方式	参数说明	返回示例
发送验证码	/api/user/send-verification	POST	email、type	返回提示信息
邮箱密码登录	/api/user/login-email	POST	email、password	返回 token 与用户信息
获取当前用户信息	/api/user/info	GET	Bearer Token	返回 user 对象
获取私聊消息	/api/chat/messages/:id	GET	路径参数为对方用户 ID	返回消息数组
发送群聊消息	/room/:roomId/messages	POST	content、messageType、quotedMessage 等	返回消息对象

统计讨论热度与参与者范围，最后把聚合结果整理成结构化的 insights 报告供前端展示。

算法层价值：这组算法赋予了聊天室极强的“可读观测能力”。系统先按时间流切割最近 30 分钟的数据窗口，再利用原生 Map/Set 结构在计算服务器主内存中实时执行高频遍历去重聚合。它能无缝剥离出哪些是“无人解答的冷门 Bug”，哪些是“正处于广泛讨论的代码异常”。基于处理结果，系统组装出一份扁平化的 insights 评估报告并生成口语化的 aiSpeech 回塞给前端呈现侧边悬浮引导框。这种由数据流反向牵引互动的设计极大地加强了学术与开发群体之间的良性共振。

5.1.3 接口设计

结合 server.js、各个路由文件和控制器实现，可以把项目常用接口分成普通 REST 接口、AI 接口和 WebSocket 通信接口三类。这里不把所有接口都展开，而是挑出最能代表系统实现特点的几组来说明。为了避免接口设计部分只停留在表格罗列层面，下面按照接口类别分别进行说明，并结合表中内容概括每一类接口承担的职责。

(1) 普通 REST 接口设计。这一部分主要包含用户认证、用户资料读取、私聊历史查询以及聊天室消息写入等标准 HTTP 请求接口。它们的共同特点是“请求一次，返回一次”，重点解决身份校验、数据持久化和列表查询等基础业务问题，是前端大多数页面功能能够正常工作的前提。普通 REST 接口的代表性设计如表 5.1 所示。

由表 5.1 可以看出，普通 REST 接口更多承担“可靠落库”和“稳定读取”的任务。其中，登录、发送验证码和获取当前用户信息属于用户域接口，私聊消息读

表 5.2 AI 接口设计

接口名称	请求路径	请求方式	参数说明	返回示例
侧边栏智能问答	/api/agent/chat	POST	question、chatType、sessionId 等	返回答案、来源和置信度
会话总结	/api/agent/summarize	POST	chatType、targetId/roomId、timeRange	返回结构化总结
划词解释	/api/agent/explain	POST	text、isFollowUp	返回解释文本
聊天室流式问答	/api/chatroom-ai/ask-stream	POST	roomId、question、useRAG	以 SSE 分块返回结果

取和聊天室消息发送则属于会话域接口。这类接口并不追求长连接式的实时推送，而是强调参数明确、返回结构稳定和便于前后端分层调试。

(2) AI 接口设计。 AI 接口主要服务于智能问答、文本解释和聊天总结等增强功能。与普通业务接口相比，它们通常需要拼接上下文、调用检索服务并进一步请求大语言模型，因此返回结果中不仅包含答案正文，还可能包含来源信息、置信度或结构化摘要内容。AI 接口的代表性设计如表 5.2 所示。

从表 5.2 可以看出，AI 接口并不是单一的一条“问答接口”，而是根据使用场景分成了侧边栏智能问答、会话总结、划词解释和聊天室流式问答四种类型。这样的拆分有助于把“单人查询”“多轮解释”和“多人协作中的 AI 介入”区分开来，也方便前端在不同页面里选择更合适的交互方式。

(3) WebSocket 通信接口设计。 WebSocket 部分主要用于处理用户上线注册、私聊消息通知、加入聊天室、群消息广播以及 AI 流式输出同步等事件。它与前两类接口最大的不同，在于它强调服务端主动推送和多端实时同步，更适合承载聊天系统里的在线状态与增量消息场景。WebSocket 通信接口的代表性设计如表 5.3 所示。

表 5.3 反映出系统实时通信层的核心职责，即把“谁上线了”“谁加入了房间”“哪条消息需要立即同步给谁”这些高频事件从普通 HTTP 请求中拆出来单独处理。尤其是 group-message 与 ai-stream-chunk 这类事件，直接决定了聊天室消息和 AI 回答能否在界面上即时呈现。

表 5.3 WebSocket 通信接口设计

接口名称	通信标识	触发方式	参数说明	返回示例
用户上线注册	login	client emit	userId	服务端登记在线状态并回推列表
私聊消息通知	private-message	双向	from、to、content 等	实时推送私聊消息
加入聊天室	join-group	client emit	roomId、userId	触发在线人数和成员广播
群消息广播	group-message	双向	roomId、content、fromName 等	房间成员收到完整消息对象
AI 增量输出	ai-stream-chunk	server emit	roomId、content、isComplete	前端逐段渲染 AI 回答

5.1.4 示例数据

为了让接口设计看起来不是空表格，下面给出两个更贴近实际返回格式的示例。字段名和控制器中的实现保持一致，只对 token 和部分 ID 做了简化。

示例 1：邮箱密码登录接口返回。返回体通常包含 message、token 和 user 三部分，其中 user 内含用户 ID、昵称、邮箱与头像地址，足以支撑前端完成登录后的首轮渲染。

示例 2：智能问答接口返回。返回体会以 success 标记请求结果，并在 data 中给出回答正文、来源片段、置信度、是否使用上下文以及检索条数等信息，用于前端展示答案与解释其生成依据。

5.1.5 通信机制说明

（1）HTTP 通信流程

普通业务请求大多走的是这条链路：前端通过 Axios 发起请求，请求头里自动附带 token；后端先经过 auth.js 做鉴权，确认用户身份没问题之后，再进入控制器；控制器根据请求内容调用 Mongoose 模型读写 MongoDB，最后把结果返回给前端。像登录、获取当前用户信息、拉取历史消息、上传文件、创建聊天室这类操作，本质上都是按这个流程往下走^[9,12]。

流程概括：token 会先从本地读取并附在请求头中，服务端经过 auth.js 完成鉴权后，再由控制器调用对应数据模型查询信息，最后把 user 等业务对象返回给前端。

（2）WebSocket 实时通信流程

实时通信和 HTTP 的分工很清楚。HTTP 更像是“把数据真正写下来”，Socket.IO 则负责“让在线成员马上看到变化”。私聊发送时，前端通常先走 REST 接口写库，再发 `private-message` 通知对方刷新；群聊发送时，前端调用消息接口后，后端控制器会主动广播 `group-message`，所以前端反而不用自己再补发一次。这样一来，消息是否保存成功、在线用户是否立刻收到，也都能分别在不同层面查到^[10]。

流程概括：客户端建立连接后会先完成 `login` 与 `join-group` 等注册动作，消息写库成功后由服务端主动广播 `group-message`，房间成员据此刷新列表和未读状态。

（3）AI 请求调用流程

AI 请求比普通接口多了一层上下文和检索处理。侧边栏 AI 助手先进入 `AgentController`，再由 `ChatAgent` 规划步骤，按顺序执行最近消息提取、RAG 检索和答案生成。聊天室里的 `@AI` 则更偏向实时协作，控制器会先拼好提示词，再请求流式模型接口，并把增量内容同步回房间^[2,5,20]。

侧边栏问答链路：请求先进入 `AgentController`，再由 `ChatAgent` 组织最近消息提取、RAG 检索和答案生成，最后把回答正文、来源和置信度一并返回。

聊天室流式链路：`ChatRoomAIController` 收到提问后，会先调用检索工具补足上下文，再请求流式模型接口，并把分块结果同时写回 HTTP 响应和房间广播事件。

5.1.6 展示内容

为了验证接口文档与控制器实现保持一致，项目在联调阶段使用 Swagger 页面对主要 REST 接口进行逐项核对，包括用户认证、聊天室消息、收藏和 AI 相关请求。接口文档界面如图 5.3 所示，它能够直接展示请求方法、路径、参数结构和调试入口，对后续接口联调和答辩演示都比较直观。

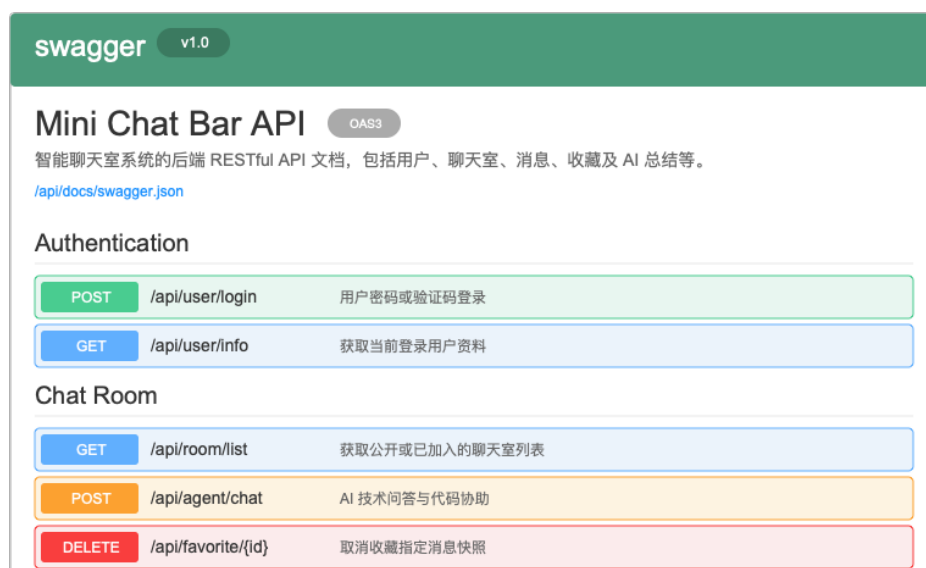


图 5.3 Swagger 接口文档

后端运行时的日志也比较有代表性。接口是否进来了、Socket 有没有连上、AI 请求有没有发出去，在控制台里基本都能看出来。论文里把这一层留一小段出来，其实挺有必要，因为它能让“系统确实跑起来了”这件事更直观。

例如登录接口返回、聊天室消息拉取、Socket 建连和 AI 接口调用，都会控制在控制台留下时间戳、访问路径和耗时信息，便于联调阶段快速定位问题。

5.2 Web 前端实现

前端实现主要集中在 ccb/src 目录下。这个项目的前端不是完全按技术层来分，而是更偏向功能模块：登录页单独做一套交互，聊天主界面负责承接会话和导航，AI 助手单独有自己的组件，消息管理和设置页也各自独立。这样的写法虽然没有做得特别“学院派”，但在阅读代码时会比较顺，想找哪个功能就直接去对应目录，不太容易迷路^[7,8,18]。

其中，聊天主界面的路由切换逻辑见附录 G，收藏页的筛选与详情回看逻辑见附录 H。

5.2.1 登录/注册页面

界面功能说明：登录和注册主要放在 components/layout/Login.vue 中。页面做成了翻转卡片形式，正面是登录，背面是注册。登录又分成邮箱密码和邮箱验证码两种方式，卡片底部另外保留了 Google 和 GitHub 的第三方登录按钮。这个页面在视觉上比较完整，不是只放了一个表单，而是把欢迎信息、输入反馈和登

录方式切换都做进去了，如图5.4和图5.5所示。

关键交互逻辑：用户在输入邮箱和密码后，前端会先做格式校验。邮箱格式不对、密码为空，或者注册时两次密码不一致，页面会先在前端挡下来，不会直接把无效请求丢给后端。验证码登录则多了一步冷却时间控制，60 秒内不能重复获取验证码，这一点也是在前端先处理的。登录真正成功以后，页面不会只保存 token 就结束，而是继续调用 `waitForSocketConnection()`，在 Socket 成功连接后发出 login 和 user-online 事件，把在线状态一起初始化。

与后端的接口交互说明：邮箱密码登录调用 `/api/user/login-email`，验证码登录调用 `/api/user/login-code`，发送验证码调用 `/api/user/send-verification`，注册调用 `/api/user/register-email`，第三方登录则直接跳转到 `/auth/google` 和 `/auth/github`。

操作流程：用户先进入登录页，根据自己的习惯选择密码登录、验证码登录或第三方登录；如果是本地登录，就输入表单并提交；后端验证通过以后返回 token 和用户资料；前端接着保存登录信息、初始化在线状态和 Socket 连接，最后跳转到聊天主界面。

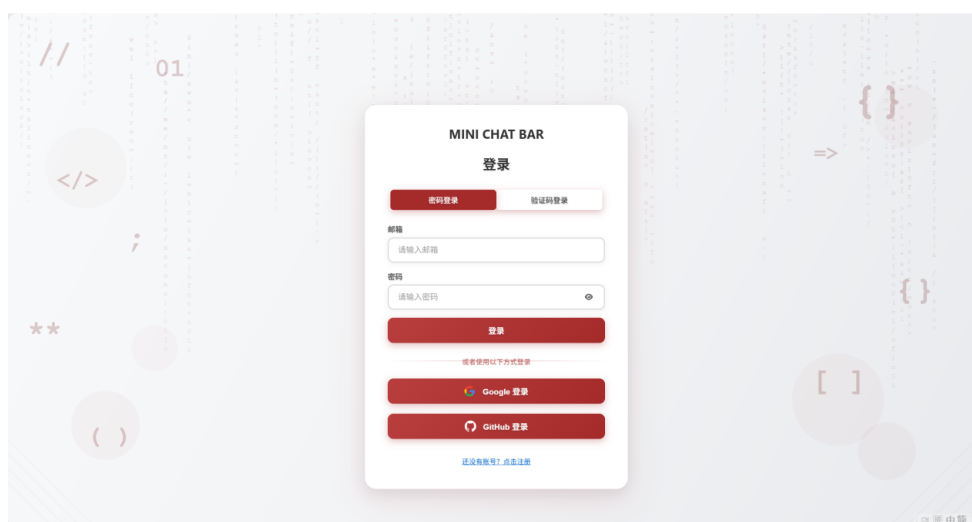


图 5.4 登录页面展示

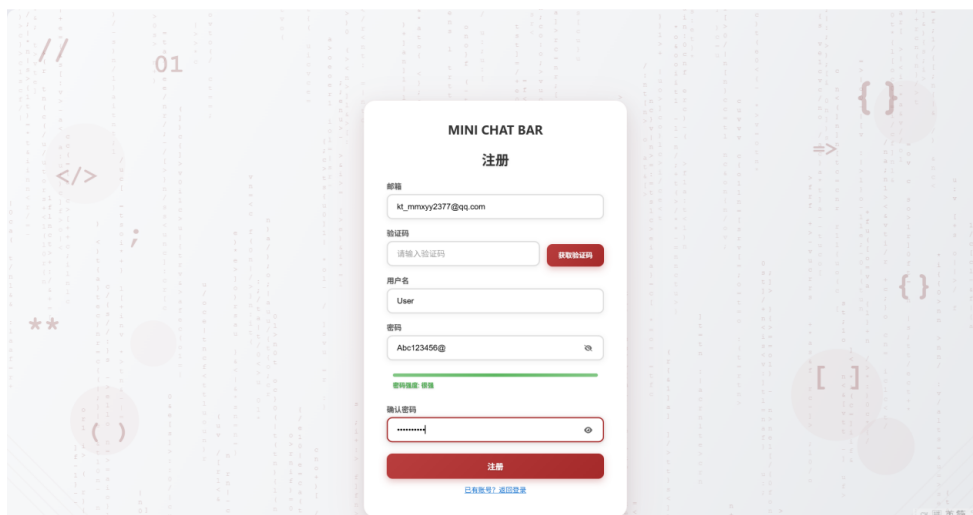


图 5.5 注册页面与表单交互展示

5.2.2 聊天主界面

界面功能说明：聊天主界面由 ChatView.vue 负责整体承接。桌面端采用典型的三栏布局，左侧是导航入口，中间是会话或联系人列表，右侧是详情区；移动端则切换成底部导航形式。实际看代码会发现，这个页面更像一个容器，而不是所有逻辑都在这里处理。它的工作重点是根据当前路由和按钮点击结果，决定中间栏和右侧区域应该显示什么。主界面总览如图5.6所示，通讯录和聊天室入口则分别如图5.7、图5.8所示。

这一部分对应的前端路由切换与界面组织代码见附录 G。

关键交互逻辑：页面初始化时，前端会先检查是否已经登录，再根据路由状态决定默认展示聊天列表、联系人还是聊天室列表。用户点击左侧导航栏或底部导航时，不是单纯隐藏某一块 DOM，而是通过路由切换对应的内容区域。用户进一步点进某个联系人或房间后，showdetail() 会把当前目标 ID 写入 useChatStore，并把昵称、头像、聊天类型等参数一并带到详情页。这一层虽然看着简单，但它把“当前到底在和谁聊”“当前打开的是私聊还是聊天室”这两个最核心的上下文传稳了，后面的消息加载和状态同步才能跟上。

与后端的接口交互说明：主界面本身不直接发送消息，但会触发好友列表、群列表、聊天室列表以及用户信息等接口的请求。常见调用包括 /api/user/info、/api/user/friends、/room/list 和 /room/chatrooms。

操作流程：用户登录进入主界面后，前端先加载基础资料和列表数据；再根据当前需求切到聊天、联系人或聊天室；选择具体会话后，前端写入当前会话上下文并跳转详情页；详情页继续拉取消息记录并建立需要的 Socket 监听。

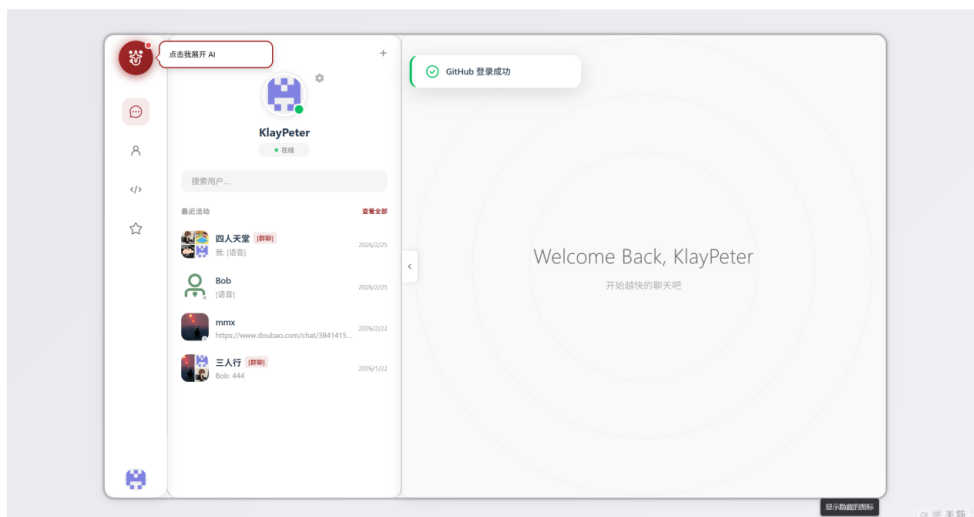


图 5.6 聊天主界面总览

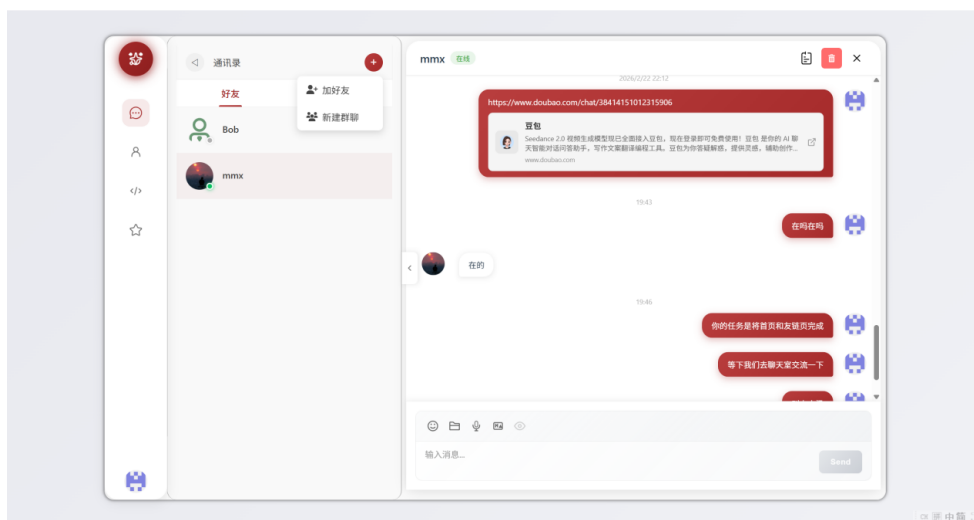


图 5.7 通讯录列表界面展示

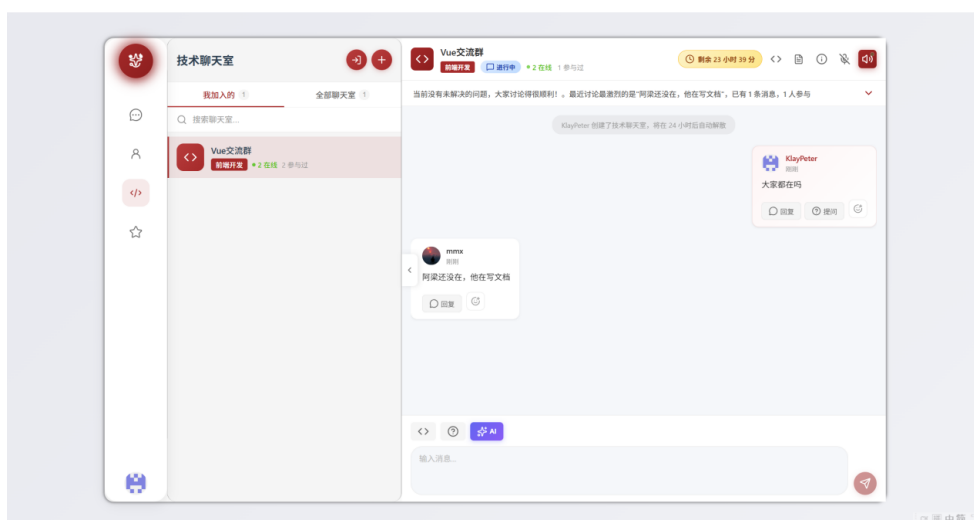


图 5.8 聊天室列表界面展示

5.2.3 AI 对话界面

界面功能说明：AI 对话相关的界面有两种典型形态。第一种是侧边栏 AI 助手，由 `AIAssistantPanel.vue` 提供，适合做单独问答、补充解释和上下文延续；第二种是聊天室里的 @AI 交互，它把 AI 回答直接放回房间消息流，更适合多人讨论时一起参考。侧边栏 AI 助手界面如图 5.9 所示，聊天室中的 AI 回答效果如图 5.10 所示。

关键交互逻辑：侧边栏模式下，用户输入的问题会先追加到本地消息数组，再调用 `/api/agent/chat` 请求答案，请求成功后再把 AI 回复和来源信息追加到界面。聊天室模式则更强调实时感。用户在输入框里直接输入 @AI 开头的问题，前端会先把原问题发进房间，再触发 `/api/chatroom-ai/ask-stream`，然后一边接收流式内容，一边刷新当前房间的消息列表。因为后端会把 AI 输出同步广播到房间，所以这个回答不是只对提问者可见，而是房间成员都能看到。

与后端的接口交互说明：侧边栏模式主要依赖智能问答和解释接口，聊天室模式则围绕流式问答、智能提示和主动答题几类接口展开。

操作流程：AI 模块的操作流程可以概括为：用户打开 AI 面板或在聊天室中输入 @AI；前端提交问题并附带当前上下文；后端结合最近消息和历史检索结果生成回答；回答返回后，前端再按普通消息或流式消息渲染出来。

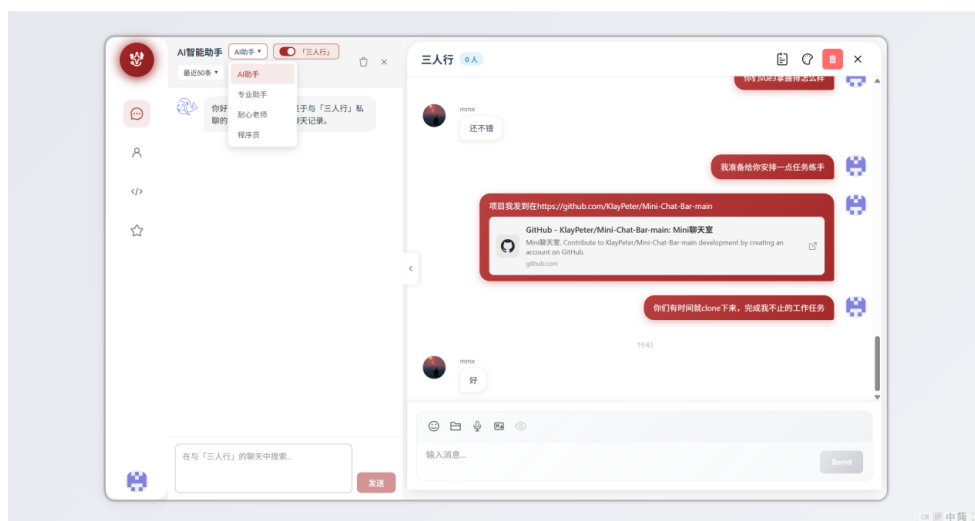


图 5.9 侧边栏 AI 助手界面

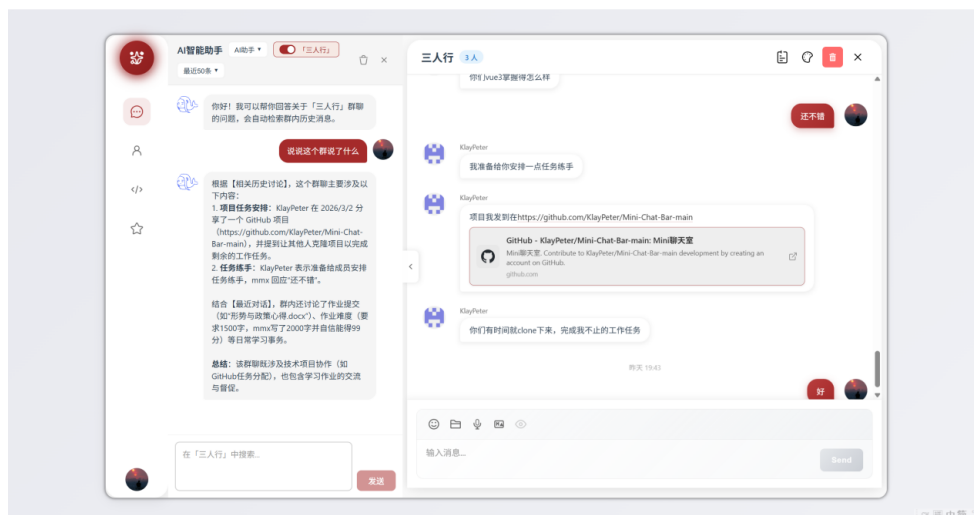


图 5.10 AI 根据历史记录检索后的回答展示

5.2.4 消息列表与会话管理

界面功能说明：会话管理相关功能主要分布在 LastChats.vue、Content.vue、ChatMessageList.vue 和 Favorites.vue。这一块不只是展示消息列表，还包括最近会话排序、未读数显示、历史消息搜索、收藏整理以及消息转发、撤回等行为。最近会话列表和消息页结合后的效果如图5.11所示，历史消息搜索和收藏页分别如图5.12、图5.13所示。

关键交互逻辑：会话列表加载时，前端会先拉好友列表，再去查每个联系人和群聊的最后一条消息以及未读数，随后按时间重新排序。当前会话打开以后，页面会继续拉完整历史记录，并对消息类型做分类渲染。用户点击搜索图标时，前端会在当前消息数组里做关键词匹配并高亮结果；点击收藏时，则把消息 ID、消息类型和会话 ID 一起发给后端，方便后面单独在收藏页回看。这里的处理比较贴近日常使用场景，不会让用户只能靠无限滚动去找旧消息。

与后端的接口交互说明：这一块会频繁调用 /api/chat/last_message/:id、/api/chat/unread、/api/chat/messages/:id、/room/:roomId/messages、/api/favorites 和 /api/favorites/:messageId 等接口。

操作流程：消息列表与会话管理的基本流程是：先加载会话摘要信息，再根据用户选择打开具体会话；进入会话后加载完整历史消息；如果用户执行搜索、收藏或删除等操作，再分别调用对应接口更新状态，并同步刷新列表。

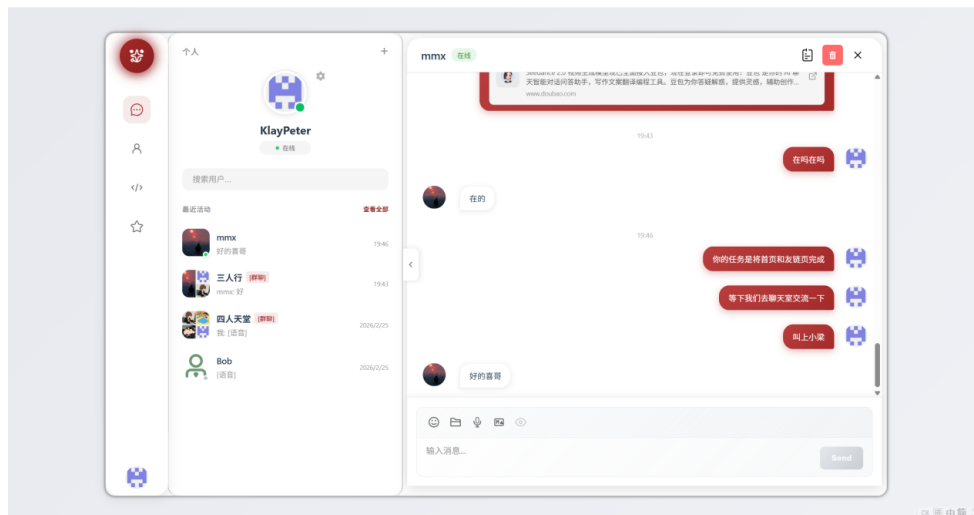


图 5.11 会话列表与私聊界面展示

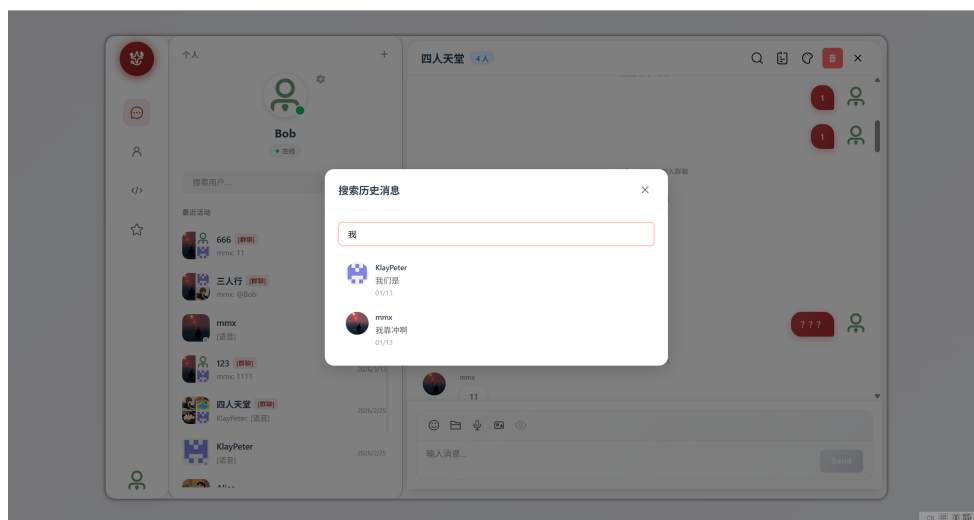


图 5.12 历史消息搜索界面展示

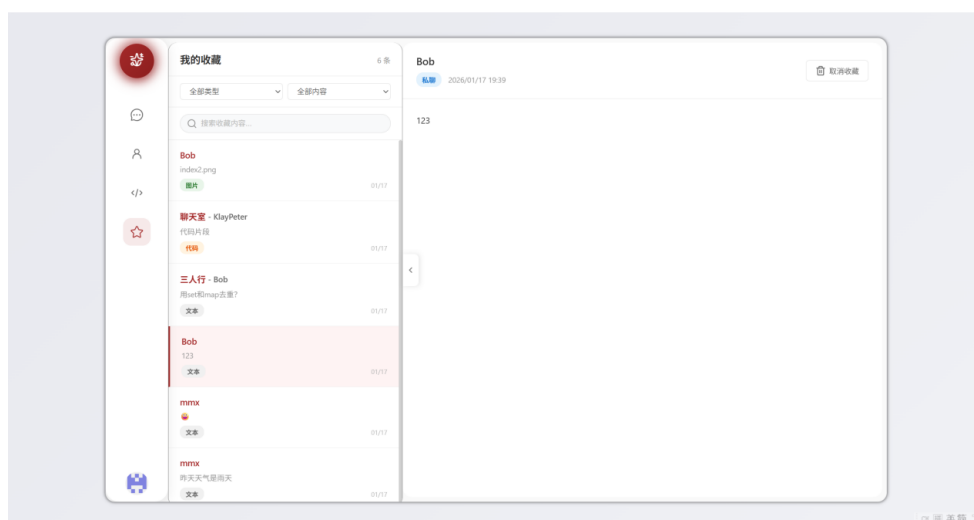


图 5.13 收藏页面展示

5.2.5 设置与个人中心

界面功能说明：设置与个人中心主要集中在 `SettingsDialog.vue` 中，包含头像上传、预设头像切换、用户名修改、主题切换和退出登录。虽然这一块功能不像聊天页那么密集，但它和主界面并不是割裂的。头像一改，最近会话列表、当前聊天页和其他在线用户看到的资料都要跟着刷新。设置与个人中心界面的实际效果如图5.14所示。

关键交互逻辑：用户上传头像时，前端先把图片发到 `/api/upload`，拿到文件地址后再调用 `/api/user/avatar` 更新资料；修改用户名时，则直接调用 `/api/user/username`。更新成功后，前端会发出 `avatar-updated` 事件，通知其他在线用户同步刷新头像。主题切换则通过 `useThemeStore` 完成，并把主题键值写入 `localStorage`，这样刷新页面后仍然保留用户上一次的选择。

与后端的接口交互说明：头像上传依赖 `/api/upload`，资料更新依赖 `/api/user/avatar` 和 `/api/user/username`，界面刷新时则重新读取 `/api/user/info`。

操作流程：设置模块的操作流程比较顺：用户打开设置弹窗，修改头像、用户名或主题；前端调用对应接口并保存结果；保存成功后，同步刷新本地资料和相关组件；如果是头像这类会影响其他用户的内容，还会继续通过 `Socket` 广播更新。

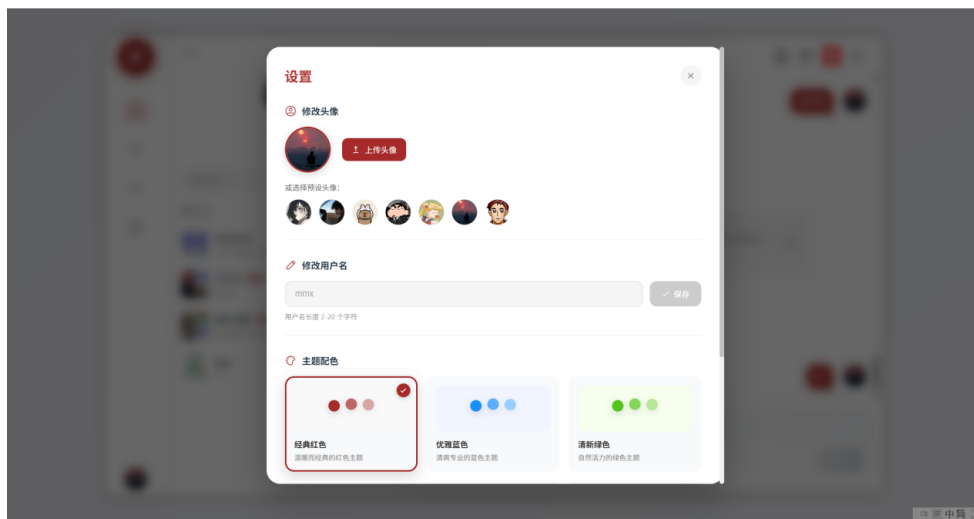


图 5.14 设置与个人中心界面展示

5.2.6 Web 端消息流转与状态管理

如果只看功能按钮，Web 端看起来像是“登录以后发消息、收消息、切会话”这么简单，但真正让它稳定运行的，是消息流转和状态管理这一层。这里可以分三部分来看。

第一部分是消息发送、接收和渲染流程。私聊发送时，Content.vue 会先调用 `/api/chat/messages/:id` 把消息写入数据库，再通过 `private-message` 事件提醒另一端刷新；群聊发送时，前端调用 `/room/:roomId/messages`，后端控制器保存成功后会主动广播 `group-message`，所以前端不需要重复发同名消息。收到消息以后，界面不是直接把原始字符串塞上去，而是会根据 `messageType` 区分成文本、图片、文件、语音、代码等不同渲染分支。这样一来，消息列表虽然看上去是一串气泡，底层其实已经做了类型分流。

第二部分是 WebSocket 的使用方式。普通聊天页大多依赖 `utils/socket.js` 那条全局连接，这条连接自带自动重连、断线提示和心跳包；聊天室详情页 ChatRoomDetail.vue 又额外维护了一组更重的房间级事件，包括 `join-group`、`online-count`、`ai-stream-chunk`、`join-voice-room` 等。这样拆的结果是，普通会话页比较轻，聊天室则能承载语音、AI 和问题追踪这些更复杂的实时交互。

第三部分是状态管理。项目虽然用了 Pinia，但并没有把所有东西都塞进全局 store。useChatStore.js 只保留了 `currentChatUser` 和按用户区分的消息列表；真正大量的会话列表、未读数、当前房间详情、搜索结果和收藏状态，还是散在各自组件的 `ref` 和自定义事件里。比如群聊消息发送后，前端会发出 `group-message-sent` 事件通知列表刷新；聊天室打开后，再发 `group-chat-opened` 事件清除未读数。这种做法不算很“重框架”，但和当前项目的体量其实比较匹配，读起来也不别扭。

5.3 桌面端实现

桌面端实际上还是同一套前端页面，只是外面加了一层 Electron 封装。所以它和 Web 端最大的不同，不在聊天主流程本身，而在窗口管理、系统托盘、本地通知和 IPC 通信这一层。

5.3.1 系统托盘功能

桌面端主进程主要写在 `ccb/main.js` 中。程序启动后会先创建主窗口，再创建托盘图标。在 Windows 下，如果用户关闭主窗口，程序不会直接退出，而是隐藏到系统托盘继续运行。托盘左键用于显示或隐藏主窗口，右键菜单则提供“打开应用”和“退出应用”两个入口。这个行为和 Web 页面的“关掉浏览器标签页就结束”完全不一样，它更适合挂在后台持续收消息的使用方式。

托盘点击事件会在“显示主窗口”和“隐藏到后台”之间切换，保证程序关闭窗口后仍可保持驻留状态和未读提醒能力。

桌面端在收到新消息后的未读提醒和任务栏反馈效果如图5.15所示。该图能

够说明桌面端并不是简单复用网页界面，而是确实增加了桌面环境下的消息提醒机制。

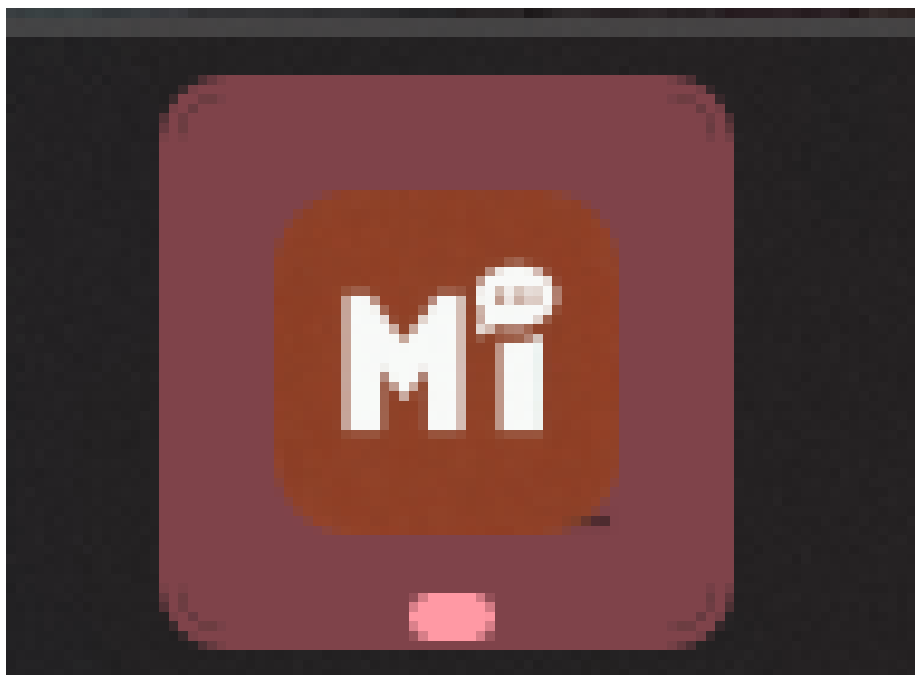


图 5.15 桌面端未读提醒展示

5.3.2 本地通知

本地通知由 `notificationManager.js`、`preload.js` 和 `main.js` 三部分配合完成。渲染进程收到新消息后，会通过 `window.electronAPI.notifyNewMessage()` 把未读数发给主进程；主进程根据平台再去设置任务栏闪烁、Overlay 图标、托盘提示和最近消息预览。用户如果从托盘预览里点击某个会话，主进程还会继续把 `open-chat` 事件发回渲染进程，前端再切到对应私聊或群聊。

渲染进程把未读数和最近消息通过 `electronAPI` 发送给主进程后，主进程再统一负责闪烁提醒、角标更新和托盘预览，从而把桌面通知逻辑与业务页面解耦。

系统托盘的图标、菜单入口与后台驻留形态如图5.16所示。托盘机制使桌面端在关闭主窗口后仍能继续接收消息并保留快速唤起入口。

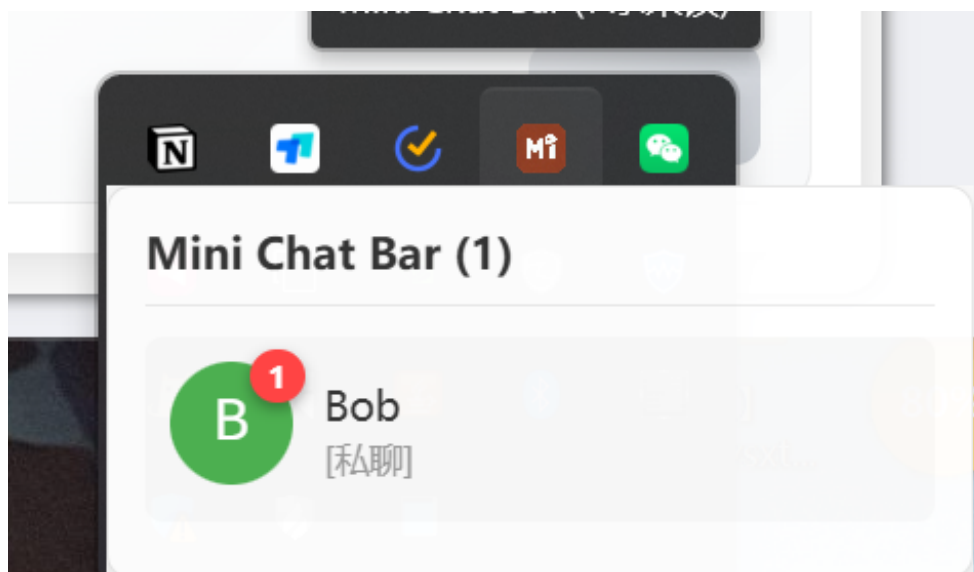


图 5.16 系统托盘展示

5.3.3 与后端和 WebSocket 通信

桌面端和后端通信的基础方式并没有变,还是 Axios 调 REST 接口,Socket.IO 负责实时同步。区别在于,桌面端比 Web 端多了一层 preload.js 暴露出来的 electronAPI,让渲染进程在不直接接触 Node API 的前提下,也能把“新消息提醒”“清空角标”“打开对应会话”这些动作交给主进程处理。

从这个角度看,桌面端并不是另一套独立业务系统,而是把 Web 端原有的聊天逻辑保留下来,再额外补上一层桌面环境能力。这样做的好处比较明显:核心业务代码只维护一份,桌面端主要负责托盘、通知和窗口行为,后续功能更新时不容易和 Web 端分叉太远。

托盘驻留、未读提醒和聊天唤起的代表性实现见附录 I。

6 系统测试

本章基于当前代码、接口、脚本和打包结果，对 Mini-Chat-Bar 的主要功能进行验证。测试对象包括 Web 端、Electron 桌面端、后端接口、Socket.IO 实时链路以及 AI 问答与总结功能。测试结果一方面用于说明系统已经跑通的主链路，另一方面也用于指出当前版本仍存在的边界和问题。

6.1 测试环境与测试方法

6.1.1 测试环境配置

测试在 Windows 11 64 位单机环境完成。后端通过 node server.js 运行在 3000 端口，Web 端通过 Vite 运行在 <http://localhost:5173>，数据库采用本地 MongoDB，桌面端测试对象为 ccb/dist-electron/win-unpacked/Mini Chat Bar.exe。为减少历史数据干扰，测试账号、聊天室和消息样本均以 ch6 前缀重新创建，主要环境配置见表 6.1。

表 6.1 测试环境配置

类别	名称	配置说明
硬件	处理器	12th Gen Intel Core i5-12450H, 12 个逻辑处理器
硬件	内存	16GB 物理内存
硬件	存储环境	本地 SSD 环境，项目代码、数据库与打包产物均部署在同一测试主机
软件	操作系统	Windows 11 64 位，内核版本 10.0.22631.0
软件	Node.js	Node.js 24.14.0, npm 11.9.0
软件	前端框架	Vue 3.5.13, Vite 6.3.5, Pinia 3.0.3, Vue Router 4.5.1
软件	后端框架	Express 5.1.0, Socket.IO 4.8.1, Mongoose 8.15.1
软件	数据库	MongoDB 8.0.13, 本地数据库名为 mini_chat_bar
软件	Redis 环境	Redis 5.0.14.1, 本机已安装，用于辅助验证运行环境完整性
软件	桌面端封装	Electron 36.8.1, 打包产物为 Mini Chat Bar.exe
软件	浏览器	Chrome 146.0.7680.165
软件	辅助工具	Python 3.13.11、Postman、PowerShell、Chrome DevTools、项目内 server/test-chatroom-ai.js 与 server/test-ai-quick.js 脚本
网络与服务	服务端口	Web 端开发地址为 5173 端口，后端接口与 Socket.IO 服务使用 3000 端口
网络与服务	向量检索模式	MessageIndexer 启动成功，采用本地 hash embedding 模式

6.1.2 测试方法与策略

测试分为两部分进行。一部分是面向用户可见链路的黑盒测试，主要检查登录、聊天、聊天室、收藏、AI 和桌面端提醒是否正常；另一部分使用项目现有脚本和接口记录接口耗时、Socket 广播时延以及资源占用，相关脚本包括 server/test-chatroom-ai.js 和 server/test-ai-quick.js。测试顺序按照“先基础链路、后集成链路、再性能验证”展开，先以 /api/user/test 完成烟雾验证，再依次检查认证、消息、收藏、AI 和并发场景，这种先验证主链路、再记录接口耗时与异常结果的安排也与 RESTful 接口测试研究中的常见做法一致^[15]。

6.2 功能测试

6.2.1 功能测试用例设计原则

功能测试优先覆盖五条最常用链路：登录与会话保持、私聊与聊天室消息、AI 问答与总结、收藏管理以及桌面端提醒。每条用例都给出操作步骤、预期结果和实际结果，保证测试结论能够回溯到具体操作。

6.2.2 接口与实时通信测试

这一部分先验证后端接口和 Socket.IO 事件是否能独立工作。测试时使用 ch6userA 与 ch6userB 两个临时账号完成登录、创建技术聊天室，并通过两条 Socket 连接模拟发送端和接收端；对应测试用例如表 6.2 所示。

表 6.2 接口与实时通信测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
IF-01	用户登录接口	调用登录接口提交用户名和密码	返回 200 状态码、JWT 和用户基本信息	接口正常返回 JWT，10 次重复调用平均耗时约 58ms
IF-02	用户信息接口	携带 Token 调用 /api/user/info	返回当前用户标识、昵称、头像和邮箱信息	接口 10 次重复调用平均耗时约 5ms，均能返回完整用户对象
IF-03	无效 Token 拦截	携带无效 Token 调用 /api/user/info	返回未授权结果，不泄露用户信息	接口返回未授权结果，符合权限校验预期

表 6.2（续表）

用例编号	测试功能	操作步骤	预期结果	实际结果
IF-04	聊天室消息发送接口	调用聊天室消息发送接口提交文本消息	返回消息对象，包含消息编号、发送者和时间信息	接口 15ms 内返回成功，消息被写入数据库
IF-05	聊天室消息列表读取	调用消息列表接口读取最近记录	返回消息列表，条数正确且时间顺序正确	列表接口稳定返回 2 条以上记录，时间线与发送顺序一致
IF-06	私聊实时通信事件	发送端触发 private-message，接收端监听同名事件	接收端能够实时收到完整消息内容	接收端在 6ms 内收到测试消息，内容与发送端一致
IF-07	Socket 重连恢复	重连后再次触发 private-message 事件	重连后仍可接收实时消息，且不重复投递	接收端重连后约 8ms 内恢复收消息能力，未出现重复消息
IF-08	AI 问答接口	调用 AI 问答接口提交技术问题	返回结构化 JSON，包含回答正文和历史来源信息	单次请求耗时约 19.5s，返回约 1500 字回答；测试语料较少时历史来源列表为 0

图 6.1 汇总了这一部分的主要测试证据，包括接口返回结果、服务端终端输出、Socket 事件记录和 AI 返回界面，可直接对应表 6.2 中的测试项。

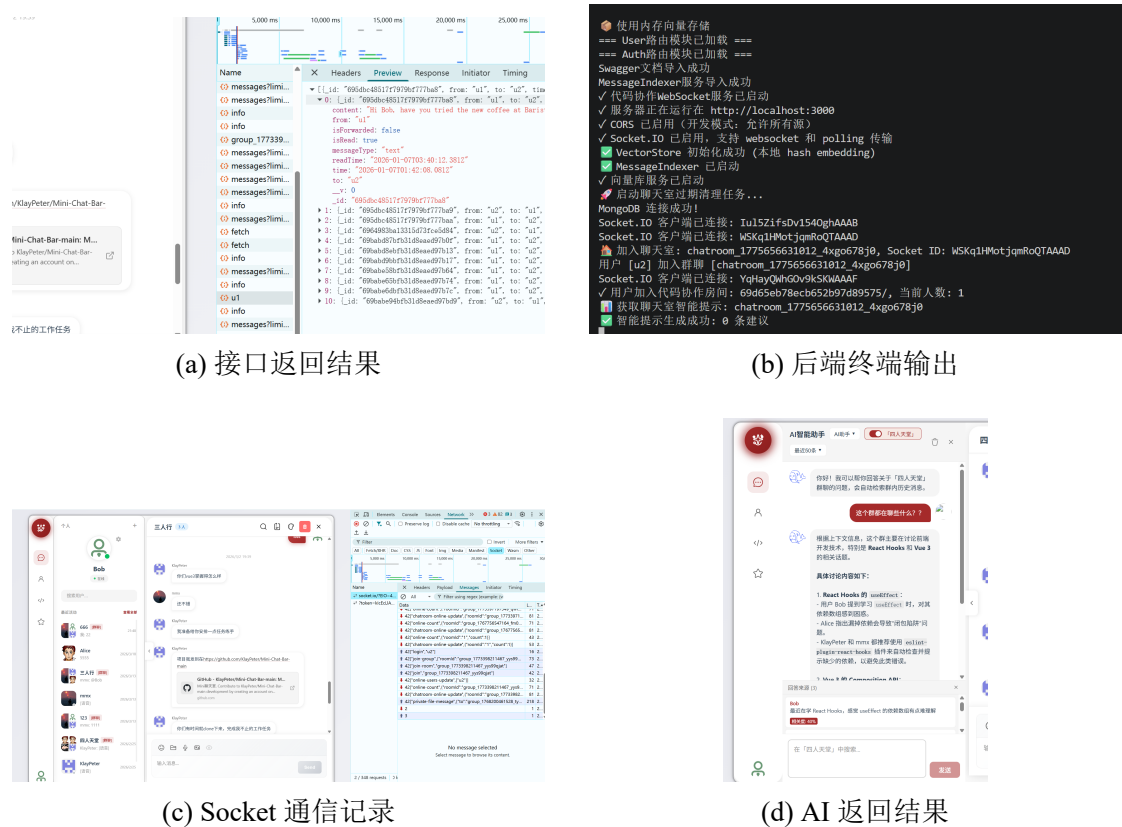


图 6.1 接口与实时通信测试证据组合图

从表 6.2 和图 6.1 可以看出，登录、资料读取、聊天室消息写入和私聊事件投递都已跑通。本地环境下 private-message 接收时延约为 6ms；AI 问答接口能够返回完整结果，但耗时明显高于普通业务接口。

6.2.3 Web 端功能测试

(1) 登录与会话保持测试

Web 端先从登录和会话保持开始验证，因为后续所有页面都依赖这条链路。测试覆盖密码登录、验证码登录、会话保持、错误凭证拦截和未登录路由保护。实测显示，登录后可从 /login 跳转至 /chats 并写入本地会话，刷新后仍能保持登录态；清空本地信息后访问受保护路由会重新跳回登录页，见表 6.3。

表 6.3 Web 端登录功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-01	密码登录	在登录页输入邮箱和密码并提交	页面跳转到主聊天界面，显示当前用户信息	登录成功，主界面正常显示，接口耗时与接口测试结果一致

表 6.3 (续表)

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-02	验证码登录	请求验证码后输入正确验证码并提交	登录成功并建立本地会话	验证码发送成功, 60s 倒计时正常, 登录后完成页面跳转
WEB-03	会话保持	刷新浏览器页面并重新访问主界面	路由守卫识别到有效 Token, 不跳回登录页	刷新后仍停留在主界面, 会话保持正常
WEB-04	错误凭证拦截	输入错误密码后提交表单	登录失败并给出错误提示, 不进入系统主页	页面弹出密码错误提示, 未发生错误跳转
WEB-05	空表单校验	不输入邮箱或密码直接点击登录	前端给出必填提示, 不发起无效登录流程	页面提示“请输入邮箱和密码”, 未进入成功登录流程
WEB-06	未登录路由拦截	直接访问收藏页或聊天室页等受保护路径	自动跳转回登录页并清除残留本地数据	访问受保护路由时立即跳转至 /login, 残留本地数据被清理

这一部分重点记录了本地存储、页面跳转和路由守卫状态, 相关证据统一汇总到图 6.2。测试结果说明登录页、路由守卫和本地会话保持都已正常工作, 错误凭证和空表单也会被前端拦截, 没有出现异常跳转。

(2) 聊天与聊天室功能测试

聊天与聊天室测试直接围绕当前前端界面展开, 验证对象包括 ChatView.vue、Content.vue、ChatRoomListPanel.vue 和 ChatRoomDetail.vue。这一部分重点检查消息是否写库、历史记录是否能回读、房间是否能正常创建和加入, 见表 6.4。

表 6.4 Web 端聊天与聊天室功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-07	私聊消息发送	在聊天界面选择联系人并发送文本消息	对话区新增消息, 后端保存对应私聊记录	消息即时回显, 数据库中生成对应私聊记录

表 6.4（续表）

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-08	私聊历史回读	重新进入会话或刷新页面后加载消息列表	能读取历史消息并按时间顺序显示	刷新后历史消息正常加载，最新消息位于列表末尾
WEB-09	创建公开技术聊天室	在聊天室页填写房间名称、技术方向和公开加入方式后提交	生成新聊天室并自动出现在房间列表中	测试房间创建成功，列表基础回读时可见新建房间
WEB-10	加入密码聊天室	在聊天室列表中选择目标房间并输入密码加入	加入成功并看到历史消息或系统加入提示	输入正确密码后成功进入房间，并生成加入系统消息
WEB-11	聊天室消息拦截	分别发送正常文本和仅空格内容	正常消息成功显示，空消息被拦截不提交	文本消息正常显示，空白输入不会触发发送，消息顺序一致

测试过程中补充记录了私聊回读、公开聊天室创建、密码房间加入和空消息拦截四条关键路径，相关证据统一汇总到图 6.2。结果显示，私聊发送、历史回读、公开房间创建和密码房间加入都能正常完成，空白消息也会在前端被拦截，没有进入后端链路。

（3）AI 问答与总结功能测试

AI 功能测试主要检查三条链路：聊天室内直接提问、输入 @AI 的快捷提问，以及讨论总结生成。为保证结果可重复，测试前先向房间写入 4 条样本消息。实测 AI 问答约 19.5s、总结约 11.8s，刷新后回复仍可回读，见表 6.5。

表 6.5 Web 端 AI 功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-12	AI 问答	在聊天室中提交技术问题并等待返回	页面显示思考状态，随后插入 AI 回复消息	AI 接口调用成功，约 19.5s 后返回约 1500 字回复，页面可继续交互

表 6.5（续表）

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-13	讨论总结	打开总结对话框并触发总结生成	返回结构化总结内容，能够概括主要讨论点	总结生成成功，耗时约 11.8s，基于 5 条消息生成约 600 字摘要
WEB-14	@AI 快捷提问	在输入框中插入 @AI 前缀问题并发送	自动触发 AI 问答并在消息区追加回复	输入 @AI 后先写入用户问题，再生成 AI 回复，流程闭环正常
WEB-15	低语料场景问答	提交一个与当前技术方向相关的问题	仍能返回有效回答，不因历史消息较少而报错	样本消息较少时仍返回有效回答，但历史来源列表为 0
WEB-16	AI 回复持久化	刷新聊天室详情页并重新拉取消息	AI 回复仍可从历史消息中读取并显示	刷新后 AI 回复仍可从历史消息中读取，说明结果已成功落库

测试中补充记录了从提交问题到结果落库的全过程，相关证据统一汇总到图 6.2。从测试结果看，AI 问答、@AI 快捷提问和总结生成都已跑通，且 AI 回复会落库，刷新后仍能回读。当前主要限制仍是耗时受外部模型服务影响，低语料场景下历史来源列表也可能为空。

（4）收藏功能测试

收藏测试围绕添加收藏、状态同步、列表回读、取消收藏和组合筛选展开。该模块与消息列表和聊天室状态都有联动，因此除了接口返回，还需要观察前端回显是否正确。基础收藏与回读均正常，但一次带 `messageType=chatroom` 与关键字的组合查询返回了 500 状态码，见表 6.6。

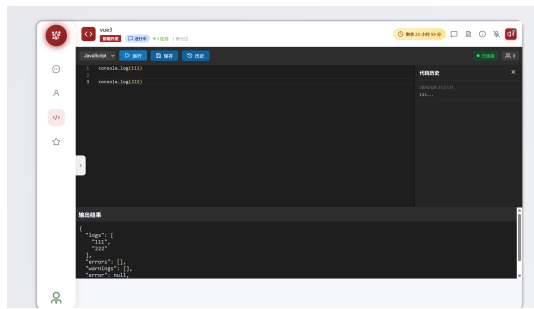
表 6.6 Web 端收藏功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-17	添加收藏	对目标消息执行收藏操作	收藏成功，生成消息快照和标签备注信息	收藏接口调用成功，消息被写入收藏集合

表 6.6（续表）

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-18	收藏状态同步	观察消息右键菜单或收藏状态标记	已收藏消息能够被正确识别与回显	返回消息区后星标状态正确，重新进入聊天室仍能识别已收藏消息
WEB-19	收藏列表回读	进入收藏页读取默认列表	能看到刚收藏的消息记录及来源信息	基础列表查询返回 1 条记录，能够显示测试消息
WEB-20	取消收藏	在收藏详情页点击“取消收藏”并刷新列表	收藏记录被删除，消息恢复未收藏状态	取消后收藏页记录消失，聊天室中的收藏状态同步取消
WEB-21	收藏组合筛选	在收藏页同时使用会话类型和关键字进行筛选	正常返回匹配记录	一次组合查询返回 500 状态码，说明筛选逻辑仍需进一步校验

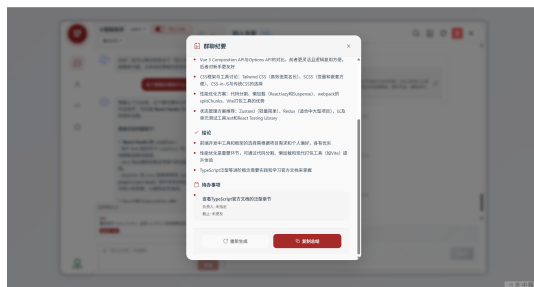
收藏测试重点记录了“收藏动作触发 - 列表回读 - 状态回显 - 取消收藏 - 组合筛选异常”这一闭环，代表性画面统一汇总到图 6.2。其中，聊天室代码协作测试界面对应前端多消息与代码块混排场景，AI 问答结果和讨论总结结果分别对应本节的 WEB-12 至 WEB-16 测试项，设置与主题切换回显则用于补充说明前端交互状态在功能测试过程中的一致性。常规路径已经可用：消息收藏后能在收藏页读到，也能正确取消。问题出现在组合筛选，带 `messageType=chatroom` 和关键字的查询仍然返回 500，这部分还需要单独排查。



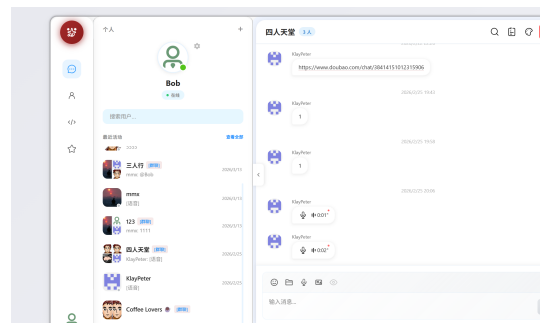
(a) 聊天室代码协作测试



(b) AI 问答结果



(c) 讨论总结结果



(d) 设置与主题切换回显

图 6.2 Web 端功能测试证据组合图

Web 端功能测试说明，登录、聊天、聊天室、AI 和收藏的主链路已经跑通，但收藏复杂筛选仍然是不稳定点。

6.2.4 桌面端功能测试

桌面端测试以 Windows 打包程序为对象，重点不是重新验证一遍所有功能，而是确认 Web 主链路迁移到 Electron 后仍可用，同时检查托盘、角标、预览和点击跳转这些桌面端特有功能，见表 6.7。

表 6.7 桌面端功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
DESK-01	桌面端启动与登录	启动 Mini Chat Bar.exe 并完成登录	主窗口正常加载，登录后进入聊天主界面	程序可正常启动，登录流程与 Web 端一致
DESK-02	托盘驻留	点击关闭按钮观察程序行为	窗口隐藏到托盘而不是直接退出程序	主窗口按设计隐藏，托盘图标仍可保留程序入口

表 6.7 （续表）

用例编号	测试功能	操作步骤	预期结果	实际结果
DESK-03	未读角标与托盘提示更新	触发新消息事件并观察任务栏角标与托盘提示变化	未读数更新，任务栏角标和托盘提示窗口获得焦点后清空角标	随未读数变化
DESK-04	托盘悬停预览	鼠标悬停托盘图标并观察预览窗口	弹出预览窗口，悬停约 300ms 后弹出预览显示最近未读会话摘要	窗，最多展示最近 5 条未读会话
DESK-05	预览点击跳转	点击预览窗口中的某条未读消息	主窗口恢复并定位到目标会话	点击预览项后主窗口被唤起，并定位到对应会话
DESK-06	焦点恢复与角标清零	点击主窗口或让应用重新获得焦点	角标被清空，主窗口恢复焦点后角标被未读状态与界面同步	清空，未读状态与界面保持一致

桌面端部分重点记录了启动、托盘驻留、未读角标变化、悬停预览和点击跳转五个节点。图 6.3 聚焦托盘悬停预览与未读提醒效果，能够直接说明 Electron 侧消息提醒联动正常。

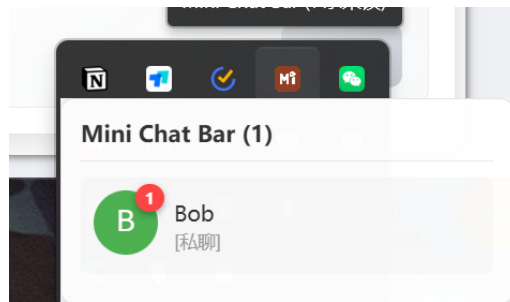


图 6.3 桌面端功能测试证据图

结果表明，桌面端已经具备可用的跨端行为：Web 端现有消息链路可以直接复用，托盘提醒和预览跳转也能正常联动。

6.2.5 功能测试结果总结

综合功能测试结果可以认为，当前版本已经跑通了登录、消息、聊天室、AI 和桌面提醒等主要演示链路。当前最明显的问题仍然是收藏组合筛选异常，以及 AI 回答质量受历史语料数量影响。

6.3 性能测试

6.3.1 响应时间测试

性能测试分为响应时间、并发广播和资源占用三部分。响应时间测试先检查日常接口的返回速度，再记录 AI 问答和聊天室总结这类耗时明显更长的调用，量化结果见表 6.8 和表 6.9。

表 6.8 响应时间测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
PERF-01	登录接口响应	连续调用登录接口 10 次并记录耗时	平均响应时间稳定且无异常失败	10 次调用全部成功，平均耗时约 58ms
PERF-02	用户信息接口响应	连续访问用户信息接口 10 次并记录耗时	查询接口保持毫秒级返回	10 次调用全部成功，平均耗时约 5ms
PERF-03	聊天室列表接口响应	连续访问聊天室列表接口 10 次并记录耗时	列表查询稳定返回	10 次调用全部成功，平均耗时约 5ms
PERF-04	消息列表接口响应	连续访问消息列表接口 10 次并记录耗时	消息读取保持毫秒级返回	10 次调用全部成功，平均耗时约 6ms
PERF-05	收藏列表接口响应	连续访问收藏列表接口 10 次并记录耗时	收藏页查询稳定返回，不出现超时	10 次调用全部成功，平均耗时约 7ms
PERF-06	AI 问答响应	调用 AI 问答接口并记录返回时间	返回完整回答，不出现超时中断	单次耗时约 19.5s，返回约 1500 字内容
PERF-07	聊天总结响应	调用聊天室总结接口并记录返回时间	返回结构化摘要内容	单次耗时约 11.8s，基于 5 条消息生成约 600 字摘要

表 6.9 响应时间测试结果

测试项	平均值	最小值	最大值	结果分析
登录接口	58ms	57ms	61ms	认证、密码比对和 JWT 生成均在可接受范围内完成
用户信息接口	5ms	4ms	6ms	读取单用户资料开销较小，适合高频调用
聊天室列表接口	5ms	4ms	7ms	列表查询结构简单，返回速度稳定
聊天室消息列表接口	6.0ms	5ms	8ms	小规模消息分页读取响应平稳
收藏列表接口	7ms	6ms	9ms	收藏数量较少时查询稳定，来源名补全未带来明显额外开销
聊天室消息发送接口	15ms	—	—	消息写入数据库并广播后仍保持毫秒级返回
私聊消息发送接口	11ms	—	—	私聊接口链路较短，返回速度快
私聊 WebSocket 投递	6ms	—	—	本地实时消息转发几乎无可感知延迟
AI 问答接口	19.5s	—	—	耗时主要来自外部大模型调用，而非本地业务逻辑
聊天室总结接口	11.8s	—	—	总结耗时低于 AI 问答，适合阶段性使用而不适合高频触发

表 6.9 显示，登录、资料、列表和消息接口都保持在毫秒级；AI 问答约为 19.5s，聊天室总结约为 11.8s。当前性能瓶颈并不在本地 Node 服务，而在外部模型调用。

6.3.2 并发性能测试

并发性能测试主要观察 Socket.IO 在单机环境下的广播表现。测试脚本通过 socket.io-client 建立本地连接，统计平均时延、P95 和最大时延，结果见表 6.10 和表 6.11。

表 6.10 并发性能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
CON-01	20 客户端广播	建立 20 个 Socket 客户端并广播一条群消息	所有客户端均能收到消息，广播延迟较低	20 个客户端全部收到消息，成功率 100%

表 6.10 （续表）

用例编号	测试功能	操作步骤	预期结果	实际结果
CON-02	50 客户端广播	建立 50 个 Socket 客户端并广播一条群消息	所有客户端均能收到消息，不出现掉线	50 个客户端全部收到消息，成功率 100%
CON-03	100 客户端广播	建立 100 个 Socket 客户端并广播一条群消息	广播仍能完成，延迟增长保持可控	100 个客户端全部收到消息，成功率 100%
CON-04	100 客户端连续广播	连续发送 10 轮群消息并统计投递情况	多轮广播后仍无掉线和明显积压	10 轮广播共 1000 次投递均成功，未观察到异常断连
CON-05	客户端断线重连后广播	广播新消息并观察重连客户端接收情况	重连客户端可恢复接收消息，在线客户端总数保持稳定	重连客户端可再次收到后续广播，房间在线状态恢复正常

表 6.11 并发性能测试结果

并发客户端数	平均送达时延	P95 时延	最大时延	接收成功率	结果分析
20	3ms	3ms	3ms	100%	小规模广播无明显抖动
50	3ms	3ms	3ms	100%	广播链路稳定，时延未出现明显恶化
100	4ms	5ms	5ms	100%	单机环境下仍可稳定广播，时延有所上升但整体可控

如图 6.4所示，并发广播测试示意图展示了单机环境下 100 客户端广播的主要观测指标。

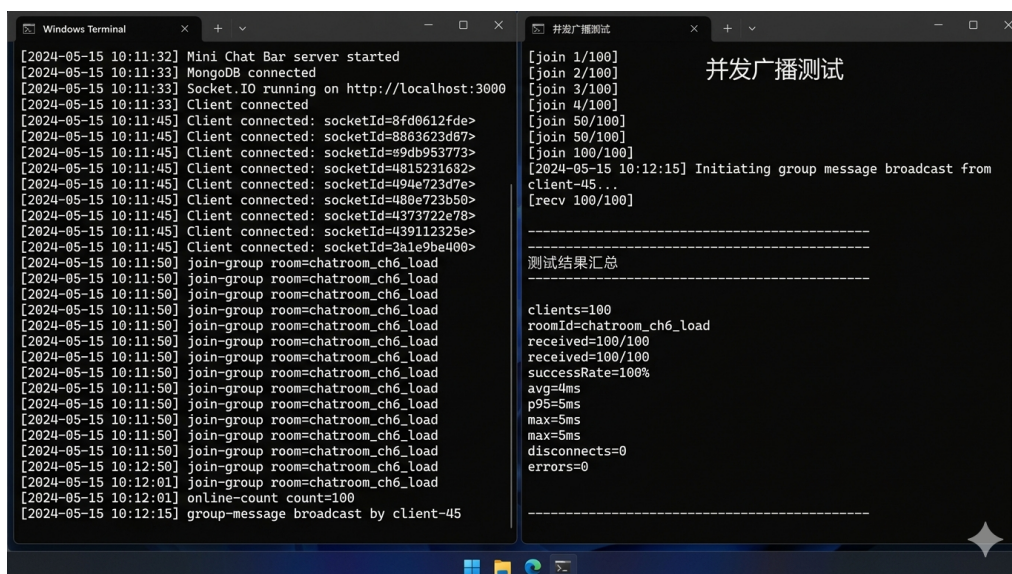


图 6.4 并发广播测试日志示意图

在 20、50 和 100 个客户端下，广播成功率均为 100%。单机 100 客户端时平均时延约为 4ms，说明当前规模下广播链路仍然稳定，但这一结果只代表本地环境。

6.3.3 资源占用测试

资源占用测试分别记录 100 客户端广播、单次 AI 问答和单次聊天室总结三种场景下的 CPU 与内存变化，结果见表 6.12 和表 6.13。

表 6.12 资源占用测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
RES-01	广播场景资源占用	建立 100 个 Socket 客户端并完成一次群消息广播	CPU 和内存占用保持在可控范围内，不出 现进程异常退出	广播测试完成，进程工 作集与私有内存均低于 200MB
RES-02	AI 问答场景资源占用	对聊天室发起一次完整 AI 问答请求并记录进程占用	在等待 AI 返回期间服务进程保持稳定	AI 请求完成后进程保 持正常，内存占用低于 120MB

表 6.12 （续表）

用例编号	测试功能	操作步骤	预期结果	实际结果
RES-03	AI 总结场景资源占用	对聊天室发起一次完整讨论总结请求并记录进程占用	总结生成期间服务进程稳定，无明显资源抖动	总结请求完成后进程保持正常，内存占用低于 115MB

表 6.13 资源占用测试结果

测试场景	持续时间	平均 CPU 占用	工作集内存	私有内存	结果分析
100 客户端广播	1.5s	2.1%	161MB	187MB	连接数增长后内存占用上升，但仍保持在较低水平
单次 AI 问答	31.0s	低于 1%	93MB	118MB	CPU 占用不明显，说明耗时主要消耗在等待外部模型返回
单次 AI 总结	12.2s	低于 1%	90MB	114MB	与 AI 问答相比总体占用接近，但等待时间更短

如图 6.5 所示，资源占用监测示意图展示了广播压力和 AI 请求压力下的服务端状态差异。

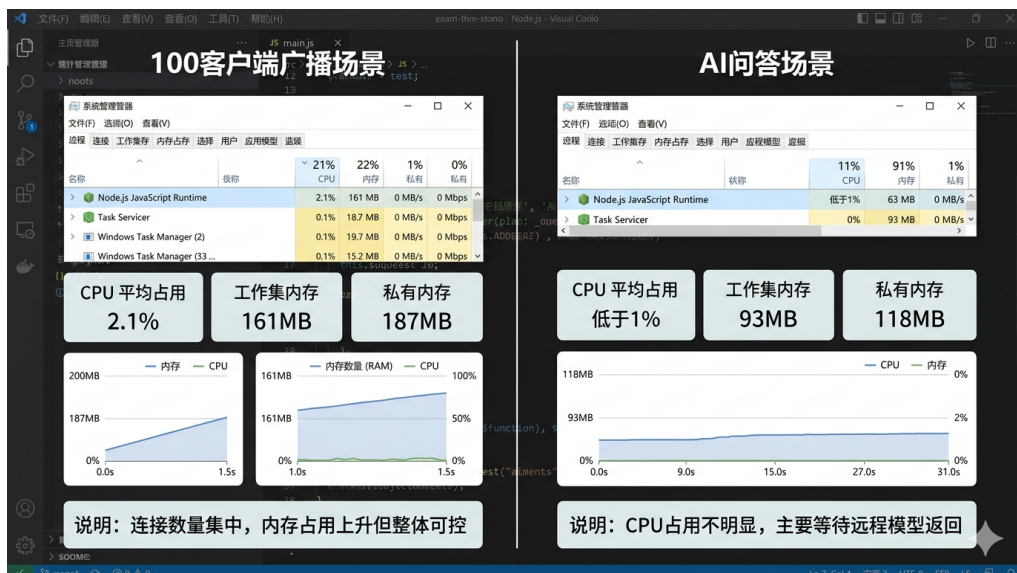


图 6.5 资源占用监测示意图

资源监测结果说明，广播场景主要抬高内存占用，AI 场景则更多消耗等待时间。本地服务端 CPU 压力不高，说明 AI 耗时主要来自等待外部模型返回。

6.3.4 性能测试结果总结

综合响应时间、并发广播和资源占用结果，系统在当前单机部署下已经能满足课程项目和答辩演示。普通接口、消息列表读取和 **Socket** 广播都保持了较稳定的响应水平，说明当前前后端主链路在小规模本地环境下可以支撑连续展示与演示操作。

从性能瓶颈分布看，当前最主要的耗时来源仍然是外部模型调用，而不是本地 **Node** 服务与数据库访问；从资源曲线看，广播场景主要推高内存占用，**AI** 场景则更多体现在等待时间拉长。后续需要优先处理的仍是 **AI** 耗时和收藏复杂筛选这两个边界问题，同时也应继续关注更高并发和更长时段运行下的稳定性变化。

7 总结与展望

7.1 工作总结

到论文撰写完成时，Mini-Chat-Bar 已经完成了从需求分析、系统设计到实现与测试的主要工作。这个系统面向的不是泛社交聊天，而是技术交流场景里更常见的几条链路：用户登录后可以进入主聊天界面，在私聊和聊天室之间切换；消息支持文本、代码、图片和文件；需要反复回看的内容可以加入收藏；讨论过程中如果需要补充说明或快速整理结论，还可以调用 AI 问答和讨论总结功能。

结合整个毕业设计的推进过程，我完成的工作主要集中在几个连续阶段。前期先梳理需求、确定论文结构，并在此基础上把系统定为前后端分离架构；随后围绕用户、私聊、聊天室、收藏、AI 对话与聊天总结等模块整理业务流程和数据结构；在实现阶段继续完成 Web 端界面、Electron 桌面端适配、后端接口、Socket.IO 实时链路以及 AI 问答与总结功能的联调；最后再把需求分析、系统设计、实现过程和测试结果整理成论文。对我来说，这次毕设不是只把一个功能做出来，而是把分析、设计、编码、测试和文档整理完整地走了一遍。

从当前实现结果看，系统的主要链路已经可以闭环运行。前端已经完成登录、主聊天界面、聊天室详情、收藏页和 AI 面板等核心页面；后端已经完成用户认证、消息处理、聊天室管理、收藏管理和 AI 流式问答等服务；MongoDB 用于保存用户、房间、私聊消息、聊天室消息、收藏和 AI 相关数据。具体到业务流程上，邮箱登录后可以进入 ChatView.vue，私聊通过 `/api/chat/messages/:id` 与 `private-message` 联动，聊天室消息通过 `/room/:roomId/messages` 和 `group-message` 同步，@AI 提问则通过 `/api/chatroom-ai/ask-stream` 与 `ai-stream-chunk` 回写结果。

第六章的测试结果也说明，这套系统已经具备基本的演示和使用条件。普通接口保持毫秒级响应，私聊消息和聊天室消息可以稳定写入并同步，桌面端的托盘驻留、未读提醒和消息预览已经能够正常工作。AI 问答与总结链路虽然明显慢于普通接口，但已经跑通了从提问、检索、生成到结果落库的完整流程。对本科毕业设计而言，系统已经完成了预期的主要目标。

7.2 实践体会

这次毕业设计让我更直接地体会到，聊天系统真正难的地方往往不在某一个页面或者某一个接口本身，而在于多条链路同时工作时能不能保持一致。私聊和聊天室都不是前端把消息回显出来就算结束，还要继续确认消息有没有写入

数据库、Socket 有没有及时推送、刷新页面后历史记录能不能重新读回来。桌面端提醒也是同样的问题，弹出通知只是最表面的结果，后面还牵涉未读数、托盘状态和窗口焦点变化。如果只看单独一个点，很多问题都不算复杂；放到完整系统里之后，前后端联动和状态同步的影响就会一起出现。

另一个比较深的体会来自 AI 模块。开始做之前，很容易把它理解成“接入一个模型接口”，真正落到系统里才发现关键并不只在模型返回内容本身，而在于历史消息有没有进入索引、检索结果能不能贴近当前讨论、AI 回复生成后能不能继续以普通消息的形式保存和回看。聊天室里的 @AI 不是一个孤立入口，它必须和现有消息流共存，这一点和对话历史管理型问答系统强调的上下文连续性是一致的^[14]。这一部分做完以后，我对 AI 接入业务系统的认识比之前具体得多，也更清楚模型能力、消息数据和场景设计之间是如何互相影响的。

从个人收获来看，这次毕设最大的价值在于把一个完整项目真正从头做到尾。以往课程作业通常围绕单个功能展开，范围比较有限；这次需要同时考虑需求分析、系统设计、代码实现、测试验证和结果说明。做完整个过程后，我对接口设计、实时通信、数据库建模、前端状态管理以及问题定位都有了更直观的认识，这些经验比单独完成某一页界面更有价值。

7.3 系统不足与展望

当前版本已经能够支撑课程项目演示和小规模使用，但距离长期稳定运行的技术交流平台还有明显差距。首先，系统目前主要在本地和单机环境下开发与测试，更适合原型验证。如果后续用户数量继续增加，服务端在并发连接、消息广播和 AI 请求调度上的压力会更明显。下一步如果继续扩展，部署方式、缓存机制和服务拆分都需要重新考虑。

其次，AI 模块虽然已经跑通了问答和总结的基本链路，但它对历史语料数量和检索结果质量依赖较强。当聊天室消息较少，或者检索结果不够准确时，回答仍然会偏通用；长上下文利用效率不稳定时，这个问题还会更明显^[22]。后续更值得投入的方向，不是单纯再换一个模型，而是继续优化消息索引、检索排序和上下文拼接策略，让 AI 返回内容更贴近房间里的真实讨论^[19]。即便模型能力继续扩展，大语言模型在业务系统中的稳定性和可控性仍然需要长期处理^[23]。

再次，系统在稳定性方面还有需要补强的地方。第六章测试已经发现，收藏模块在组合筛选条件下仍会出现 500 错误。这说明主路径虽然已经跑通，但复杂路径下的异常处理还不够扎实。相比继续叠加新功能，后续更应该优先补上异常

处理、回归测试和日志分析，把现有链路先做稳。

最后，从产品方向看，当前系统已经覆盖了聊天、收藏和 AI 辅助三条主线，但对技术团队更深层次的协作支持还不够。例如，多人代码协同、任务跟踪、知识归档的结构化整理等能力，目前还没有形成完整方案。后续如果继续推进，可以在保持轻量化的前提下逐步补充这些能力，使系统从“技术讨论工具”进一步发展为“面向技术交流的协作平台”。

总体来看，Mini-Chat-Bar 已经验证了一个比较清楚的方向：技术交流系统不应只负责消息传递，还应支持历史回看、结果沉淀和适度的 AI 辅助。当前版本还算不上成熟产品，但主框架已经建立起来，后续只要继续围绕稳定性、检索质量和协作深度推进，系统仍然有较大的完善空间。

参考文献

- [1] 韩玉婷, 叶自强. 文本型聊天系统的设计与实现[J]. 计算机软件及计算机应用, 2024: 1-5.
- [2] 任海玉, 刘建平, 王健, 等. 基于大语言模型的智能问答系统研究综述[J]. 计算机工程与应用, 2025, 61(7): 1-24.
- [3] 龙举兵, 罗忠巍. HTTP 轮询、SSE 和 WebSocket 的 IoT 的 Web 管理系统的研究和应用[J]. 互联网技术, 2026(2): 1-6.
- [4] OpenAI. GPT-4 Technical Report[J]. arXiv preprint arXiv:2303.08774, 2023.
- [5] 刘雪颖, 云静, 李博, 等. 基于大型语言模型的检索增强生成综述[J]. 计算机工程与应用, 2025, 61(13): 1-25.
- [6] 周思源. 针对开发者聊天室对话的摘要方法研究[D]. 浙江工商大学, 2025.
- [7] Vue.js Team. Vue.js Guide[EB/OL]. Vue.js(2026)[2026-03-29]. <https://vuejs.org/guide/introduction.html>.
- [8] Pinia Team. Pinia Introduction[EB/OL]. Pinia(2026)[2026-03-29]. <https://pinia.vuejs.org/introduction.html>.
- [9] Express.js Team. Express 5.x API Reference[EB/OL]. Express.js(2026)[2026-03-29]. <https://expressjs.com/en/5x/api.html>.
- [10] Socket.IO Team. Socket.IO Documentation[EB/OL]. Socket.IO(2026)[2026-03-29]. <https://socket.io/docs/v4/>.
- [11] Lodderstedt T, Fett D, Campbell B, et al. The OAuth 2.0 Authorization Framework: Security Best Current Practice[EB/OL]. Internet Engineering Task Force(2025)[2026-03-29]. <https://www.rfc-editor.org/rfc/rfc9700.html>.
- [12] MongoDB, Inc. MongoDB Manual: Data Modeling Introduction[EB/OL]. MongoDB(2026)[2026-03-29]. <https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>.
- [13] 李仲岐. 基于 WebSocket 即时通信系统设计与实现[C]//第三十七届中国(天津)2023'IT、网络、信息技术、电子、仪器仪表创新学术会议论文集. 天津, 2023: 1-5.
- [14] 甘克勤, 高亮, 肖宝坤, 等. 基于检索增强生成与对话历史管理的标准智能问答系统研究[J]. 计算机应用, 2026: 1-6.
- [15] 孙业超. 基于 RESTful API 的前后端分离项目接口测试方法研究[J]. 互联网技术, 2025: 1-5.

- [16] 张龙. 基于国密 SM2/SM3 算法的保密即时通信系统设计[J]. 网络安全技术与应用, 2024 (7): 25-27.
- [17] 杨小军, 刘威. 主流安全身份验证和授权技术之 OAuth2.0 研究[J]. 网络安全技术与应用, 2024(4): 9-12.
- [18] Electron Maintainers. Electron Documentation[EB/OL]. Electron(2026)[2026-03-29]. <https://www.electronjs.org/docs/latest/>.
- [19] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[J]. arXiv preprint arXiv:2312.10997, 2024.
- [20] 张强, 高颖, 任豆豆. 融合 DeepSeek-R1 和 RAG 技术的先秦文化元典智能问答研究[J]. 现代情报, 2026, 46(1): 173-186.
- [21] 荣国伟, 孙宝, 姜旭磊, 等. 基于大语言模型的算法平台构建与实现[J]. 网络安全技术与应用, 2024(7): 48-51.
- [22] Liu N F, Lin K, Hewitt J, et al. Lost in the Middle: How Language Models Use Long Contexts [J]. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173.
- [23] Zhao W X, Zhou K, Li J, et al. A Survey of Large Language Models[J]. arXiv preprint arXiv:2303.18223, 2023.

致谢

行文至此，这篇论文也接近尾声，大学四年的时光仿佛也在这一页页文字里慢慢落定。回望从选题、设计、实现到反复修改的过程，我并不是独自走到这里。一路上，老师的指引、同学的陪伴与帮助，让许多原本紧张而忙碌的时刻，都多了几分踏实与从容。

衷心感谢我的指导老师刘利副教授，在论文选题、系统设计和写作修改过程中给予我耐心而细致的指导。老师严谨认真的治学态度，也让我明白，完成毕业设计不仅是完成一项任务，更是学着把问题想清楚、把事情做扎实。

同时，也感谢身边的同学和朋友。在系统开发和论文撰写过程中，大家陪我一起讨论思路、排查问题、完善细节。许多看似平常的交流与提醒，都会在回头看时显出分量，它们像夜里一直亮着的灯，让这段毕业时光不只是匆忙，也留下了温度。

在此，向所有曾给予我帮助、鼓励与关心的老师、同学和朋友，致以真诚的谢意。那些一起思考、一起赶进度、一起把问题慢慢解决的日子，也会成为我大学生活中很珍贵的一部分。

附录

本附录 A 至附录 I 主要对应第 5 章“系统实现”中的关键功能模块，具体覆盖第 5 章第 1 节“后端实现”、第 2 节“Web 前端实现”和第 3 节“桌面端实现”的代表性源码片段。为控制篇幅，附录仅保留能够说明实现方式的核心代码，每个模块只选取必要部分。

附录 A 用户认证与登录模块核心源码

对应正文：第 5 章第 1 节“后端实现”中“用户模块”部分。

源码位置：server/controllers/UserController.js

注：这一段补到验证码登录分支开始前，便于连续观察邮箱格式校验、密码比对、JWT 下发和异常返回的完整收尾。

```
// 邮箱密码登录
static async loginWithEmail(req, res) {
  const { email, password } = req.body;

  try {
    // 验证邮箱格式
    if (!PasswordValidator.validateEmail(email)) {
      return res.status(400).json({ message: "请输入有效的邮箱地址" });
    }

    const user = await Users.findOne({ uEmail: email });

    if (user) {
      const isMatch = await bcrypt.compare(password, user.Password);
      if (isMatch) {
        const token = jwt.sign(
          { userId: user.uID, username: user.uName, email: user.uEmail },
          SECRET_KEY,
          { expiresIn: "24h" }
        );
        res.status(200).json({
          message: "登录成功",
          token: token,
          user: {
            id: user.uID,
            name: user.uName,
            email: user.uEmail,
            ava: user.uAvatar,
          },
        });
      } else {
        res.status(401).json({ message: "密码错误" });
      }
    } else {

```

```
    res.status(404).json({ message: " 邮箱未注册" });
  }
} catch (err) {
  console.error(" 登录失败", err);
  res.status(500).json({ message: " 服务器内部错误" });
}
}

// 邮箱验证码登录
static async loginWithVerificationCode(req, res) {
  const { email, verificationCode } = req.body;

  try {
    // 验证必填字段
    if (!email || !verificationCode) {
      return res.status(400).json({ message: " 邮箱和验证码不能为空" });
    }

    // 验证邮箱格式
    if (!PasswordValidator.validateEmail(email)) {
      return res.status(400).json({ message: " 请输入有效的邮箱地址" });
    }

    // 验证验证码
    const codeVerification = EmailService.verifyCode(email, verificationCode, 'login');
    if (!codeVerification.success) {
      return res.status(400).json({ message: codeVerification.message });
    }
  }
}
```

附录 B 私聊消息收发模块核心源码

对应正文：第 5 章第 1 节“后端实现”中“聊天模块”部分。

源码位置：server/controllers/ChatController.js

注：片段向后扩展到单条消息删除逻辑，方便将历史读取、消息写入和权限校验放在同一段里阅读。

```
// 获取聊天消息列表
static async getMessages(req, res) {
  try {
    const { id } = req.params;
    const currentUserId = req.user.userId;

    const messages = await Msg.find({
      $or: [
        { from: currentUserId, to: id },
        { from: id, to: currentUserId }
      ]
    }).sort({ time: 1 });

    res.status(200).json(messages);
  } catch (err) {
    console.error("获取聊天消息失败", err);
    res.status(500).json({ message: "服务器内部错误" });
  }
}

// 发送消息
static async sendMessage(req, res) {
  try {
    const { id } = req.params;
    const { content, messageType = 'text', fileInfo, isForwarded = false, forwardedFrom } = req.body;
    const currentUserId = req.user.userId;

    const newMessage = new Msg({
      from: currentUserId,
      to: id,
      content: content,
      messageType: messageType,
      time: new Date(),
      fileInfo: fileInfo,
    });
```

```
isForwarded: isForwarded,
forwardedFrom: forwardedFrom,
isRead: false // 新消息默认为未读
});

const savedMessage = await newMessage.save();
res.status(200).json({
  message: " 消息发送成功",
  data: savedMessage
});
} catch (err) {
  console.error(" 发送消息失败", err);
  res.status(500).json({ message: " 服务器内部错误" });
}
}

// 删除单条消息
static async deleteMessage(req, res) {
  try {
    const { messageId } = req.params;
    const currentUserId = req.user.userId;

    const message = await Msg.findById(messageId);
    if (!message) {
      return res.status(404).json({ message: " 消息不存在" });
    }

    if (message.from !== currentUserId && message.to !== currentUserId) {
      return res.status(403).json({ message: " 无权删除此消息" });
    }

    await Msg.findByIdAndDelete(messageId);
    res.status(200).json({ message: " 消息删除成功" });
  } catch (err) {
    console.error(" 删除消息失败", err);
    res.status(500).json({ message: " 服务器内部错误" });
  }
}
```

附录 C 技术聊天室创建模块核心源码

对应正文：第 5 章第 1 节“后端实现”中“聊天模块”部分。

源码位置：server/controllers/GroupController.js

注：这里继续保留创建房间后的列表返回部分，使邀请码、有效期、系统消息和聊天室在线人数补全为一条完整业务链。

```
exports.createGroup = async (req, res) => {
  try {
    const { groupName, memberIds, type, techDirection, joinType, password, announcement, duration } = req.body
    const creator = req.user.userId

    // 获取创建者信息
    const creatorInfo = await Users.findOne({ uID: creator })
    if (!creatorInfo) {
      return res.status(404).json({ message: '用户不存在' })
    }

    // 生成唯一的群 ID
    const prefix = type === 'chatroom' ? 'chatroom' : 'group'
    const roomId = `${prefix}_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`

    // 构建成员列表（包含创建者）
    const members = [{
      userID: creator,
      Nickname: creatorInfo.uName,
      Avatar: creatorInfo.uAvatar,
      joinedAt: new Date()
    }]

    // 添加其他成员
    if (memberIds && memberIds.length > 0) {
      for (const memberId of memberIds) {
        if (memberId !== creator) {
          const memberInfo = await Users.findOne({ uID: memberId })
          if (memberInfo) {
            members.push({
              userID: memberId,
              Nickname: memberInfo.uName,
              Avatar: memberInfo.uAvatar,
              joinedAt: new Date()
            })
          }
        }
      }
    }
  }
}
```

```
    })
  }
}
}
}

// 生成邀请码（如果需要）
let inviteCode = ""
if (joinType === 'invite') {
  inviteCode = Math.random().toString(36).substr(2, 8).toUpperCase()
}

// 计算过期时间（仅聊天室有时间限制）
let expiresAt = null
const roomDuration = duration || 24 // 默认 24 小时
if (type === 'chatroom') {
  expiresAt = new Date(Date.now() + roomDuration * 60 * 60 * 1000)
}

// 创建群聊/聊天室
const newRoom = new Room({
  RoomID: roomId,
  RoomName: groupName,
  Creator: creator,
  Admins: [creator],
  Members: members,
  type: type || 'normal',
  techDirection: techDirection || "",
  joinType: joinType || 'public',
  inviteCode: inviteCode,
  password: password || "",
  Announcement: announcement || "",
  duration: type === 'chatroom' ? roomDuration : undefined,
  expiresAt: expiresAt
})

await newRoom.save()

// 创建系统消息
const systemMessage = new GroupMessage({
  roomId: roomId,
  from: 'system',
  fromName: '系统消息',
```



```
content: type === 'chatroom'
  ? `${creatorInfo.userName} 创建了技术聊天室，将在 ${roomDuration} 小时后自动解散`
  : `${creatorInfo.userName} 创建了群聊`,
messageType: 'system'
}))
await systemMessage.save()

res.json({
  success: true,
  message: type === 'chatroom' ? '聊天室创建成功' : '群聊创建成功',
  room: newRoom
})
} catch (err) {
  console.error('创建群聊失败:', err)
  res.status(500).json({ message: '创建群聊失败', error: err.message })
}
}

// 获取用户的所有群聊
exports.getUserGroups = async (req, res) => {
  try {
    const userId = req.user.userId

    const groups = await Room.find({
      'Members.userId': userId,
      type: { $ne: 'chatroom' } // 排除聊天室
    }).sort({ updatedAt: -1 })

    res.json({
      success: true,
      groups: groups
    })
  } catch (err) {
    console.error('获取群聊列表失败:', err)
    res.status(500).json({ message: '获取群聊列表失败' })
  }
}

// 获取用户的所有技术聊天室
exports.getChatRooms = async (req, res) => {
  try {
    const userId = req.user.userId
```

```
const rooms = await Room.find({
  $or: [
    { 'Members.userID': userId, type: 'chatroom' },
    { type: 'chatroom', joinType: 'public' },
    { type: 'chatroom', joinType: 'password' } // 密码方式的聊天室也显示在列表中
  ]
}).sort({ updatedAt: -1 })

// 获取在线人数（从 roomUsers Map）
const roomSocketModule = require('../sockets/room')
const roomUsers = roomSocketModule.roomUsers

const roomsWithOnlineCount = rooms.map(room => {
  const roomObj = room.toObject()
  // 获取该房间的在线用户数（按 userId 去重）
  roomObj.onlineCount = roomUsers.has(room.RoomID) ? roomUsers.get(room.RoomID).size : 0
  return roomObj
})

res.json({
  success: true,
  rooms: roomsWithOnlineCount
})
```

附录 D 聊天室 AI 流式回复核心源码

对应正文：第 5 章第 1 节“后端实现”中“AI 智能检索与流式交互（RAG & SSE）”部分。

源码位置：server/controllers/ChatRoomAIController.js

代码片段 1：增量结果解析与分块回写

注：第一段向前补入流式请求参数，便于看清外部模型返回的数据如何被逐块解析，并同步写回 *SSE* 与房间广播。

```
const aiResponse = await axios.post(
  DEEPSEEK_API_URL,
  {
    model: 'deepseek-chat',
    messages: [
      { role: 'system', content: '你是一个专业的技术助手，擅长解答编程和技术问题。' },
      { role: 'user', content: context }
    ],
    temperature: 0.7,
    max_tokens: 2000,
    stream: true
  },
  {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${DEEPSEEK_API_KEY}`
    },
    responseType: 'stream',
    timeout: 60000
  }
)

let fullAnswer = ""
const io = req.app.get('io')

aiResponse.data.on('data', (chunk) => {
  const lines = chunk.toString().split('\n').filter(line => line.trim() !== "")

  for (const line of lines) {
    if (line.startsWith('data: ')) {
      const data = line.slice(6)
```

```

    if (data === '[DONE]') {
      return
    }

    try {
      const parsed = JSON.parse(data)
      const content = parsed.choices[0]?.delta?.content || ""
      if (content) {
        fullAnswer += content
        res.write(`data: ${JSON.stringify({ content })}\n\n`)

        // 通过 Socket.IO 实时推送增量内容
        if (io) {
          io.to(roomId).emit('ai-stream-chunk', {
            roomId,
            content,
            isComplete: false
          })
        }
      }
    } catch (e) {
      // 忽略解析错误
    }
  }
}

aiResponse.data.on('end', async () => {

```

代码片段 2： AI 消息落库与房间广播

注：第二段补足结束事件和错误分支，用来说明流式回答生成完毕后如何落库、广播和回传来源信息。

```

// 保存 AI 消息到数据库
const aiMessage = new GroupMessage({
  roomId: roomId,
  from: 'AI',
  fromName: 'AI 助手',
  fromAvatar: '/images/ai-avatar.png',
  content: fullAnswer,
  messageType: 'text',

```

```
        time: new Date()
    })

    await aiMessage.save()

    // 通过 Socket.IO 推送完整消息
    if (io) {
        io.to(roomId).emit('group-message', {
            roomId: roomId,
            from: 'AI',
            fromName: 'AI 助手',
            fromAvatar: '/images/ai-avatar.png',
            content: fullAnswer,
            messageType: 'text',
            time: new Date(),
            _id: aiMessage._id,
            isAI: true
        })
    }

    res.write(`data: ${JSON.stringify({
        done: true,
        sources,
        messageId: aiMessage._id
    })}\n\n`)
    res.end()
})

aiResponse.data.on('error', (error) => {
    console.error('流式响应错误:', error)
    res.write(`data: ${JSON.stringify({ error: '流式响应失败' })}\n\n`)
    res.end()
})

} catch (err) {
    console.error('AI 问答失败 (流式):', err)
    res.write(`data: ${JSON.stringify({ error: 'AI 问答失败' })}\n\n`)
    res.end()
}
```

附录 E 聊天室实时通信与在线状态核心源码

对应正文：第 5 章第 1 节“后端实现”中“WebSocket 模块”部分。

源码位置：server/sockets/room.js

注：这一段延长到群消息广播结束，能够连续展示入房、离房、在线人数维护、消息头像补全和索引队列写入。

```
// 加入群聊房间
socket.on("join-group", async ({ roomId, userId }) => {
  try {
    socket.userId = userId
    socket.currentRoom = roomId

    // 加入 Socket.IO 房间
    socket.join(roomId)

    // 记录用户连接 - 使用 userId 而不是 socket.id
    if (!roomUsers.has(roomId)) {
      roomUsers.set(roomId, new Set())
    }
    roomUsers.get(roomId).add(userId)

    console.log(`用户 [${userId}] 加入群聊 [${roomId}]`)

    // 通知其他成员
    socket.to(roomId).emit("member-joined", {
      userId: userId,
      timestamp: new Date()
    })

    // 返回当前在线成员数（按 userId 去重）
    const onlineCount = roomUsers.get(roomId).size
    io.to(roomId).emit("online-count", { roomId: roomId, count: onlineCount })

    // 同时广播给所有客户端（用于更新聊天室列表）
    io.emit("chatroom-online-update", { roomId: roomId, count: onlineCount })

  } catch (err) {
    console.error("加入群聊失败:", err)
    socket.emit("error", { message: "加入群聊失败" })
  }
})
```

```
  })

  // 离开群聊房间
  socket.on("leave-group", ({ roomId, userId }) => {
    socket.leave(roomId)

    // 从房间用户列表中移除该用户
    if (roomUsers.has(roomId)) {
      roomUsers.get(roomId).delete(userId)
      if (roomUsers.get(roomId).size === 0) {
        roomUsers.delete(roomId)
      }
    }
  })

  console.log(`用户 [${userId}] 离开群聊 [${roomId}]`)

  // 通知其他成员
  socket.to(roomId).emit("member-left", {
    userId: userId,
    timestamp: new Date()
  })

  // 更新在线人数
  const onlineCount = roomUsers.has(roomId) ? roomUsers.get(roomId).size : 0
  io.to(roomId).emit("online-count", { roomId: roomId, count: onlineCount })

  // 同时广播给所有客户端（用于更新聊天室列表）
  io.emit("chatroom-online-update", { roomId: roomId, count: onlineCount })
})

// 发送群消息
socket.on("group-message", async (data) => {
  try {
    // 如果消息中没有头像信息，从数据库获取
    let fromAvatar = data.fromAvatar
    if (!fromAvatar && data.from && data.from !== 'system') {
      try {
        const Users = require('../models/Users')
        const user = await Users.findOne({ uID: data.from })
        if (user) {
          fromAvatar = user.uAvatar
        }
      } catch (err) {
```

```
        console.error(' 获取用户头像失败:', err)
      }
    }

    // 广播数据，确保包含头像
    const broadcastData = {
      ...data,
      fromAvatar: fromAvatar || '/images/avatar/default-avatar.webp',
      timestamp: new Date()
    }

    io.to(data.roomId).emit("group-message", broadcastData)

    // 异步索引消息（不阻塞消息发送）
    if (data.messageType === 'text' && data.content) {
      messageIndexer.addToQueue({
        content: data.content,
        senderName: data.senderName,
        from: data.senderId,
        roomId: data.roomId,
        time: broadcastData.timestamp,
        messageType: 'text'
      });
    }

  } catch (err) {
    console.error(" 发送群消息失败:", err)
    socket.emit("error", { message: " 发送消息失败" })
  }
})
```


附录 F 消息收藏写入模块核心源码

对应正文：第 5 章第 1 节“后端实现”中“数据模型与辅助模块”部分。

源码位置：server/controllers/FavoriteController.js

代码片段 1：收藏快照写入

注：第一段保留完整写入流程，包括重复收藏判断、原消息回查、发送者信息组装以及附件快照保留。

```
static async addFavorite(req, res) {
  try {
    const userId = req.user.userId
    const { messageId, messageType, chatId, tags, note } = req.body

    console.log('收藏请求:', { userId, messageId, messageType, chatId })

    if (!messageId || !messageType || !chatId) {
      return res.status(400).json({ message: '缺少必要参数' })
    }

    // 检查是否已收藏
    const existing = await Favorite.findOne({ userId, messageId })
    if (existing) {
      return res.json({
        success: true,
        message: '已收藏',
        favorite: existing
      })
    }

    // 获取消息详情
    let messageData
    console.log('查找消息:', messageType, messageId)

    if (messageType === 'private') {
      messageData = await Message.findById(messageId)
    } else if (messageType === 'group' || messageType === 'chatroom') {
      messageData = await GroupMessage.findById(messageId)
    }

    console.log('消息数据:', messageData ? '找到' : '未找到')
```

```
if (!messageData) {
  return res.status(404).json({ message: '消息不存在' })
}

// 创建收藏
const favoriteData = {
  userId,
  messageId,
  messageType,
  chatId,
  content: messageData.content || messageData.fileInfo?.fileName || '无内容',
  contentType: messageData.messageType || 'text',
  codeInfo: messageData.codeInfo || null,
  sender: {
    id: messageData.from || messageData.senderID,
    name: messageData.fromName || messageData.senderName,
    avatar: messageData.fromAvatar || messageData.senderAvatar
  },
  tags: tags || [],
  note: note || ''
}

// 如果是图片或文件消息，添加文件信息到 content
if (messageData.fileInfo) {
  favoriteData.fileInfo = messageData.fileInfo
}

console.log('创建收藏数据:', favoriteData)

const favorite = new Favorite(favoriteData)
await favorite.save()

console.log('收藏保存成功')

res.json({
  success: true,
  message: '收藏成功',
  favorite
})

} catch (err) {
  console.error('添加收藏失败:', err)
```

```
console.error(' 错误堆栈:', err.stack)
res.status(500).json({ message: ' 添加收藏失败', error: err.message })
```

代码片段 2：收藏筛选条件与分页查询

注：第二段补充列表查询逻辑，展示收藏页的消息类型筛选、关键字匹配和分页读取在后端的实现方式。

```
static async getFavorites(req, res) {
  try {
    const userId = req.user.userId
    const { messageType, contentType, search, page = 1, limit = 20 } = req.query

    const query = { userId }

    if (messageType) {
      query.messageType = messageType
    }

    if (contentType) {
      query.contentType = contentType
    }

    // 搜索功能：支持搜索内容、发送者名称、备注
    if (search && search.trim()) {
      const searchRegex = new RegExp(search.trim(), 'i')
      query.$or = [
        { content: searchRegex },
        { 'sender.name': searchRegex },
        { note: searchRegex },
        { 'codeInfo.fileName': searchRegex }
      ]
    }

    const skip = (page - 1) * limit

    const [favorites, total] = await Promise.all([
      Favorite.find(query)
        .sort({ createdAt: -1 })
        .skip(skip)
        .limit(parseInt(limit))
        .lean(),
```

```
Favorite.countDocuments(query)
})
```

附录 G 聊天主界面路由切换核心源码

对应正文：第 5 章第 2 节“Web 前端实现”中“聊天主界面”部分。

源码位置：ccb/src/components/chat/ChatView.vue

注：该片段继续保留到挂载阶段，便于把聊天上下文计算、路由跳转、侧栏折叠和初始化状态处理放在一起说明。

```
// 当前聊天上下文（供 AI 助手使用）
const privateChatContext = computed(() => {
  const userId = route.query.userId
  if (!userId) return null

  // 通过 userId 判断是群聊还是私聊
  if (userId.startsWith('group_')) {
    return {
      chatType: 'group',
      roomId: userId,
      roomName: route.query.uname || '群聊'
    }
  }

  return {
    chatType: 'private',
    targetId: userId,
    targetName: route.query.uname || '好友'
  }
})

// 切换 AI 面板
function toggleAIPanel() {
  showAIPanel.value = !showAIPanel.value
}

// 切换 section2 折叠状态
function toggleSection2() {
  isSection2Collapsed.value = !isSection2Collapsed.value
}

//这里处理 contacts 板块的显示和隐藏
function handlehidechat(message) {
  showlastchats.value = false
}
```

```
}  
function handleshowchat(message) {  
  router.push('/chats')  
}  
  
//这里处理 contacts 板块的显示和隐藏  
function handleshowcontacts(message) {  
  router.push('/contacts')  
}  
  
function handlehidecontacts(message) {  
  showcontacts.value = false  
}  
  
//这里处理 chatrooms 板块的显示和隐藏  
function handleshowchatrooms(message) {  
  router.push('/chatrooms')  
}  
  
function handlehidechatrooms(message) {  
  showchatrooms.value = false  
}  
  
function setcolor(data) {  
  theme.value = data.color  
  container.value.setAttribute('data-theme', theme.value)  
}  
  
//显示聊天内容  
function showdetail({ uname, img, userId, chatType = 'private', groupMembers = [], showDetail = false }) {  
  showcontent.value = true  
  
  // 设置当前聊天用户  
  if (userId) {  
    chatStore.switchChatUser(userId)  
  }  
  
  const query = { uname, img, userId, chatType }  
  
  // 如果是群聊，添加群成员信息  
  if (chatType === 'group' && groupMembers.length > 0) {  
    query.groupMembers = JSON.stringify(groupMembers)  
  }  
}
```

```
// 如果需要显示详情，添加参数
if (showDetail) {
  query.showDetail = 'true'
}

router.push({
  path: '/chatdetail',
  query: query,
})
}

function showAI() {
  // 移动端点击 AI 按钮也打开面板
  showAIPanel.value = true
}

function hidecontent() {
  if (showcontent.value) {
    showcontent.value = false
    // Return to base route when closing content to show dashboard
    router.push('/')
  }
}

function checkScreen() {
  isMobile.value = window.innerWidth <= 768
}

// Check current route to set showcontent state
function checkRoute() {
  const path = router.currentRoute.value.path
  const meta = router.currentRoute.value.meta

  // 根据路由 meta 设置显示状态
  if (meta.showChats) {
    showlastchats.value = true
    showcontacts.value = false
    showchatrooms.value = false
    showcontent.value = false
  } else if (meta.showContacts) {
    showlastchats.value = false
    showcontacts.value = true
```

```
showchatrooms.value = false
showcontent.value = false
} else if (meta.showChatRooms) {
  showlastchats.value = false
  showcontacts.value = false
  showchatrooms.value = true
  showcontent.value = true // 显示 router-view 中的 ChatRoomWelcome
} else if (path.includes('/chatdetail') || path.includes('/chatroom-detail')) {
  showcontent.value = true
  // 自动折叠列表（仅在聊天室详情页）
  if (path.includes('/chatroom-detail')) {
    isSection2Collapsed.value = true
    // 保持聊天室列表显示
    showlastchats.value = false
    showcontacts.value = false
    showchatrooms.value = true
  }
} else if (path === '/' || path === '/chatbox') {
  showcontent.value = false
  // 返回首页时默认显示聊天列表
  showlastchats.value = true
  showcontacts.value = false
  showchatrooms.value = false
}
}
}

onMounted(() => {
```


附录 H 收藏页面筛选与回看核心源码

对应正文：第 5 章第 2 节“Web 前端实现”中的收藏页交互逻辑。

源码位置：ccb/src/views/Favorites.vue

注：这里向前补入标题、预览和时间格式化等辅助函数，使筛选、回看、删除和统计刷新能够串成完整的前端交互流程。

```
function getPreview(favorite) {
  if (favorite.contentType === 'code') {
    return favorite.codeInfo?.fileName || '代码片段'
  }
  if (!favorite.content) {
    return '无内容'
  }
  return favorite.content.substring(0, 50) + (favorite.content.length > 50 ? '...' : '')
}

function formatTimeShort(time) {
  if (!time) return ''
  const date = new Date(time)
  const now = new Date()
  const diff = now - date

  if (diff < 60000) return '刚刚'
  if (diff < 3600000) return `${Math.floor(diff / 60000)} 分钟前`
  if (diff < 86400000) return `${Math.floor(diff / 3600000)} 小时前`
  if (diff < 604800000) return `${Math.floor(diff / 86400000)} 天前`

  return date.toLocaleDateString('zh-CN', { month: '2-digit', day: '2-digit' })
}

async function copyCode(code) {
  try {
    await navigator.clipboard.writeText(code)
    toast.success('代码已复制')
  } catch (err) {
    toast.error('复制失败')
  }
}

function getTypeLabel(type) {
```

```
const map = {
  private: '私聊',
  group: '群聊',
  chatroom: '聊天室'
}
return map[type] || type
}

function getContentLabel(type) {
  const map = {
    text: '文本',
    code: '代码',
    image: '图片',
    file: '文件'
  }
  return map[type] || type
}

function formatTime(time) {
  if (!time) return ''
  const date = new Date(time)
  return date.toLocaleString('zh-CN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    hour: '2-digit',
    minute: '2-digit'
  })
}

async function loadFavorites() {
  try {
    const token = localStorage.getItem('token')
    const params = {}

    if (filterMessageType.value) {
      params.messageType = filterMessageType.value
    }

    if (filterContentType.value) {
      params.contentType = filterContentType.value
    }
  }
}
```

```
if (searchQuery.value.trim()) {
  params.search = searchQuery.value.trim()
}

const res = await axios.get(`${baseUrl}/api/favorites`, {
  headers: { Authorization: `Bearer ${token}` },
  params
})

if (res.data.success) {
  favorites.value = res.data.favorites
}
} catch (err) {
  console.error('加载收藏失败:', err)
  toast.error('加载收藏失败')
}
}

function handleSearch() {
  // 防抖：延迟搜索
  if (searchTimeout.value) {
    clearTimeout(searchTimeout.value)
  }

  searchTimeout.value = setTimeout(() => {
    loadFavorites()
    selectedFavorite.value = null
  }, 500)
}

function clearSearch() {
  searchQuery.value = ''
  loadFavorites()
  selectedFavorite.value = null
}

function getFileUrl(url) {
  if (!url) return ''
  if (url.startsWith('http')) return url
  return baseUrl + url
}

function formatFileSize(bytes) {
```

```
if (!bytes) return '未知大小'
if (bytes < 1024) return bytes + ' B'
if (bytes < 1024 * 1024) return (bytes / 1024).toFixed(2) + ' KB'
return (bytes / (1024 * 1024)).toFixed(2) + ' MB'
}

function previewImage(fileInfo) {
  window.open(getFileUrl(fileInfo.fileUrl), '_blank')
}

function downloadFile(fileInfo) {
  const link = document.createElement('a')
  link.href = getFileUrl(fileInfo.fileUrl)
  link.download = fileInfo.fileName
  link.click()
}

async function loadStats() {
  try {
    const token = localStorage.getItem('token')
    const res = await axios.get(`${baseUrl}/api/favorites/stats`, {
      headers: { Authorization: `Bearer ${token}` }
    })

    if (res.data.success) {
      stats.value = res.data.stats
    }
  } catch (err) {
    console.error('加载统计失败:', err)
  }
}

async function removeFavorite(favorite) {
  try {
    const token = localStorage.getItem('token')
    await axios.delete(`${baseUrl}/api/favorites/${favorite.messageId}`, {
      headers: { Authorization: `Bearer ${token}` }
    })

    toast.success('取消收藏成功')

    // 如果删除的是当前选中的，清空选中
    if (selectedFavorite.value?._id === favorite._id) {
```

```
        selectedFavorite.value = null
    }

    await loadFavorites()
    await loadStats()
  } catch (err) {
    console.error('取消收藏失败:', err)
    toast.error('取消收藏失败')
  }
}

watch([filterMessageType, filterContentType], () => {
  loadFavorites()
  selectedFavorite.value = null
})

onMounted(() => {
  loadFavorites()
  loadStats()
})
```

附录 I 桌面端托盘与消息提醒核心源码

对应正文：第 5 章第 3 节“桌面端实现”中“系统托盘功能”和“本地通知”部分。

源码位置：ccb/main.js

代码片段 1：未读消息提醒与状态清理

注：第一段补齐从渲染进程通知主进程到打开对应聊天并清理未读状态的整条通知链路。

```
// IPC 通信：接收渲染进程的消息
ipcMain.on('new-message', (event, count) => {
  console.log('主进程收到新消息通知, 未读数:', count, '窗口焦点:', mainWindow.isFocused());

  // 临时测试：总是闪烁（不管窗口焦点）
  startFlashing();
  setBadge(count);
});

ipcMain.on('clear-badge', () => {
  console.log('主进程清除 badge');
  clearBadge();
  stopFlashing();
});

// 接收最近消息数据
ipcMain.on('update-recent-messages', (event, messages) => {
  recentMessages = messages;
  if (previewWindow) {
    updatePreviewContent();
  }
});

// 处理从通知打开聊天
ipcMain.on('open-chat-from-notification', (event, data) => {
  console.log('主进程收到打开聊天请求:', data);
  const { userId, type, index } = data;

  // 显示并聚焦主窗口
  if (mainWindow) {
    console.log('显示主窗口');
```

```

mainWindow.show();
mainWindow.focus();

// 发送消息给渲染进程，让它打开对应的聊天
console.log('□ 发送 open-chat 事件到渲染进程:', { userId, type });
mainWindow.webContents.send('open-chat', { userId, type });

// 从列表中移除已读的消息
if (index !== undefined && recentMessages[index]) {
  recentMessages.splice(index, 1);
  // 重新计算未读数
  unreadCount = recentMessages.reduce((sum, msg) => sum + (msg.count || 0), 0);
  setBadge(unreadCount);
  updatePreviewContent();
}

```

代码片段 2：托盘菜单与窗口显隐控制

注：第二段保留托盘菜单和主窗口显隐控制，用来展示桌面端常驻后台时的交互组织方式。

```

// 创建托盘图标函数
function createTray() {
  // 托盘图标路径（建议放在项目的 build 或 assets 目录）
  const iconPath = path.join(__dirname, 'public/images/logo.png');

  // 创建托盘对象
  tray = new Tray(iconPath);

  // 鼠标悬停提示
  tray.setToolTip('Mini Chat Bar');

  tray.displayBalloon({
    icon: iconPath,
    title: '提示',
    content: 'Mini Chat Bar 正在后台运行'
  });

  // 托盘右键菜单
  const contextMenu = Menu.buildFromTemplate([
    {
      label: '打开 Mini Chat Bar',

```

```
click: () => {
  if (mainWindow) {
    mainWindow.show();
    mainWindow.focus();
  }
},
{
  type: 'separator'
},
{
  label: '退出应用',
  click: () => {
    app.quit();
  }
}
]);

// 设置菜单
tray.setContextMenu(contextMenu);

// 点击托盘图标时显示/隐藏主窗口
tray.on('click', () => {
  if (!mainWindow) return;
  if (mainWindow.isVisible()) {
    mainWindow.hide();
  } else {
    mainWindow.show();
    mainWindow.focus();
  }
});
```


附录 J 毕业设计相关表格